



OMAR DEV NOTES · COMPLETE EDITION

مرجع PHP والويب الكامل

كل محتوى الدوكس الحالي في كتاب واحد: 75 درسًا بالعربية والإنجليزية، من رحلة الطلب وأساسيات PHP إلى التشغيل والأمان وقواعد البيانات.

إصدار ٢١ سبتمبر ٢٠٢٦

Arabic + English

6 مسارات مترابطة

فهرس الكتاب

يبدأ كل مسار وكل درس في صفحة مستقلة لتسهيل القراءة والطباعة والمراجعة.

English edition

1. [Programming and web fundamentals](#)— 13 lessons
2. [The PHP language](#)— 16 lessons
3. [PHP runtime and production](#)— 14 lessons
4. [Object-oriented programming](#)— 8 lessons
5. [Security, identity, and authorization](#)— 19 lessons
6. [Databases and PDO](#)— 5 lessons

النسخة العربية

1. أساسيات البرمجة والويب — 13 درسًا
2. لغة PHP — 16 درسًا
3. تشغيل PHP والإنتاج — 14 درسًا
4. البرمجة كائنية التوجه — 8 درسًا
5. الأمان والهوية والصلاحيات — 19 درسًا
6. قواعد البيانات و PDO — 5 درسًا

النسخة العربية

نسخة منظمة للطباعة تشمل كل المسارات والدروس المنشورة في الدوكس.

أساسيات البرمجة والويب

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

أساسيات الويب والشبكات

مسار شامل ومصنّح للصفحات 1-11: من كتابة الرابط حتى تنفيذ PHP وعودة الاستجابة.

هذا القسم يحوّل الصفحات 1-11 من الملاحظات إلى مسار منظم. احتفظنا بكل الأفكار المكتوبة، وصححنا الاختصارات غير الدقيقة، ثم أضفنا أمثلة عملية وتفاصيل تساعدك على فهم ما يحدث قبل أن يبدأ كود PHP.

خريطة الدروس

1. الإنترنت والويب ودورة الطلب — Client/Server والرسم الكامل من المتصفح إلى التطبيق.
2. DNS وعناوين IP — Local Cache و Resolver و Root و TLD و Authoritative DNS.
3. TCP و UDP والحزم — Handshake و ACK والترتيب وإعادة الإرسال والتحكم في التدفق والازدحام.
4. URL والمنافذ و HTTP — أجزاء الرابط، Socket، المنافذ المشهورة، ومعنى HTTP.
5. HTTP Request و Response والحالة — Methods و Headers و Body و Status Codes و Cookies/Sessions/Tokens.
6. HTTPS و TLS والشهادات — التشفير المتماثل وغير المتماثل و Handshake وأنواع التحقق.
7. داخل الخادم: Load Balancer و Nginx و PHP-FPM — الملفات الثابتة والديناميكية و Virtual Hosts والتطبيق وقاعدة البيانات.
8. التفكير البرمجي والخوارزميات — تحليل المشكلة و Pseudocode وبنى البيانات و Big O.
9. كيف يعرض المتصفح الصفحة؟ — DOM و CSSOM و Layout و Paint و DevTools.
10. HTTP Caching والضغط — Cache-Control و ETag و Vary و gzip و Brotli.
11. Same-Origin Policy و Origins — CORS و Preflight و Credentials والإعداد الآمن.
12. الاتصال اللحظي و Polling — Webhooks و SSE و WebSocket و Webhooks.
13. تصميم REST — APIs و RPC و GraphQL والأخطاء والصفحات والإصدارات.



طريقة المذاكرة

ابدأ بالرسم، ثم افتح أدوات المطور في المتصفح وتابع طلبًا حقيقيًا من تبويب Network. اربط كل حقل تراه بالدرس المناسب.

الخريطة المختصرة

```
URL → DNS → IP + Port → TCP/QUIC + TLS → HTTP Request
→ CDN/WAF/Load Balancer → Web Server → PHP → Database
→ HTTP Response → Browser
```

هذه خريطة تعليمية؛ بعض المراحل اختيارية، وقد تختصرها Cache أو وصلة مفتوحة مسبقًا.

1. الإنترنت والويب ودورة الطلب

الفرق بين الإنترنت والويب، نموذج العميل والخادم، والرسم الكامل لرحلة الطلب والاستجابة.

الإنترنت ليس هو الويب

- الإنترنت: بنية عالمية من الشبكات والأجهزة والراوترات والبروتوكولات.
- الويب: خدمة فوق الإنترنت تستخدم HTTP/HTTPS للوصول إلى موارد عبر URLs.
- البريد الإلكتروني ونقل الملفات والمكالمات والألعاب خدمات أخرى تستخدم الإنترنت.

نموذج Client/Server

الـ **Client** برنامج يطلب خدمة: متصفح، تطبيق هاتف، أو أداة مثل `curl`. والـ **Server** دور يستقبل الطلبات ويعالجها؛ قد يعني برنامجًا أو جهازًا. أثناء التطوير يمكن أن يعمل الطرفان على جهازك نفسه.

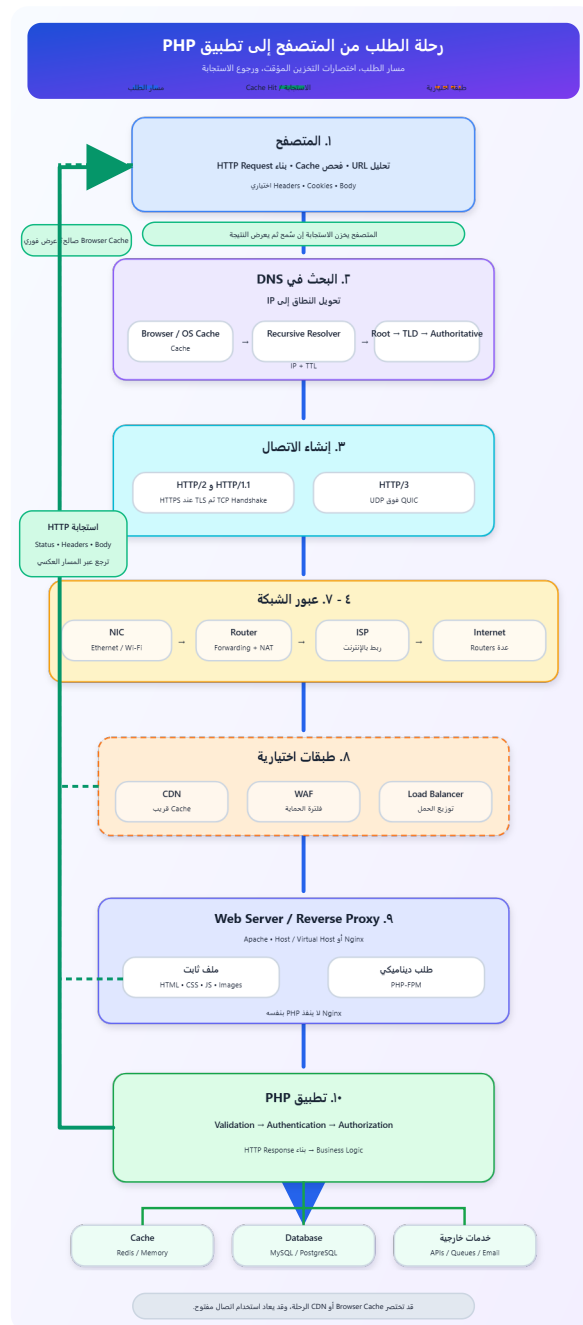
```

المتصفح → المستخدم → HTTP Request → الخادم
                        ↓
                        PHP تطبيق ← Database
                        ↓
HTTP Response ← المستخدم ← عرض النتيجة
  
```

ماذا يحدث عند فتح رابط؟

عند فتح `https://shop.example/products/42?currency=EGP#reviews`:

1. يحلل المتصفح URL ويبحث في الـ Cache والسياسات المحلية.
2. يبحث DNS عن عنوان IP إذا لم يكن مخزنًا.
3. يختار نظام التشغيل الواجهة والمسار، وترسل NIC البيانات عبر Wi-Fi أو Ethernet.
4. تمر الحزم عبر الراوتر وISP وعدة شبكات.
5. يُنشأ اتصال TCP HTTP/1.1 أو HTTP/2، أو QUIC فوق UDP HTTP/3.
6. ينفذ TLS Handshake عند HTTPS.
7. يرسل المتصفح HTTP Request.
8. قد يستقبله CDN أو WAF أو Load Balancer قبل الخادم الأصلي.
9. يعيد Web Server ملفًا ثابتًا أو يمرر الطلب الديناميكي إلى PHP-FPM.
10. يتحقق التطبيق من المدخلات والهوية والصلاحيات، وقد يستخدم Cache أو Database.
11. تعود HTTP Response بالحالة والرؤوس والجسم، ثم يفسرها المتصفح ويعرضها.



افتح الرسم بالحجم الكامل



ملاحظة

الرسم شامل وليس مسارًا إلزاميًا. قد يعيد Browser Cache أو CDN النتيجة دون الوصول إلى PHP، وقد لا يملك التطبيق Load Balancer أو قاعدة بيانات أصلاً.

مثال يمكن مشاهدته

```
curl -i "https://example.com/"
```

الخيار `-i` يعرض Response Headers مع الجسم. ولرؤية خطوات DNS والاتصال وTLS:

```
curl -v "https://example.com/" -o NUL
```

على Linux/macOS استبدل `NUL` بـ `/dev/null`.

من المسؤول عن ماذا؟

المكوّن	مسؤوليته الأساسية
Browser	بناء الطلب، إدارة Cache/Cookies، تفسير الاستجابة
DNS	تحويل الاسم إلى عنوان IP
TCP/QUIC	نقل البيانات وخصائص الاتصال
TLS	السرية والسلامة والتحقق من هوية الخادم
HTTP	معنى الطلب والاستجابة
Web Server	استقبال HTTP والملفات الثابتة وتمرير الديناميكي
PHP	منطق التطبيق وإنشاء الاستجابة
Database	حفظ البيانات واسترجاعها

فهم الحدود يمنع أخطاء مثل القول إن DNS «يفتح الصفحة»، أو أن NIC «تنفذ HTTP»، أو أن Nginx «ينفذ PHP» بنفسه.

2. DNS وعناوين IP

كيف يتحول اسم النطاق إلى IP عبر Cache و Resolver و Root و TLD و Authoritative DNS.

عنوان IP

عنوان IP يعرّف واجهة على شبكة ويتيح توجيه الحزم إليها. أمثلة:

- عنوان IPv4 محلي خاص، و `203.0.113.10` مثال توثيقي.
- عنوان IPv6: `2001:db8::10` مثال توثيقي.
- `127.0.0.1` و Loopback `::1`، أي جهازك نفسه.
- `localhost` اسم يشير عادة إلى Loopback.

العنوان الخاص داخل منزلك ليس بالضرورة العنوان العام على الإنترنت. غالبًا يجري الراوتر NAT بينهما.

لماذا DNS؟

البشر يتذكرون `example.com` أسهل من IP، ومالك الخدمة قد يغيّر الخوادم دون تغيير الاسم. DNS نظام موزع يعيد records، وليس مجرد «جدول اسم وIP».

أشهر السجلات:

Record	الوظيفة
A	اسم إلى IPv4
AAAA	اسم إلى IPv6
CNAME	اسم مستعار لاسم آخر
MX	خوادم البريد
TXT	نصوص تحقق وسياسات
NS	الخوادم المسؤولة عن zone

رحلة DNS المكتوبة في الورق

1. يفحص Browser Cache.
2. يفحص نظام التشغيل وملف hosts والـ OS Cache.
3. يسأل Recursive Resolver، وغالبًا يوفره ISP أو خدمة عامة.

4. إذا لم تكن الإجابة مخزنة، يسأل resolver خادم Root.
5. يشير Root إلى خوادم TLD مثل `.com`.
6. يشير TLD إلى Authoritative Name Server.
7. يعيد Authoritative Server السجل النهائي.
8. تُخزن النتيجة حسب TTL ثم تعود إلى المتصفح.

```
Browser/OS Cache
  ↓ miss
Recursive Resolver
  ↓
Root → TLD → Authoritative
      ↑       |
      └── IP + TTL
```



تصحيح

المتصفح لا يسأل Root وTLD بنفسه عادة، ولا تتكرر الرحلة الكاملة لكل طلب. الـ Resolver يؤديها عند غياب الإجابة من Cache.

TTL وأكثر من عنوان

TTL مدة تسمح بتخزين الإجابة قبل إعادة الاستعلام. وقد يعيد الاسم عدة عناوين لأسباب مثل توزيع الحمل أو القرب الجغرافي. DNS يحدد عنوانًا محتملًا؛ لا ينقل صفحة HTML ولا يشغل PHP.

تجربة عملية

```
nslookup example.com
nslookup -type=AAAA example.com
nslookup -type=MX example.com
```

ولفهم اختلاف طبقات الـ Cache، غيّر السجل في بيئة تجريبية ولاحظ أن الانتشار ليس «زّرا فورّيّا»؛ كل مخزن ينتهي وفق TTL وسياساته.

3. TCP و UDP والحزم

المصافحة، المصافحة، Frames، ACK و Packets وإعادة الإرسال والتحكم في التدفق والازدحام.

من البيانات إلى السلك

تُغلف البيانات أثناء نزولها خلال طبقات الشبكة:

```

HTTP Data
↓
TCP Segment أو UDP Datagram
↓
IP Packet
↓
Ethernet/Wi-Fi Frame
↓
Bits/Signals

```

كل طبقة تضيف Header. من المعلومات الممكنة: Source/Destination Ports في طبقة النقل، Source/Destination IP في طبقة الشبكة، Sequence Number و Flags في TCP، و TTL/Hop Limit في IP. لا توجد «Packet واحدة» دائمًا؛ المحتوى الكبير يُقسّم ويعاد تجميعه.

الـ **NIC** أو Network Interface Card تتعامل مع الاتصال المحلي و Frames والإشارات. الراوتر يختار القفزة التالية للحزمة، والـ ISP يربط شبكتك بالشبكات الأخرى.

TCP: Transmission Control Protocol

TCP يقدم تدفق **bytes** موثوقًا ومرتبًا بين نقطتين. يبدأ عادة بمصافحة ثلاثية:

```

Client                                Server
| ----- SYN -----> |
| <----- SYN + ACK ----- |
| ----- ACK -----> |
| ===== Application Data =====> |

```

- **SYN**: طلب بدء ومزامنة Sequence Numbers.
- **SYN-ACK**: قبول ومزامنة الطرف الثاني.
- **ACK**: تأكيد الاستلام.
- الاتصال يُعرّف بأربع قيم تقريبًا: source IP/port و destination IP/port.

الاعتمادية والترتيب

يرفّم TCP الـ bytes/segments، ويؤكد ما وصل باستخدام ACK. إذا لم يصل التأكيد في الوقت المتوقع أو استنتج المرسل فقدًا، يعيد الإرسال. يمكن أن تصل IP Packets بترتيب مختلف، بينما يعرض TCP للتطبيق stream مرتبًا.

```
Send 1, 2, 3, 4
Receive 1, 2, 4
ACK indicates missing data
Retransmit 3
Application reads 1, 2, 3, 4
```



ملاحظة

ACK لا يعني أن تطبيق PHP أنهى العمل؛ يعني أن طبقة TCP استلمت البيانات المطلوبة.

Congestion Control و Flow Control

- **Flow control:** يمنع المرسل من إغراق جهاز الاستقبال. يعلن المستقبل Receive Window عن المساحة المتاحة.
 - **Congestion control:** يمنع إغراق الشبكة. يحتفظ المرسل بـ Congestion Window ويزيد السرعة تدريجيًا ويخفضها عند دلائل الفقد أو الازدحام.
 - حد الإرسال الفعلي يتأثر بالأصغر بين receive window و congestion window.
- إذن فقد Packet لا يعالج فقط بإعادة الإرسال؛ يؤثر أيضًا في معدل الإرسال.

UDP: User Datagram Protocol

UDP يرسل Datagrams بلا اتصال تقليدي ولا يضمن بنفسه الوصول أو الترتيب أو منع التكرار. لذلك Header أبسط والحمل الإضافي أقل.

الخاصية	TCP	UDP
Handshake تقليدي	نعم	لا
ضمان الترتيب وإعادة الإرسال	يوفرهما	لا يوفرهما
شكل البيانات	Byte stream	Datagrams مستقلة
أمثلة	HTTP/1.1، HTTP/2، بريد، ملفات	DNS، صوت مباشر، ألعاب، بث
السرعة	ليست ضمانًا؛ حمل أكبر	ليس دائمًا أسرع؛ حمل أقل

HTTP/3 يعمل فوق **QUIC المبنى على UDP**، لكن QUIC نفسه يضيف الاعتمادية والتشفير والتحكم؛ لذلك «UDP غير موثوق» لا يعني أن التطبيق المبنى فوقه يجب أن يظل غير موثوق.

تدريب

جرب:

```
ping example.com
tracert example.com
```

على Linux/macOS استخدم `ping` . `traceroute` يستخدم ICMP لا TCP، لكنه يوضح زمن الذهاب والعودة والفقد التقريبي.

4. URL والمنافذ وHTTP

تشرح الرابط، معنى Port وSocket، نطاقات المنافذ، وبروتوكول HTTP.

HTTP

HTTP = Hypertext Transfer Protocol، وهو بروتوكول طبقة التطبيق الذي يحدد طريقة تبادل رسائل request وresponse حول HTTP resources. ليس اتصال الشبكة نفسه؛ يعمل فوق وسائل نقل مثل TCP أو QUIC.

HTTP في أصله عديم الحالة: كل request رسالة مستقلة. استمرار تسجيل الدخول أو السلة يحتاج آلية إضافية مثل Cookies/Sessions/Tokens.

تشرح URL

```
https://user:pass@example.com:443/products/42?currency=EGP#reviews
```

scheme
host
port
path
query
fragment

الجزء	المعنى
Scheme	البروتوكول/طريقة الوصول مثل <code>http</code> و <code>https</code>
User info	موجود نحوياً في بعض URIs لكنه غير مناسب لكلمات المرور
Hostname	اسم المضيف مثل <code>example.com</code>
Port	الخدمة على الجهاز؛ يُحذف إذا استُخدم الافتراضي
Path	المورد داخل الموقع
Query	أزواج إضافية بعد <code>?</code>
Fragment	موضع محلي بعد <code>#</code> ولا يُرسل عادة إلى الخادم



تحذير

لا تضع كلمة مرور أو token أو بيانات حساسة في URL. قد تظهر في Browser History وLogs وReferer Analytics.

ما المنفذ؟

عنوان IP يحدد الواجهة/الجهاز على الشبكة، والمنفذ يحدد خدمة منطقية داخل نظام التشغيل. رقم المنفذ bit-16 من 0 إلى 65535.

- System/Well-Known Ports : 0-1023 .
- User/Registered Ports : 1024-49151 .
- Dynamic/Private Ports : 49152-65535 وفق نطاق IANA؛ قد تختلف نطاقات المنافذ المؤقتة فعليًا بين الأنظمة.

المنفذ الشائع	الخدمة
21	FTP control
22	SSH
25	SMTP
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL

المنفذ الشائع convention/configuration وليس حماية. يمكن تشغيل HTTP محليًا على 8000 أو 8080 .

Connection و Socket

Socket endpoint يمكن تبسيطه إلى IP + port + protocol. اتصال TCP يُميز غالبًا بهذه الرباعية:

```
(client IP, client port, server IP, server port)
192.0.2.5:53014 → 203.0.113.8:443
```

يستخدم العميل عادةً منفذًا مؤقتًا، بينما يستمع الخادم على منفذ معروف. لذلك يمكن لآلاف العملاء الاتصال بالمنفذ 443 نفسه دون أن تختلط اتصالاتهم.

مثال PHP محلي

```
php -S localhost:8000
```

في `http://localhost:8000/hello.php` :

- `scheme = http`
- `host = localhost`
- `port = 8000`
- `path = /hello.php`

عدم كتابة المنفذ في HTTPS يعني عادة 443، وفي HTTP يعني 80؛ لا يعني عدم وجود منفذ.

5. HTTP Request و Response والحالة

بنية رسائل HTTP و Methods و Headers و Body و Status Codes و Cookies و Sessions و Tokens.

Request

في HTTP/1.1 النصي يتكون الطلب من start line و headers و سطر فارغ ثم body اختياري:

```
POST /api/orders?notify=1 HTTP/1.1
Host: shop.example
Accept: application/json
Content-Type: application/json
Authorization: Bearer ey...
Cookie: session=abc123
Content-Length: 29

{"product_id":42,"count":2}
```

- **Method**: نية الطلب.
- **Target/Path + Query**: المورد والمعاملات.
- **Version**: نسخة بروتوكول الرسالة.
- **Headers: metadata** والتحكم.
- **Body**: المحتوى، وليس موجودًا في كل طلب.

HTTP/2 و HTTP/3 يشقّران الرسائل بصيغة ثنائية وليست الأسطر النصية نفسها، لكن المعاني تبقى methods/fields/content.

Methods

Method	الدلالة المعتادة	؟Safe	؟Idempotent
GET	جلب representation	نعم	نعم
HEAD	مثل GET دون response content	نعم	نعم
POST	معالجة خاصة بالمورد/إنشاء شائعًا	لا	لا غالبًا
PUT	استبدال الحالة الكاملة	لا	نعم
PATCH	تعديل جزئي	لا	يعتمد على التصميم
DELETE	إزالة المورد	لا	نعم دلاليًا
OPTIONS	إمكانات الاتصال	نعم	نعم

Safe و Idempotent دلالات يجب أن يحترمها التطبيق، وليستا آلية صلاحيات. لا يجوز تنفيذ حذف حقيقي عبر GET، وعلى الخادم التحقق من الهوية والصلاحيات في كل Method.

Headers شائعة

- **Host**: اسم الموقع المطلوب، مهم مع Virtual Hosts.
- **Accept**: أنواع الاستجابة المقبولة.
- **Content-Type**: نوع body المرسل.
- **Authorization**: بيانات اعتماد، مثل Bearer token.
- **Cookie**: cookies المطابقة.
- **User-Agent**: معلومات العميل.
- **Cache-Control**: سياسة Cache.

Response

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/orders/901
Cache-Control: no-store

{"id":901,"status":"pending"}
```

تتكون من protocol version و status code و reason phrase اختياري في HTTP/1.1، ثم headers و body اختياري.

الفئة	المعنى	أمثلة
1xx	معلومات مؤقتة	100 Continue
2xx	نجاح	200 OK , 201 Created , 204 No Content
3xx	Redirect/Cache	301 , 302 , 304
4xx	مشكلة في الطلب	400 , 401 , 403 , 404 , 422
5xx	فشل جهة الخادم	500 , 502 , 503

401 يعني غالبًا أن المصادقة مطلوبة أو غير صالحة، و 403 أن الطلب مفهوم لكن غير مسموح. لا تستخدم 200 لكل شيء؛ الحالة جزء من عقد API.

Stateless لا يعني بلا حالة في التطبيق

HTTP لا يتذكر تلقائيًا الطلب السابق. التطبيق يبني الاستمرارية بواسطة:

- **Cookie**: قيمة يخزنها المتصفح ويرسلها حسب domain/path/security rules.
- **Session**: بيانات server-side مرتبطة غالبًا بمعرف في Cookie.
- **Token: credential** يرسله العميل، شائع في APIs. ليس كل JWT token، وJWT ليس Session بديلًا تلقائيًا.

مثال PHP

```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'GET') {
    http_response_code(405);
    header('Allow: GET');
    echo json_encode(['error' => 'Method not allowed']);
    exit;
}

$name = trim((string) ($_GET['name'] ?? 'زائر'));

echo json_encode(
    ['message' => "مرحبًا {$name}"],
    JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR
);
```

جرب `curl -i "http://localhost:8000/hello.php?name=Omar"`، ثم افحص الطلب من Network في DevTools.

6. HTTPS و TLS والشهادات

ما الذي يحميه HTTPS، كيف يعمل TLS Handshake، المفاتيح، Session Keys، والشهادات DV و OV و EV.

HTTPS

HTTPS = HTTP over TLS. لا يغيّر دلالات GET و POST أو status codes، لكنه ينقل رسائل HTTP داخل قناة محمية.

يوفر TLS ثلاث خصائص رئيسية:

1. **Confidentiality**: منع قراءة المحتوى في الطريق.

2. **Integrity**: كشف تعديل البيانات.

3. **Authentication**: التحقق من أن الخادم يملك الهوية/المفتاح المرتبط بالشهادة.

HTTPS لا يصلح SQL Injection أو XSS أو ضعف الصلاحيات، ولا يخفي بالضرورة عنوان IP أو اسم النطاق عن كل طبقات الشبكة.

التشفير المتماثل وغير المتماثل

- **Asymmetric cryptography**: زوج Public/Private Key مناسب للتوقيع والاتفاق الآمن على الأسرار، لكنه أعلى كلفة.
- **Symmetric cryptography**: مفتاح مشترك مثل Session Key لتشفير بيانات الاتصال بسرعة، مثل AES-GCM أو ChaCha20-Poly1305.

الملاحظات تشرح أن العميل ينشئ Session Key ويشفره بـ Public Key. هذا يصف RSA key exchange القديم بصورة مبسطة. في TLS 1.3 يجري عادة اتفاق **مفاتيح مؤقتة ECDHE**؛ لا يُرسل Session Key النهائي نفسه، بل يشتق الطرفان مفاتيح متماثلة من السر المشترك وبيانات المصافحة. هذا يوفر Forward Secrecy.

TLS 1.3 Handshake مبسط

```
ClientHello
- supported TLS versions
- cipher suites
- key share
- server name
↓
ServerHello
- chosen parameters
- server key share
- certificate + signature
↓
Client validates certificate and signature
Both derive traffic keys
↓
Encrypted HTTP data
```

قد تُستأنف جلسة سابقة فتقل الخطوات. ومع QUIC/HTTP/3 يندمج TLS 1.3 داخل بروتوكول QUIC.

ما الشهادة الرقمية؟

تربط الشهادة Public Key باسم نطاق، وتوقعها Certificate Authority موثوقة أو سلسلة تنتهي في Root CA موجودة في Trust Store.

يفحص العميل عادة:

- اسم النطاق في SAN.
- مدة الصلاحية.
- سلسلة التوقيع حتى Root موثوق.
- Key Usage/Extended Key Usage.
- حالة الإلغاء عند دعم آليته.
- توقيع الخادم الذي يثبت امتلاك Private Key.

امتلاك الشهادة لا يعني أن الموقع «آمن من كل شيء»؛ يعني أن الاتصال بهوية النطاق المطلوبة موثق ومشفر ضمن حدود TLS.

مستويات التحقق

النوع	ما تتحقق منه CA
DV	السيطرة على النطاق
OV	النطاق وهوية المؤسسة وفق إجراءات CA
EV	تحقق مؤسسي أوسع وفق سياسة EV



تصحيح

OV وEV لا يعنيان خوارزمية تشفير أقوى من DV. قوة القناة تعتمد على TLS والإعدادات؛ الفرق في مقدار التحقق من هوية الجهة، كما أن المتصفحات الحديثة لا تعرض EV بالطريقة البارزة القديمة.

أخطاء شائعة

- تجاهل تحذير certificate في الإنتاج.
- استخدام http:// لإرسال كلمة مرور.
- تعطيل TLS verification في عميل API.

- حفظ Private Key داخل Git.
- الاعتقاد أن Base64 تشفير؛ هو encoding فقط.

تجربة

افتح Security → DevTools أو نفذ:

```
curl -v https://example.com/ -o NUL
```

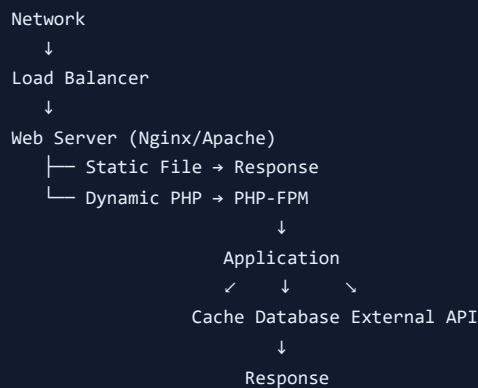
راقب نسخة TLS والشهادة وALPN الذي قد يختار HTTP/2. لا تطبع أسرارًا أو tokens أثناء التصحيح.

7. داخل الخادم: Load Balancer و Nginx و PHP-FPM

رحلة الطلب داخل البنية الخلفية من الشبكة إلى الخادم والتطبيق وقاعدة البيانات ثم الاستجابة.

من الإنترنت إلى التطبيق

الجزء الأخير من الرسم المكتوب في الصفحتين 10 و 11 هو:



كل طبقة اختيارية ويمكن دمج عدة أدوار على جهاز واحد في مشروع صغير.

Load Balancer

يوزع الطلبات على أكثر من backend لرفع التوافر والسعة. قد يعمل على Layer 4 باستخدام TCP/UDP، أو Layer 7 ويفهم HTTP.

وظائف محتملة:

- Health checks واستبعاد الخادم غير السليم.
- TLS termination.
- توزيع round-robin أو least-connections أو خوارزمية أخرى.
- Sticky sessions عند الضرورة، مع أن shared session store غالبًا أسهل للتوسع.
- إضافة forwarding headers.



تنبيه

لا تثق في **X-Forwarded-For** من أي عميل مباشرة. اضبط قائمة trusted proxies، وإلا يمكن تزوير عنوان العميل.

أمثلة: Nginx/HAProxy أو خدمات سحابية مثل AWS Elastic Load Balancing. والـ Load Balancer ليس بديلًا لقاعدة البيانات أو كود التطبيق.

Virtual Host و Web Server

يستقبل Nginx أو Apache الاتصال ويختار الموقع باستخدام IP/port واسم **Host** أو SNI/TLS configuration.

- **Static file**: صورة أو CSS أو JS يمكن إعادته مباشرة، مع Cache وCompression.
- **Dynamic PHP**: يُمرر إلى PHP runtime، غالبًا PHP-FPM عبر FastCGI.
- **Reverse proxy**: قد يمرر الطلب إلى خدمة أخرى.



تصحيح

Nginx لا ينفذ PHP داخل عملياته. يمرر الطلب إلى PHP-FPM ثم يستقبل الناتج. Apache يمكن أن يستخدم PHP-FPM أيضًا أو إعدادات أخرى.

PHP-FPM والتطبيق

يدير PHP-FPM مجموعة workers. يستقبل FastCGI parameters والطلب، يشغل entry point مثل **public/index.php**، ثم يعيد headers/body ل خادم الويب.

التطبيق بدوره:

1. يطابق Route.
2. يشغل Middleware.
3. يتحقق من المدخلات والمصادقة والصلاحيات.
4. ينفذ business logic.
5. يستخدم Database/Cache/Queue/API عند الحاجة.
6. يبني Response.


```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');

try {
    $product = ['id' => 42, 'name' => 'Keyboard'];
    echo json_encode($product, JSON_THROW_ON_ERROR);
} catch (Throwable $e) {
    error_log($e);
    http_response_code(500);
    echo json_encode(['error' => 'Internal server error']);
}
```

لا ترسل stack trace أو كلمات مرور قاعدة البيانات للمستخدم.

رجوع الاستجابة

تعود النتيجة إلى Web Server ثم Load Balancer/CDN ثم الشبكة والمتصفح. قد تضيف طبقات وسيطة headers أو compression أو caching.

راقب:

- **Content-Type** الصحيح.
- Status code الصحيح.
- **Cache-Control** للبيانات الخاصة والعامة.
- Security headers الملائمة.
- Request ID لتتبع نفس الطلب في logs عبر الخدمات.
- زمن كل طبقة، لا زمن PHP فقط.

حالات مهمة

Cache hit

Browser/CDN Cache → response
PHP and database are skipped

Static file

Nginx → /public/app.css → response
PHP is skipped

Nginx → PHP-FPM → App → Database → response

فشل backend

قد يعيد proxy حالة **502 Bad Gateway** عندما لا يحصل على استجابة صحيحة من upstream، أو **504 Gateway Timeout** عند انتهاء المهلة. أما **500** فعادة خطأ داخل التطبيق/الخادم الذي عالج الطلب.

تدريب نهائي

1. افتح Network tab وسجل method و status و content type.
2. أعد تحميل الصفحة ولاحظ **304** أو **from memory cache**.
3. أضف ملف CSS ثابتًا و endpoint PHP ديناميكيًا وقارن.
4. أرسل request ID في header وسجله داخل PHP.
5. ارسم مسارك الفعلي وحدد الطبقات غير الموجودة بدل نسخ المخطط حرفيًا.

8. التفكير البرمجي والخوارزميات

تحويل المشكلة إلى خطوات، كتابة Pseudocode، اختيار بنية البيانات، وفهم التعقيد بصورة عملية.

من المشكلة إلى برنامج

لا تبدأ بالكود. اكتب أولًا:

1. المدخلات: ما البيانات القادمة؟ وما حدودها؟
2. المخرجات: ما النتيجة المطلوبة بدقة؟
3. القواعد: ما الحالات العادية والاستثنائية؟
4. التقسيم: حوّل المشكلة إلى خطوات صغيرة قابلة للاختبار.
5. التحقق: جرّب مثالًا عاديًا، حدًا أدنى، حدًا أقصى، ومدخلًا غير صالح.

مثال: حساب إجمالي سلة بعد خصم.

```
INPUT items, discountPercent
IF discountPercent < 0 OR discountPercent > 100
    RETURN error
total = 0
FOR EACH item IN items
    IF item.price < 0 OR item.quantity < 1
        RETURN error
    total = total + item.price * item.quantity
RETURN total * (1 - discountPercent / 100)
```

الـ Pseudocode يشرح المنطق بلا ارتباط بصياغة لغة، ثم يمكن تحويله إلى PHP أو JavaScript أو غيرهما.

اختيار بنية البيانات

الحاجة	بنية مناسبة
ترتيب عناصر والسماح بالتكرار	List/Array
الوصول بقيمة مفتاح	Map/Associative Array
منع التكرار	Set
أول داخل أول خارج	Queue
آخر داخل أول خارج	Stack

اختيار البنية يؤثر في وضوح الحل وكلفته. لا تستخدم nested loops تلقائيًا إذا كان Map يستطيع تحويل البحث المتكرر إلى lookup مباشر.

التعقيد Big O

Big O يصف نمو الزمن أو الذاكرة مع نمو حجم الإدخال، وليس عدد الثواني الفعلي.

التعقيد	مثال
$O(1)$	قراءة عنصر معروف المفتاح
$O(\log n)$	Binary search في بيانات مرتبة
$O(n)$	المرور مرة على القائمة
$O(n \log n)$	خوارزميات فرز فعالة شائعة
$O(n^2)$	مقارنة كل عنصر بكل عنصر

الألوية للصحة والوضوح، ثم القياس قبل التحسين. خوارزمية $O(n)$ قد تكون أبطأ على بيانات صغيرة من حل أبسط بسبب الثوابت.

تمرين

لديك قائمة طلبات، وكل طلب يحوي `customer_id` و `total`. صمّم خوارزمية تعيد إجمالي كل عميل:

- اكتب المدخل والمخرج.
- عالج القائمة الفارغة والقيم السالبة.
- استخدم Map حتى يكون المرور $O(n)$.
- اكتب ثلاث حالات اختبار قبل كتابة الكود.

9. كيف يعرض المتصفح الصفحة؟

Navigation و DOM و CSSOM و Render Tree وتنفيذ JavaScript واستخدام DevTools لفهم الأداء.

من الاستجابة إلى Pixels

```
HTML bytes -> decode -> tokens -> DOM
CSS bytes -> parse -> CSSOM
DOM + CSSOM -> Render Tree -> Layout -> Paint -> Composite
```

- **DOM**: شجرة العناصر والمحتوى.
- **CSSOM**: القواعد المحللة التي تحدد الشكل.
- **Layout**: حساب الأحجام والمواقع.
- **Paint**: رسم النصوص والألوان والحدود.
- **Composite**: تركيب الطبقات لإظهار الإطار النهائي.

CSS اللازمة للعرض قد تؤخر الرسم، وJavaScript التقليدية قد توقف تحليل HTML حتى تُحمّل وتنفّذ.

JavaScript والتحميل

```
<script src="/app.js" defer></script>
<script type="module" src="/main.js"></script>
```

defer يسمح باستمرار parsing وينفذ بعد اكتمال DOM وبالترتيب. modules مؤجلة افتراضياً ولها dependency graph. أما **async** فينفذ عند اكتمال التحميل دون ضمان ترتيب.

تغيير layout بصورة متكررة داخل loop قد يسبب layout thrashing. اجمع القراءات ثم الكتابات، وحرك العناصر غالباً عبر **transform** عندما يناسب.

دورة حدث مبسطة

JavaScript يعمل عادة على main thread مع event loop. المهام الطويلة تؤخر input والرسم. قس Largest Contentful Paint و Cumulative Layout Shift و Interaction to Next Paint.

DevTools

1. افتح **Network** وفعل Disable cache أثناء التجربة.
2. راقب DNS و Connection و TLS و Waiting/TTFB و Download.

3. افحص headers والحجم المنقول ونوع البروتوكول.

4. استخدم **Performance** لتحديد long tasks و layout و paint.

5. استخدم **Elements** لرؤية DOM والcomputed styles.



نصيحة

TTFB مرتفع قد يكون شبكة أو CDN أو PHP أو Database. اربط Network trace مع server logs و profiling قبل تحديد السبب.

10. HTTP Caching والضغط

Cache-Control و ETag و Vary و CDN و ضغط gzip و Brotli.

لماذا نخزن الاستجابة؟

cache تقلل latency واستهلاك الشبكة والحمل على PHP وقاعدة البيانات. قد توجد نسخ في المتصفح أو proxy/CDN أو التطبيق، ولكل طبقة سياسة مستقلة.

Validation و Freshness

```
Cache-Control: public, max-age=60, s-maxage=300
ETag: "product-42-v7"
Vary: Accept-Encoding, Accept-Language
```

- `max-age` مدة freshness للعميل.
- `s-maxage` يمكن أن يخصص shared caches.
- `private` يسمح بالتخزين الخاص ولا يسمح عادة لـ CDN بالمشاركة.
- `no-store` يطلب عدم التخزين.
- `no-cache` لا يعني عدم التخزين؛ يعني إعادة التحقق قبل الاستخدام.
- `Vary` يجعل مفتاح النسخة يعتمد على headers محددة.

بعد انتهاء freshness يرسل العميل `If-None-Match`. إذا لم يتغير المورد يعيد الخادم `304 Not Modified` بلا body. ويمكن استخدام `Last-Modified` و `If-Modified-Since` عندما يناسب.

مثال PHP

```
$etag = '' . hash('sha256', $json) . '';
header('Cache-Control: public, max-age=60');
header("ETag: {$etag}");

if (trim($_SERVER['HTTP_IF_NONE_MATCH'] ?? '') === $etag) {
    http_response_code(304);
    exit;
}

header('Content-Type: application/json; charset=utf-8');
echo $json;
```

لا تجعل بيانات مستخدم خاصة `public`، ولا تعتمد على query string عشوائي كسياسة invalidation.

gzip وBrotli يقللان الحجم النصي مثل HTML وCSS وJSON. طَبَّق الضغط غالبًا في Web Server/CDN، ولا تضغط صورًا مضغوطة أصلاً بلا قياس. يجب أن تراعي cache قيمة **Accept-Encoding** عادة عبر **Vary**.

Checklist

- حدّد من يستطيع التخزين ومدة الصلاحية.
- استخدم fingerprinted filenames للأصول الثابتة طويلة العمر.
- اختبر invalidation قبل مدة طويلة.
- لا تخزن استجابة فيها **Set-Cookie** أو بيانات شخصية بصورة عامة.
- راقب hit ratio والحجم والزمن.

مرجع

- RFC 9111: HTTP Caching

CORS و Same-Origin Policy 11

تعريف Origin وقيود المتصفح و Simple Requests و Preflight و Credentials والإعداد الآمن.

ما Origin؟

الorigin يتكوّن من **scheme + host + port**:

```
https://app.example:443
```

ويختلف عن `http://app.example` أو `https://api.example` أو `https://app.example:8443`.

Same-Origin Policy تمنع JavaScript من قراءة كثير من موارد origin أخرى بلا سماح. لا تمنع إرسال كل الطلبات، ولا تحمي server-to-server clients.

CORS

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
Vary: Origin
```

لا يمكن جمع credentials مع * wildcard. طابق origin مع allow-list صريحة، ولا تعكس أي قيمة مرسله بلا تحقق.

Preflight

```
OPTIONS /api/orders HTTP/1.1
Origin: https://app.example
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, Authorization
```

يرد الخادم بالـ methods والـ headers المسموحة ومدة حفظ القرار. تعامل مع **OPTIONS** قبل auth middleware التي تتطلب credential غير موجودة في preflight.

```
$allowed = ['https://app.example', 'https://admin.example'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';

if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: {$origin}");
    header('Vary: Origin');
    header('Access-Control-Allow-Credentials: true');
}

if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    header('Access-Control-Allow-Methods: GET, POST');
    header('Access-Control-Allow-Headers: Content-Type, Authorization');
    http_response_code(204);
    exit;
}
```



تحذير

CORS ليست Authentication أو Authorization وليست بديلاً عن CSRF protection. عميل غير متصفح يستطيع تجاهلها، والخادم يجب أن يتحقق من كل طلب.

12. الاتصال اللحظي وWebhooks

Polling و Long Polling و SSE و WebSocket و Webhooks وكيف تختار النمط المناسب.

ليست كل التحديثات Request/Response تقليدية

النمط	الاتجاه	مناسب لـ
Polling	العميل يسأل دوريًا	تحديثات قليلة وحل بسيط
Long Polling	طلب ينتظر حدثًا	توافق واسع مع latency أقل
SSE	Server → Browser	إشعارات و feeds نصية
WebSocket	اتجاهان مستمران	دردشة وألعاب وتعاون لحظي
Webhook	Server → Server	إشعار نظام خارجي بحدث

اختيار WebSocket لمجرد أنه "لحظي" يضيف إدارة connections و heartbeat و backpressure وتوسّع أفقي. ابدأ بأبسط نمط يحقق المطلوب.

Server-Sent Events

```
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
header('X-Accel-Buffering: no');

echo "event: order.updated\n";
echo 'data: ' . json_encode(['id' => 42, 'status' => 'paid']) . "\n\n";
flush();
```

SSE يعيد الاتصال تلقائيًا في المتصفح ويمكن استخدام `id` و `Last-Event-ID` للاستئناف. لا تشغل FPM workers بلا حدود؛ راجع بنية الخادم والمهلات قبل streams طويلة.

Webhooks

المستقبل يجب أن:

1. يقرأ raw body.
2. يتحقق من توقيع HMAC و timestamp قبل parsing الموثوق.
3. يرفض replay خارج نافذة زمنية.
4. يسجل event ID ويمنع المعالجة المكررة.

5. يعيد نجاحًا سريعًا ثم ينقل العمل الثقيل إلى queue.

```
$expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
if (!hash_equals($expected, $signature)) {
    http_response_code(401);
    exit;
}
```

المُرسل يعيد المحاولة عند الفشل؛ لذلك idempotency ليست تحسينًا اختياريًا.

التشغيل

- ضع حدودًا لعدد الاتصالات والرسائل والحجم.
- استخدم ping/heartbeat واكتشف الاتصالات الميتة.
- طبّق authentication عند الاتصال وauthorization لكل قناة/حدث.
- راقب reconnect rate وqueue lag وdelivery failures.

13. تصميم APIs

REST و GraphQL وتصميم الموارد والحالات والأخطاء والصفحات والإصدارات والعقود.

اختر الأسلوب حسب المشكلة

- **REST**: موارد وعقود HTTP؛ مناسب لمعظم CRUD والـ public APIs.
- **RPC**: عمليات صريحة مثل `calculateShipping`؛ مناسب عندما يكون الفعل أهم من المورد.
- **GraphQL**: العميل يحدد الحقول عبر schema؛ قوي للواجهات المتنوعة لكنه يحتاج حدود `complexity` و `authorization` لكل `resolver`.

الاسم لا يضمن الجودة. العقد الواضح، التوافق، الأمان والمراقبة أهم من الشعار.

موارد و HTTP

```
GET    /api/orders/42
POST   /api/orders
PATCH /api/orders/42
DELETE /api/orders/42
```

- استخدم nouns متسقة.
- أعد 201 مع Location عند الإنشاء، و 204 عندما لا يوجد body.
- ميّز 409 conflict، 404 absence، 403 permission، 401 authentication، 400 parsing، و validation 422 وفق عقدك.
- لا تجعل كل نتيجة 200.

Problem Details و Validation

```
{
  "type": "https://docs.example/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": {"email": ["Invalid format"]},
  "request_id": "req_01j..."
}
```

استخدم شكل خطأ ثابتًا، ولا تكشف stack trace أو SQL. معيار Problem Details يعرّف حقولًا قابلة للامتداد.

```
GET /api/orders?status=paid&limit=20&cursor=eyJpZCI6OTAwfQ
```

ضع حدًا أقصى للصفحة. Cursor pagination أفضل غالبًا للبيانات الكبيرة المتغيرة، بينما offset أبسط للتنقل المحدود.

التوافق والإصدارات

- فضّل التغيير الإضافي backward-compatible.
- لا تغيّر معنى حقل قائم بصمت.
- أعلن deprecation وموعد الإزالة.
- اختبر consumers بعقود آلية.
- يمكن وضع الإصدار في path أو header؛ الاتساق أهم من الاختيار.

العقد والتشغيل

وثّق OpenAPI أو schema قابلة للاختبار، وأضف authentication، rate limits، idempotency، request IDs، timeouts وobservability. صمّم الـ API للفشل الجزئي وإعادة المحاولة لا للمسار السعيد فقط.

مراجع

02

المسار ٠ 16 درسًا

لغة PHP

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

مسار شامل ومصحّح لأساسيات PHP اعتمادًا على الصفحات 12-60 من الملاحظات.

هذا القسم يحوّل محتوى الصفحات 12-60 من الملاحظات إلى دروس قابلة للمذاكرة والتطبيق. احتفظنا بكل الأفكار المكتوبة، وصححنا النقاط التي تغيّرت أو تحتاج دقة، ثم أضفنا أمثلة عملية وحالات شائعة وأخطاء يجب تجنبها.

خريطة الدروس

1. مقدمة وصياغة PHP — ما PHP؟ وكيف تعمل؟ والوسوم والتعليقات والدمج مع HTML.
2. المتغيرات والنطاق و Superglobals — الإسناد بالقيمة والمرجع والمتغيرات المتغيرة ونطاق المتغير.
3. أنواع البيانات ونظام الأنواع — الأنواع الأساسية والمركبة و callable و iterable و Union/Intersection والكتابة الصارمة.
4. الإخراج والتصحيح والثوابت — echo و print و var_dump و const و define.
5. الشروط والحلقات — if و switch و match والحلقات و foreach بالمرجع.
6. التعبيرات والمؤثرات — الأولوية والمقارنة والمنطق والمصفوفات والتنفيذ و Pipe Operator.
7. الدوال وال Callbacks وتضمين الملفات — الدوال المجهولة و Arrow Functions و First-class Callables و include و require.
8. الملفات وال Streams و JSON و CSV — أوضاع fopen والصلاحيات وال wrappers وال contexts والفلاتر.
9. رفع الملفات والكوكيز والجلسات — رفع آمن، إعدادات Cookies، ودورة حياة Session.
10. Namespaces و Autoloading — تنظيم الأسماء والاستيراد وربط PSR-4.
11. الأخطاء والاستثناءات — Throwable والاستثناءات المخصصة وحدود المعالجة.
12. المصفوفات وأدوات التحويل — map و filter و reduce والفرز والذاكرة.
13. النصوص و Unicode و UTF-8 — Regex و mbstring والتطبيع وال patterns الآمنة.
14. التاريخ والوقت — UTC و DateTimeImmutable و timezones و DST.
15. Request و Router و Front Controller — Response و JSON و Middleware.
16. ميزات PHP الحديثة — PHP 8.4 و PHP 8.5 و Hooks و Pipe و URI و Clone With.



طريقة المذاكرة

نقّذ كل مثال، ثم غيّر المدخلات وتوقع النتيجة قبل التشغيل. الأمثلة الأمنية ليست إضافات اختيارية؛ هي جزء من الاستخدام الصحيح لـ PHP.

```
<?php

declare(strict_types=1);

function greet(string $name): string
{
    return "مرحبًا {$name}";
}

echo greet('عمر');
```

الأمثلة مكتوبة لـ PHP 8.x، وأي ميزة تحتاج إصدارًا أحدث مذكور إصدارها بجوارها.

1. مقدمة وصياغة PHP

ماهية PHP، آلية تنفيذ الطلب، الصياغة، الدمج مع HTML، التعليقات وحساسية الأحرف.

ما PHP؟

بدأ الاسم تاريخيًا بمعنى **Personal Home Page**، ثم أصبح الاسم الرسمي المتكرر **PHP: Hypertext Preprocessor**. هي لغة مفتوحة المصدر، عامة الاستخدام، ومشهورة خصوصًا في برمجة جانب الخادم.

عند طلب صفحة PHP لا يصل كود PHP إلى المتصفح:

1. يرسل المتصفح طلب HTTP.
2. يستقبل خادم الويب الطلب ويمرر ملف `.php` إلى PHP.
3. تنفذ PHP الكود وتتواصل عند الحاجة مع قاعدة البيانات أو الملفات والخدمات.
4. تعيد ناتجًا مثل HTML أو JSON.
5. يرسل الخادم الاستجابة إلى المتصفح.



تصحيح مهم

وصف PHP بأنها «تُفسَّر سطرًا بسطر فقط» تبسيط غير دقيق؛ يحوّل المحرك الكود إلى opcodes ثم ينفذها، وتوجد JIT في الإصدارات الحديثة. المهم عمليًا أن المستخدم لا يحتاج خطوة build تقليدية لكل تعديل.

ماذا تستطيع PHP أن تفعل؟

- معالجة النماذج وطلبات HTTP وبناء صفحات ديناميكية وواجهات API.
- القراءة والكتابة في قواعد البيانات والملفات.
- العمل من سطر الأوامر لتشغيل المهام المجدولة وعمليات المعالجة وإرسال البريد.
- إنشاء صور أو ملفات PDF عند تثبيت المكتبات المناسبة.
- التشفير والتحقق من كلمات المرور والتعامل مع JSON.
- البرمجة الإجرائية والبرمجة كائنية التوجه.

يمكن بناء تطبيق سطح مكتب بأدوات خارجية، لكنه ليس الاستخدام الشائع أو الاختيار الأول لـ PHP.

يمكن تثبيت المكونات منفردة أو استخدام حزمة محلية:

- **XAMPP** لأنظمة متعددة.
- **WAMP** لويندوز.
- **MAMP** لماك.
- **LAMP** للينكس.

الحروف تشير عادة إلى نظام التشغيل، وApache، وMySQL/MariaDB، وPHP. ويمكن كذلك تشغيل خادم التطوير المدمج:

```
php -S localhost:8000 -t public
```

هذا الخادم للتطوير المحلي، وليس للإنتاج.

الوسوم والدمج مع HTML

```
<?php
$title = 'متجري';
?>
<!doctype html>
<html lang="ar" dir="rtl">
<head><title><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></title></head>
<body>
  <?php if ($title !== ''): ?>
    <h1 class="page-title"><?= htmlspecialchars($title) ?></h1>
  <?php else: ?>
    <p>لا يوجد عنوان</p>
  <?php endif; ?>
</body>
</html>
```

- يبدأ الكود بـ `<?php`.
- الاختصار `<?= $value` يعني إخراج القيمة.
- الصياغة البديلة `if: ... endif;` مناسبة للقوالب، ومنها أفكار قوالب Blade.
- في ملف PHP خالص يُفضّل حذف وسم الإغلاق `>` لتجنب إرسال مسافات غير مقصودة.
- تنتهي التعليمات البسيطة بـ `;`، أما كتلة `if { ... }` فلا تحتاج فاصلة بعد `}`.



الإخراج وHeaders

يجب استدعاء `header()` و `setcookie()` و `session_start()` قبل إرسال أي HTML أو نص، وإلا قد يظهر خطأ "headers already sent".

```
<?php
header('Content-Type: application/json; charset=utf-8');
echo json_encode(['status' => 'ok'], JSON_UNESCAPED_UNICODE);
```

التعليقات وحساسية الأحرف

```
// تعليق سطر واحد
# تعليق سطر واحد أيضا
/*
    تعليق متعدد الأسطر
*/
$username = 'Omar';
echo $username; // صحيح
// echo $username; // متغير مختلف
```

أسماء المتغيرات حساسة لحالة الأحرف. كلمات اللغة مثل `if` ليست حساسة عمليًا، وأسماء الدوال والفئات تُطابق دون حساسية، لكن التزم دائمًا بالحالة المعرّفة ومعايير PSR. الثوابت الحديثة حساسة للحالة.

2. المتغيرات والنطاق و Superglobals

قواعد تسمية المتغيرات، القيمة والمرجع، النطاق، static والمتغيرات الفائقة.

المتغيرات

PHP ديناميكية النوع؛ يبدأ اسم المتغير بـ `$` ثم حرف أو underscore، ويمكن أن تتبعه أرقام. الاسم حساس لحالة الأحرف ولا يبدأ برقم.

```
$name = 'Omar';
$count = 3;
$price2 = 19.5;

echo "$name لديه {$price2} جنيه";
echo 'single quotes لا يُستبدل داخل $name';
```

استخدم `{ $name }` داخل النص المركب. الصيغة `{ $name }` القديمة داخل النصوص deprecated؛ لا تعتمد عليها.

الإسناد بالقيمة والمرجع

```
$a = 10;
$b = $a; // نسخة من القيمة
$b++;
echo $a; // 10

$x = 10;
$y =& $x; // الاسمان يشيران إلى الحاوية نفسها
$y++;
echo $x; // 11
unset($y); // يفك الربط، ولا يحذف $x
```

المرجع ليس pointer يدويًا مثل C. لا تستخدمه إلا عند حاجة واضحة؛ فهو يجعل تتبع التغييرات أصعب.

المتغيرات المتغيرة

```
$field = 'email';
$$field = 'omar@example.com';
echo $email;
```



تنبيه

المتغيرات المتغيرة موجودة، لكن المصفوفات أو الكائنات أو `match` أوضح وأكثر أمانًا عندما تأتي الأسماء من مدخل المستخدم.

النطاق

```
$total = 100;

function addTax(float $rate): float
{
    global $total;
    return $total * (1 + $rate);
}

function addTaxExplicit(float $rate): float
{
    return $GLOBALS['total'] * (1 + $rate);
}
```

المتغير خارج الدالة `global`، وداخلها `local`. الأفضل تمرير القيم كوسائط بدل `global` و `$GLOBALS`.

```
function nextId(): int
{
    static $id = 0;
    return ++$id;
}

echo nextId(); // 1
echo nextId(); // 2
```

المتغير `static` المحلي يُهيأ مرة ويحفظ قيمته بين استدعاءات الدالة داخل العملية الحالية.

الملف المضمّن بـ `include` يرث نطاق السطر الذي ضُمّن فيه. وإذا كان التضمين داخل دالة، تكون متغيراته في نطاق الدالة.

Superglobals

تتاح هذه المصفوفات في كل النطاقات:

المتغير	الاستخدام
<code>\$_GET</code>	query string
<code>\$_POST</code>	جسم نموذج POST
<code>\$_SERVER</code>	معلومات الطلب والخادم
<code>\$_FILES</code>	الملفات المرفوعة
<code>\$_COOKIE</code>	Cookies القادمة
<code>\$_SESSION</code>	بيانات الجلسة بعد <code>session_start()</code>
<code>\$GLOBALS</code>	جدول المتغيرات العامة

```
$method = $_SERVER['REQUEST_METHOD'] ?? 'CLI';
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT) ?: 1;
$selected = $_GET['check_orders'] ?? []; // ?check_orders[]=10&check_orders[]=20

if (!is_array($selected)) {
    $selected = [];
}
$selected = array_map('intval', $selected);
```

وجود القيمة في Superglobal لا يعني أنها آمنة. تحقق من النوع والنطاق المسموح، واستخدم escaping عند الإخراج وprepared statements لقاعدة البيانات.

eval()

`eval()` ينفذ نصًا ككود PHP:

```
$code = 'echo "hello";';
eval($code);
```



تحذير

لا تمرر أي مدخل مستخدم إلى `eval()`. في التطبيقات العادية لا تحتاجها أصلًا؛ استخدم دوالًا أو خرائط callbacks أو classes بدلًا منها.

3. أنواع البيانات ونظام الأنواع

الأنواع الأساسية والمركبة والخاصة، التحويل، callable و iterable وتصريحات النوع.

خريطة الأنواع

- Scalar: `bool` و `int` و `float` و `string`.
- مركبة: `array` و `object`.
- خاصة: `null` و `resource` و `callable` و `iterable` و `mixed` و `void` و `never`.
- يعرفها المطور: `enums` و `interfaces` و `classes`.
- مركبة في التصريحات: Union مثل `int|string` و Intersection مثل `Countable&Iterator`.
- Singleton types مثل `true` و `false`.

bool و null

يكون المتغير `null` إذا أُسندت إليه `null`، أو لم يُعرّف، أو أزيل بـ `unset`. استخدم `is_null($x)` أو `$x === null`.

القيم falsey هي: `false` و `0` و `0.0` و `""` و `"0"` و `[]` و `null`. لذلك استخدم المقارنة الصارمة عندما يهم النوع.

```
var_dump(0 == false); // true
var_dump(0 === false); // false
```

float و int

```
$decimal = 42;
$octal = 0o52;
$hex = 0x2A;
$binary = 0b101010;

echo 7 / 2; // 3.5
echo intdiv(7, 2); // 3
echo (int) 3.9; // 3، تقطيع لا تقريب
```

عند تجاوز مجال `int` قد تتحول النتيجة إلى `float`. والكسور الثنائية ليست دقيقة تمامًا:

```
var_dump(0.1 + 0.2 === 0.3); // غالبًا false
echo round(0.1 + 0.2, 2); // 0.3
```

للأموال استخدم أصغر وحدة صحيحة مثل القروش، أو مكتبة `decimal` مناسبة.

```
$name = 'Omar';
$message = "Hello {$name}";
$joined = 'PHP' . ' ' . '8';
echo $message[0];
echo $message[-1];
```

```
$heredoc = <<<TEXT
Hello $name
Multiple lines
TEXT;

$nowdoc = <<<'TEXT'
$name is not interpolated here
TEXT;
```

لا تعتمد على التحويل الحسابي الضمني للنصوص؛ تحقق بـ `filter_var` أو `is_numeric` ثم حوّل صراحة.

المصفوفات والكائنات و Enum و Resource

المصفوفة في PHP قد تكون قائمة، associative map، أو متعددة الأبعاد:

```
$colors = ['red', 'blue'];
$user = ['id' => 7, 'name' => 'Omar'];
$matrix = [[1, 2], [3, 4]];
```

الكائن instance من class يجمع properties و methods. وال enum يمثل مجموعة محدودة من الحالات:

```
enum OrderStatus: string
{
    case Pending = 'pending';
    case Paid = 'paid';
}
```

`resource` مقبض لمورد خارجي مثل stream. كثير من الامتدادات الحديثة أصبحت تعيد objects بدل resources، لذا افحص التوثيق و `get_debug_type()`.

void و never و mixed

- `void`: الدالة لا تسمح بإرجاع قيمة مفيدة؛ يمكن كتابة `return;`.
- `never`: الدالة لا تعود طبيعيًا لأنها ترمي exception أو تستدعي `exit` أو لا تنتهي.
- `mixed`: يقبل كل الأنواع، ومنها `null`؛ استخدم نوعًا أدق إن أمكن.


```
function logMessage(string $message): void
{
    error_log($message);
}

function fail(string $message): never
{
    throw new RuntimeException($message);
}
```

callable و Closure و First-class callable

```
$double = function (int $n): int { return $n * 2; };
$short = fn (int $n): int => $n * 2;

class Formatter
{
    public function upper(string $value): string
    {
        return strtoupper($value);
    }

    public function __invoke(string $value): string
    {
        return trim($value);
    }
}

$f = new Formatter();
$callables = [$double, [$f, 'upper'], $f];
$upper = $f->upper(...); // First-class callable syntax
```

Callback هو callable يتم تمريره ليُستدعى لاحقًا. Closure كائن يمثل دالة مجهولة. والكائن يصبح callable إذا عرّف `__invoke()`.

iterable و Generator

`iterable` يعني array أو كائنًا يطبق `Traversable`. و `Iterator` يعرف `rewind/current/key/next/valid`.

```
function numbers(int $max): iterable
{
    for ($i = 1; $i <= $max; $i++) {
        yield $i;
    }
}
```

كل `Generator` هو `Iterator`، وليس كل `Iterator` مولّدًا. `yield` يوقف التنفيذ مؤقتًا ويستأنف من نفس النقطة، أما `return` فينهيّه.

تصريحات النوع والكتابة الصارمة

```
<?php
declare(strict_types=1);

function findUser(int $id, ?string $locale = null): array|null
{
    return $id > 0 ? ['id' => $id, 'locale' => $locale] : null;
}
```

بدون strict mode قد تحول PHP الأنواع scalar الممكنة. الكتابة الصارمة قرار لكل ملف مستدي، وتنطبق على الأنواع scalar مع استثناء قبول `int` حيث يُطلب `float`. افحص الأنواع بـ `get_debug_type()` و `var_dump()`.

4. الإخراج والتصحيح والثوابت

echo و print أدوات فحص القيم والثوابت العادية والسحرية والمسبقة.

echo و print

كلاهما language construct وليس دالة عادية.

الخاصية	echo	print
القيمة المرجعة	لا شيء	دائمًا 1
عدد الوسائط	يقبل عدة وسائط بلا أقواس	وسيط واحد
داخل expression	لا	نعم، بسبب القيمة المرجعة
الاختصار	<?= ... ?>	لا يوجد

```
echo 'Hello', ' ', 'PHP', PHP_EOL;
$result = print 'Printed';
var_dump($result); // int(1)
```

فرق السرعة غير مهم عمليًا؛ اختر `echo` عادة، ولا تطبع مدخل مستخدم داخل HTML دون `htmlspecialchars`.

أدوات التصحيح

```
$user = ['id' => 7, 'active' => true];

var_dump($user);           // النوع والقيمة والتفاصيل
print_r($user);            // عرض أسهل للقراءة
$text = print_r($user, true); // أعد العرض كنص
echo get_debug_type($user); // array
```

في أطر العمل قد تجد `dump()` و `dd()`؛ الثانية تطبع ثم توقف التنفيذ. لا تترك بيانات حساسة أو debugging في الإنتاج. استخدم logger وبيئة تطوير منفصلة.

```
const APP_NAME = 'Omar Notes';
define('APP_VERSION', '1.0.0');

echo APP_NAME;
```

الثابت لا يبدأ بـ `$`، وهو متاح عالميًا. الفرق الدقيق:

- `define()` استدعاء وقت التشغيل، لذلك يمكن وضعه داخل شرط؛ ينشئ `constant` عامة.
- `const` تصريح لغوي وله قيود موضوعية، ويمكن استخدامه لتعريف `class constants`.
- لا يمكن استخدام `define()` لتعريف `class constant`.
- لا تضع `const` داخل `function` أو `block` شرطي.

```
class HttpStatus
{
    public const OK = 200;
    public const NOT_FOUND = 404;
}
```

Predefined constants و Magic

```
echo __LINE__;
echo __FILE__;
echo __DIR__;
echo __FUNCTION__;
echo __CLASS__;

echo PHP_VERSION;
echo PHP_OS_FAMILY;
echo PHP_EOL;
```

تتغير Magic constants حسب مكانها في المصدر. `__DIR__` مهم لبناء مسارات ثابتة لا تعتمد على `working directory`:

```
$config = require __DIR__ . '/../config/app.php';
```

5. الشروط والحلقات

if و switch و match و for و while و do-while و foreach مع أهم الفروق والأخطاء.

if و elseif و else

```
$score = 82;

if ($score >= 90) {
    $grade = 'A';
} elseif ($score >= 75) {
    $grade = 'B';
} else {
    $grade = 'C';
}
```

في القوالب يمكن استخدام الصياغة البديلة:

```
<?php if ($grade === 'A'): ?>
    <strong>ممتاز</strong>
<?php else: ?>
    <span>استمر</span>
<?php endif; ?>
```

Nullsafe و Null coalescing و Ternary

```
$label = $active ? 'نشط' : 'متوقف';
$username = $_GET['username'] ?? 'guest';
$postId = $user->latestPost()->id; // null إذا انقطعت السلسلة عند null
```

?? يفحص الوجود وعدم **null** بطريقة تشبه **isset**. وال **nullsafe operator** هو **->**، لا **?**.

switch

```
switch ($role) {
    case 'admin':
        $permissions = ['all'];
        break;
    case 'editor':
        $permissions = ['read', 'write'];
        break;
    default:
        $permissions = ['read'];
}
```

نسيان `break` يسبب fall-through وقد يكون مقصودًا أو خطأ. تاريخيًا يستخدم `switch` مقارنة غير صارمة، لذلك تجنب خلط الأنواع.

match

```
$message = match ($status) {
    200, 201 => 'نجاح',
    404 => 'غير موجود',
    default => 'خطأ غير متوقع',
};

$category = match (true) {
    $age < 13 => 'طفل',
    $age < 18 => 'مراهق',
    default => 'بالغ',
};
```

expression `match` تعيد قيمة، تستخدم `===`، لا يحدث فيها fall-through، ولا تحتاج `break`. وإذا لم يوجد arm مطابق ولا `default` ترمي `UnhandledMatchError`.

for

```
for ($i = 0; $i < 5; $i++) {
    echo $i;
}

$names = ['Ali', 'Mona', 'Omar'];
for ($i = 0, $count = count($names); $i < $count; $i++) {
    echo $names[$i];
}
```

الأجزاء الثلاثة اختيارية: `for (;;)` حلقة لا نهائية وتحتاج `break` أو نهاية للعملية. توجد صياغة `for (...): ... endfor;` للقوالب.

do-while و while

```
$attempts = 0;
while ($attempts < 3) {
    $attempts++;
}

do {
    $input = readline();
} while ($input === '');
```

`while` تفحص قبل التنفيذ وقد لا تعمل مرة واحدة. `do-while` تنفذ الجسم مرة على الأقل.

```
$users = [
    ['id' => 1, 'name' => 'Ali'],
    ['id' => 2, 'name' => 'Mona'],
];

foreach ($users as $index => ['id' => $id, 'name' => $name]) {
    echo "{$index}: {$id} - {$name}";
}
```

التعديل بالمرجع:

```
$prices = [10, 20, 30];

foreach ($prices as &$amp;price) {
    $price *= 1.14;
}

unset($price); // إزالة المرجع الباقي من آخر عنصر
```

استخدم `continue` لتخطي الدورة الحالية و `break` لإنهاء الحلقة. ويمكن `break 2` للخروج من حلقتين متداخلتين، وهو أوضح غالبًا من `goto`.

6. التعبيرات والمؤثرات

قيم التعبيرات، الأولوية، الحساب والإسناد والمقارنة والمنطق والمصفوفات والتنفيذ وال Pipe.

التعبير والأولوية

كل ما ينتج قيمة يُعد expression. حتى الإسناد يعيد القيمة المسندة:

```
$b = $a = 5;
$result = 1 + 5 * 3; // 16
$grouped = (1 + 5) * 3; // 18
```

الأقواس أفضل من حفظ جدول الأولوية. الفخ الأشهر:

```
$ok = true && false; // false
$ok = true and false; // ($ok = true) and false، true فتبقى
```

استخدم `&&` و `||` عادة، وضع أقواسًا عندما يختلط أكثر من مؤثر.

الحساب والزيادة والإسناد

```
$a = 10 + 2;
$b = 10 - 2;
$c = 10 * 2;
$d = 10 / 4;
$rest = 10 % 3;
$power = 2 ** 3;

$x = 5;
echo $x++; // 5 ثم تصبح 6
echo ++$x; // تصبح 7 ثم تُطبع
```

الإسنادات المختصرة: `+= -= *= /= %= **= . = ??=`.

المقارنة

- `==` مساواة مع تحويلات؛ `===` قيمة ونوع.
- `!=` و `<>` عدم مساواة، و `!==` عدم تطابق صارم.
- `< > <= >=` للترتيب.
- `<=>` يعيد -1 أو 0 أو 1.


```
$numbers = [30, 10, 20];
usort($numbers, fn (int $a, int $b): int => $a <=> $b);
```

المؤثرات المنطقية والنصوص والمصفوفات

```
$allowed = $loggedIn && ($isAdmin || $ownsResource);
$denied = !$allowed;
$title = 'Hello' . ' ' . 'PHP';
$title .= ' 8';
```

```
$left = ['a' => 1, 'shared' => 'left'];
$right = ['b' => 2, 'shared' => 'right'];
$union = $left + $right;
// a=1, shared=left, b=2
```

مؤثر **+** للمصفوفات **union حسب المفاتيح** ويحتفظ بقيمة اليسار عند تكرار المفتاح؛ ليس بديلًا مطابقًا لـ `array_merge()` و `==` يقارن أزواج المفتاح/القيمة، أما `===` فيطلب أيضًا النوع والترتيب نفسه.

```
if ($service instanceof Cacheable) {
    $service->warm();
}
```

إسناد object إلى متغير آخر يجعل المتغيرين يشيران عادة إلى الكائن نفسه؛ استخدم `clone` لنسخة كائن مستقلة، مع الانتباه للكائنات الداخلية.

Error control والتنفيذ

`@expression` يخفي عرض أخطاء التعبير. تجنيه لأنه يطمس السبب؛ عالج النتيجة، ارم `exception`، وسجل الخطأ. يمكن تخصيص معالجة بعض الأخطاء بـ `set_error_handler()`.

Backticks تنفذ أمر shell مثل `shell_exec()`:

```
$output = `whoami`;
```



تحذير

لا تُدخل بيانات المستخدم في backticks أو `exec` أو `system` أو `shell_exec` أو `proc_open`. هذا قد يتحول إلى Remote Code Execution. استخدم API آمنًا، وإذا كان التنفيذ ضروريًا فمرر arguments منفصلة وقيد الأوامر والصلاحيات.

```
$slug = ' PHP 8.5 Released '  
|> trim(...)  
|> (fn (string $s) => str_replace(' ', '-', $s))  
|> strtolower(...);
```

يمرر `|>` ناتج اليسار كوسيط وحيد إلى callable في اليمين، فيجعل السلسلة تُقرأ من أعلى لأسفل. هذا المثال لن يعمل على **PHP 8.4** أو أقدم.

7. الدوال وال Callbacks وتضمين الملفات

تعريف الدوال والوسائط والمراجع وال closures وال arrow functions وال include وال require.

تعريف الدالة

الدالة كتلة قابلة لإعادة الاستخدام تنفذ مهمة محددة. الدوال العادية في PHP ذات نطاق عام (مع مراعاة namespace)، ولا يمكنك إعلان دالتين بالاسم نفسه في النطاق نفسه للتعامل مع توقعات مختلفة.

```
function calculateTotal(float $price, int $quantity = 1): float
{
    return $price * $quantity;
}

echo calculateTotal(19.5, 3);
echo calculateTotal(quantity: 3, price: 19.5); // named arguments
```

Parameters هي الأسماء في التعريف، و **arguments** هي القيم عند الاستدعاء. يمكن محاكاة حالات متعددة بـ **union** و **types** و **variadics**:

```
function sum(int ...$numbers): int
{
    return array_sum($numbers);
}

echo sum(1, 2, 3);
```

فضل **return** كي تكون الدالة قابلة للاختبار والتركيب، و اترك **echo** لطبقة العرض.

التمرير بالقيمة والمراجع

```
function increment(int $number): int
{
    return $number + 1;
}

function incrementInPlace(int &$number): void
{
    $number++;
}
```

المراجع يغيّر متغير المستدعي، لذلك اجعله واضحًا ونادرًا.

Callbacks و Variable functions

```
function greet(string $name): string
{
    return "Hello {$name}";
}

$functionName = 'greet';
echo $functionName('Omar');

$routes = [
    'home' => fn (): string => 'Home',
    'health' => fn (): array => ['status' => 'ok'],
];
$response = $routes[$route] ?? fn () => 'Not found';
```

استدع callback بعد التحقق بـ `is_callable()` إذا لم يكن النوع مضمونًا.

Arrow Function و use و Closure

```
$tax = 0.14;

$withTax = function (float $price) use ($tax): float {
    return $price * (1 + $tax); // القيمة $tax التقط
};

$counter = 0;
$next = function () use (&$counter): int {
    return ++$counter; // التقاط بالمرجع
};

$withTaxShort = fn (float $price): float => $price * (1 + $tax);
```

Arrow function تلتقط متغيرات النطاق الخارجي تلقائيًا بالقيمة وتحتوي expression واحدة. الدالة المجهولة العادية تستخدم `use`، ويمكن أن تلتقط بالمرجع.

```
$names = ['ali ', 'mona '];
$clean = array_map(trim(...), $names); // First-class callable
```

إعلان دالة داخل دالة ممكن، لكن الدالة الداخلية لا تُعلن إلا بعد تنفيذ الخارجية وتصبح في نطاق الدوال؛ تجنب هذا الأسلوب، واستخدم Closure بدلًا منه.

require و include

كلاهما language construct يقرأ ملفًا وينفذه:

```
$config = require __DIR__ . '/../config/app.php';
include __DIR__ . '/partials/header.php';
```

- فشل `require` يوقف المسار الحالي برمّي `Error` في PHP الحديثة.
- فشل `include` يصدر `E_WARNING` ويكمل التنفيذ غالبًا.
- `require_once` و `include_once` يمنعان تحميل الملف نفسه أكثر من مرة.
- إذا لم يُرجع الملف قيمة صريحة، يكون ناتج التضمين الناجح عادة `1`.
- `return` داخل الملف المضمن ينهي ذلك الملف ويعيد القيمة.
- الملف المضمن يرث نطاق مكان التضمين.

: config/app.php

```
<?php
return [
    'name' => 'Omar Notes',
    'debug' => false,
];
```



نصيحة

استخدم `__DIR__` بدل الاعتماد على `current working directory`. استخدم `require_once` لتعريفات لا يجوز تكرارها، واعتمد Composer autoload للفئات في المشاريع الحقيقية.

goto

```
goto done;
echo 'لن نُنفذ';
done:
echo 'تم';
```

`goto` يقفز إلى label داخل الملف والنطاق نفسه، ولا يجوز القفز إلى داخل loop أو switch. نادرًا ما يكون أوضح من دالة صغيرة أو `break` أو `continue`.

8. الملفات وال Streams و JSON و CSV

نظام الملفات، أوضاع الفتحة، الصلاحيات، streams و wrappers و contexts و JSON و CSV.

Streams

ال stream واجهة موحدة للتعامل مع تدفق بيانات من مصدر أو إلى وجهة: ملف، ذاكرة، شبكة، URL، أو عملية أخرى. يتكون المفهوم من:

1. **Wrapper** يحدد البروتوكول مثل `file://` و `http://` و `ftp://` و `php://` و `data://` و `zlib://`. ويمكن تسجيل wrapper مخصص بـ `stream_wrapper_register()`.
2. **Context** يمرر `options` و `parameters` مثل `timeout` و `HTTP headers`.
3. **Filter** يحول البيانات أثناء القراءة أو الكتابة.

```
$context = stream_context_create([
    'http' => [
        'timeout' => 3,
        'header' => "Accept: application/json\r\n",
        'ignore_errors' => true,
    ],
]);

$body = file_get_contents('https://example.com/api', false, $context);
```



تنبيه

`allow_url_fopen` يتحكم في استخدام URL-aware wrappers مع دوال الملفات. لا تفعل `allow_url_include`؛ تضمين كود PHP من URL مخاطرة شديدة. لطلبات HTTP الحقيقية استخدم عملياً يدعم TLS والتحقق من الحالة وإعادة المحاولة.

ال streams قد تعمل محلياً أو عبر الشبكة وتستخدم `buffers` و `chunks`. راقب أخطاء الاتصال والصلاحيات والتسجيل، وافحص `false` بدل إخفاء الخطأ بـ `@`.

```
$handle = fopen(__DIR__ . '/data.txt', 'rb');

if ($handle === false) {
    throw new RuntimeException('تعذر فتح الملف');
}

try {
    while (($line = fgets($handle)) !== false) {
        echo rtrim($line), PHP_EOL;
    }
} finally {
    fclose($handle);
}
```

الوضع	المعنى
<code>r</code>	قراءة من البداية؛ الملف يجب أن يوجد
<code>r+</code>	قراءة وكتابة؛ الملف يجب أن يوجد
<code>w</code>	كتابة مع تفريغ الملف أو إنشائه
<code>w+</code>	قراءة وكتابة مع التفريغ أو الإنشاء
<code>a</code>	كتابة في النهاية أو إنشاء
<code>a+</code>	قراءة وكتابة؛ الكتابة دائمًا في النهاية
<code>x</code> / <code>x+</code>	إنشاء جديد فقط؛ يفشل إن كان موجودًا
<code>c</code> / <code>c+</code>	إنشاء إن لزم بلا تفريغ؛ المؤشر في البداية

أضف `b` مثل `rb` للملفات الثنائية، خصوصًا للتوافق بين الأنظمة.

```
$handle = fopen(__DIR__ . '/app.log', 'ab');
if ($handle === false) {
    throw new RuntimeException('Cannot open log');
}

try {
    if (!flock($handle, LOCK_EX)) {
        throw new RuntimeException('Cannot lock log');
    }
    fwrite($handle, date(DATE_ATOM) . " started\n");
    fflush($handle);
    flock($handle, LOCK_UN);
} finally {
    fclose($handle);
}
```

`fwrite()` قد يعيد عدد bytes أقل من المطلوب، و `fclose()` يغلق المقبض. يمكن استخدام `file_get_contents` و `file_put_contents` للملفات الصغيرة، مع `LOCK_EX` عند الحاجة.

الصلاحيات

في Unix: read=4، write=2، execute=1، وتُجمع لكل من owner و group و others.

- `0644`: المالك يقرأ ويكتب، والباقيون يقرؤون.
- `0755`: المالك كامل الصلاحيات، والباقيون قراءة وتنفيذ.
- `0600`: المالك فقط يقرأ ويكتب.

```
chmod($path, 0640);
```

لا تجعل `0777` حلاً افتراضياً. `chmod` وسلوك الملكية يختلفان على Windows، وتتحكم صلاحيات نظام التشغيل والمستخدم الذي يشغل PHP في النتيجة.

JSON و Serialization

```
$json = json_encode(
    ['name' => 'عمر', 'active' => true],
    JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR
);

$data = json_decode($json, true, flags: JSON_THROW_ON_ERROR);

if (function_exists('json_validate')) {
    var_dump(json_validate($json)); // PHP 8.3+
}
```

`serialize()` و `unserialize()` يحتفظان ببنية PHP، لكن:



تحذير

لا تستخدم `unserialize()` على بيانات غير موثوقة؛ قد يؤدي إلى Object Injection. استخدم JSON للبيانات المتبادلة، أو قيّد `allowed_classes` عند الضرورة.


```
$out = fopen(__DIR__ . '/users.csv', 'wb');
fputcsv($out, ['id', 'name']);
fputcsv($out, [1, 'Omar']);
fclose($out);

$in = fopen(__DIR__ . '/users.csv', 'rb');
while (($row = fgetcsv($in)) !== false) {
    [$id, $name] = $row;
}
fclose($in);

$row = str_getcsv('2,"Mona Ahmed"');
```

استخدم دوال CSV بدل `explode(',')` لأنها تتعامل مع علامات الاقتباس والفواصل داخل الحقول.

9. رفع الملفات والكوكيز والجلسات

رفع الملفات بأمان، خيارات Cookies، و CSRF، ودورة حياة الجلسة وحمايتها.

رفع الملفات

يجب أن يكون النموذج POST وب multipart/form-data :

```
<form method="post" enctype="multipart/form-data">
  <input type="file" name="avatar" accept="image/png,image/jpeg">
  <button>رفع</button>
</form>
```

تضع PHP البيانات في \$_FILES وتنقل الملف أولاً إلى upload_tmp_dir. لا تثق في الاسم أو MIME القادم من المتصفح.

```
<?php
declare(strict_types=1);

$file = $_FILES['avatar'] ?? null;

if (!is_array($file) || $file['error'] !== UPLOAD_ERR_OK) {
    throw new RuntimeException('فشل الرفع');
}

if ($file['size'] > 2 * 1024 * 1024) {
    throw new RuntimeException('2MB الحد الأقصى');
}

$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($file['tmp_name']);
$extensions = ['image/jpeg' => 'jpg', 'image/png' => 'png'];

if (!isset($extensions[$mime])) {
    throw new RuntimeException('نوع غير مسموح');
}

$name = bin2hex(random_bytes(16)) . '.' . $extensions[$mime];
$target = __DIR__ . '/../storage/uploads/' . $name;

if (!move_uploaded_file($file['tmp_name'], $target)) {
    throw new RuntimeException('تعدّر حفظ الملف');
}
```

افحص UPLOAD_ERR_* والحجم الحقيقي و MIME ب finfo ، وأعد تسمية الملف، و خزنه خارج public web root إن أمكن. لا تنفذ الملف، واضبط upload_max_filesize و post_max_size .

Cookies

الكوكي قيمة صغيرة يخزنها المتصفح ويرسلها مع الطلبات المطابقة للنطاق والمسار.

```
setcookie('theme', 'dark', [
    'expires' => time() + 60 * 60 * 24 * 30,
    'path' => '/',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Lax',
]);

$theme = $_COOKIE['theme'] ?? 'light';
```

- **Secure**: الإرسال عبر HTTPS فقط.
- **HttpOnly**: يمنع JavaScript من قراءة الكوكي، فيقلل سرقة session عبر XSS.
- **SameSite=Lax/Strict/None**: يقيّد الطلبات **None**. cross-site. يتطلب **Secure**.
- هذه الإعدادات تساعد ضد CSRF لكنها لا تستبدل CSRF token في العمليات الحساسة.
- لا تضع أسرارًا أو بيانات حساسة خامًا في Cookies.

Sessions

بيانات الجلسة تُحفظ عادة على الخادم، بينما يحتفظ المتصفح بمعرف session في Cookie. عند `session_start()` تنشئ PHP جلسة أو تستعيدها وتملأ `$_SESSION`.

```
session_start([
    'use_strict_mode' => true,
    'cookie_httponly' => true,
    'cookie_secure' => true,
    'cookie_samesite' => 'Lax',
]);

$_SESSION['cart'][] = 42;
$userId = $_SESSION['user_id'] ?? null;
```

بعد نجاح تسجيل الدخول أو رفع الصلاحية، غير المعرف قبل تثبيت حالة المصادقة الجديدة:

```
session_regenerate_id();
$_SESSION['user_id'] = $user->id;
$_SESSION['authenticated_at'] = time();
```



تصحيح أمني مهم

كتابة `session_regenerate_id(true)` وحذف الجلسة القديمة فورًا تبدو أكثر أمانًا، لكنها قد تسبب فقد الجلسة أو race conditions مع الطلبات المتزامنة والشبكات غير المستقرة. في نظام حساس استخدم timestamps وفترة انتقال قصيرة وفق تصميم موثق، ولا تجمع حذفًا فوريًا عشوائيًا مع `session_destroy()`.

```
session_start();
$_SESSION = [];

if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '', [
        'expires' => time() - 42000,
        'path' => $p['path'],
        'domain' => $p['domain'],
        'secure' => $p['secure'],
        'httponly' => $p['httponly'],
        'samesite' => $p['samesite'] ?? 'Lax',
    ]);
}

session_destroy();
```

- `session_unset()` يزيل متغيرات الجلسة، ويمكن أيضًا تعيين `$_SESSION = []`.
- `session_destroy()` يحذف بيانات التخزين الحالية، لكنه لا يسمح تلقائيًا مصفوفة `$_SESSION` أو Cookie عند العميل.
- `session_write_close()` / `session_commit()` يحفظ ويغلق القفل مبكرًا.
- للقراءة فقط: `session_start(['read_and_close' => true]);`

يمكن تخصيص التخزين في قاعدة بيانات أو Redis عبر `SessionHandlerInterface` أو `session_set_save_handler()`. طبق انتهاءً زمنيًا للخمول والعمر الكلي، وسجل الجلسات النشطة، ولا تعتمد على garbage collection وحده. للتفاصيل الأمنية والهجمات المرتبطة بالجلسات، تابع [حماية الجلسات](#).

10. Namespaces و Autoloading

تنظيم الأسماء والاستيراد والـ `aliases` وحل الأسماء وربط Namespace بمجلد عبر PSR-4 Composer.

لماذا Namespace؟

تمنع تعارض أسماء `classes/functions/constants` بين مكتبات مختلفة، وتعطي الكود اسمًا منطقيًا مستقلًا عن اسم الملف.

```
<?php
declare(strict_types=1);

namespace App\Billing;

final class InvoiceService {}
```

يجب أن يأتي إعلان `namespace` مبكرًا بعد `declare` إن وُجد. الـ `namespace` لا تحمّل الملف تلقائيًا؛ `autoloader` يفعل ذلك.

الاستيراد والـ `aliases`

```
namespace App\Http;

use App\Billing\InvoiceService;
use App\Contracts\Logger as LoggerContract;
use function App\Support\normalize_email;
use const App\Config\MAX_RETRIES;

$service = new InvoiceService();
```

- الاسم الذي يبدأ بـ `\` كامل من `global namespace`.
- الاسم المستورد يُحل عبر `use`.
- الاسم غير المستورد داخل `namespace` يبدأ من `namespace` الحالية.
- لا تضع `use` ديناميكيًا داخل `function`؛ `imports compile-time` وعلى مستوى الملف.

Composer و PSR-4

```
{
  "autoload": {
    "psr-4": {
      "App\\\\": "src/"
    }
  }
}
```

`App\Billing\InvoiceService` يصبح عادةً في `src/Billing/InvoiceService.php` :

```
composer dump-autoload
```

في الإنتاج استخدم autoloader المحسن حسب حجم التطبيق وطريقة النشر، ولا تكتب سلسلة `require` يدوية لكل class.

قواعد تنظيم

- class رئيسية واحدة في الملف غالبًا.
- طابق case الاسم والمسار لأن Linux حساس للحروف.
- لا تربط domain namespaces باسم framework بلا ضرورة.
- استخدم `App\Tests` أو `autoload-dev` للاختبارات.
- لا تعتمد على تنفيذ side effects عند تحميل ملف class.



تنبيه

Namespace ليست filesystem path إجباريًا في اللغة؛ PSR-4 هو الاتفاق الذي يربطهما عبر Composer.

مرجع

- [PHP Namespaces](#)
- [Composer PSR-4](#)

11. الأخطاء والاستثناءات

Exception و Error و Throwable و try/catch/finally والاستثناءات المخصصة وحدود المعالجة والتسجيل الآمن.

Error أم Exception؟

كل من **Exception** و **Error** يطبقان **Throwable**. الاستثناء يمثل غالبًا فشلًا متوقعًا في العملية، بينما **Error** يشمل أخطاء لغة/نوع وتشغيل لا ينبغي تحويلها كلها إلى "نجاح".

```
try {
    $receipt = $payments->charge($order);
} catch (PaymentDeclined $e) {
    // فشل مجال متوقع
} catch (Throwable $e) {
    // حد التطبيق: سجل ثم حوّل لاستجابة عامة
} finally {
    $lock?->release();
}
```

finally ينفذ سواء نجح المسار أو وُثمي exception، لذلك يناسب تحرير resource. لا تستخدم catch فارغًا.

استثناءات المجال

```
final class InsufficientStock extends DomainException
{
    public function __construct(public readonly int $productId)
    {
        parent::__construct('Insufficient stock');
    }
}
```

اجعل النوع يحمل معنى يمكن للطبقة العليا ترجمته إلى **409** أو رسالة مناسبة. لا تستخدم نص الرسالة لاتخاذ قرار برمجي.

Error reporting

في التطوير:

```
display_errors=On
error_reporting=E_ALL
```

في الإنتاج:

```
display_errors=Off
log_errors=On
error_reporting=E_ALL
```

إظهار stack trace للمستخدم قد يكشف paths وأسرارًا وSQL. أعطِ المستخدم رسالة عامة وrequest ID، وسجّل التفاصيل في قناة محمية.

Global boundary

```
set_exception_handler(function (Throwable $e): void {
    $requestId = bin2hex(random_bytes(8));
    error_log("[{$requestId}] {$e}");

    if (!headers_sent()) {
        http_response_code(500);
        header('Content-Type: application/json');
    }

    echo json_encode(['error' => 'Internal error', 'request_id' => $requestId]);
});
```

الhandler شبكة أمان، وليس بديلاً عن معالجة الفشل المتوقع قرب سياقه.

قواعد

- لا تستخدم @ لإخفاء الأخطاء.
- لا تعرض رسالة exception الخام للعميل.
- احتفظ بـ `previous` عند wrapping.
- لا تسجل password أو token أو body كاملاً بلا تنقية.
- أعد المحاولة فقط للأخطاء المؤقتة وبعملية idempotent.

12. المصفوفات وأدوات التحويل

Lists و Maps و destructuring و unpacking و map و filter و reduce والفرز والأداء.

PHP array أكثر من نوع

array في PHP خريطة مرتبة تستخدم كـ list أو map. افحص شكلها عند حدود التطبيق بدل افتراض أن المفاتيح متتالية.

```
$ids = [10, 20, 30];
$user = ['id' => 7, 'name' => 'Omar'];

[$first, $second] = $ids;
['id' => $id, 'name' => $name] = $user;
```

array_is_list() يفرق list ذات مفاتيح 0..n-1 عن map. حذف عنصر من list لا يعيد الفهرسة تلقائيًا؛ استخدم array_values() عند الحاجة، خصوصًا قبل JSON.

map و filter و reduce

```
$paidTotals = array_map(
    static fn (array $order): int => $order['total_cents'],
    array_filter(
        $orders,
        static fn (array $order): bool => $order['status'] === 'paid',
    ),
);

$sum = array_reduce(
    $paidTotals,
    static fn (int $carry, int $total): int => $carry + $total,
    0,
);
```

هذه الأدوات مناسبة لتحويلات واضحة. loop عادية أفضل أحيانًا إذا احتجت عدة نتائج أو early exit أو أردت تجنب arrays وسيطة.

```
$defaults = ['timeout' => 3, 'retries' => 1];
$config = [...$defaults, ...$environment];

function sum(int ...$numbers): int {
    return array_sum($numbers);
}

$total = sum(...[2, 4, 6]);
```

في المفاتيح النصية تفوز القيمة اللاحقة عند unpacking. أما `+` بين arrays فيحافظ على قيمة الطرف الأيسر للمفتاح الموجود؛ لا تخطئهما بلا قصد.

الفرز

- `sort()/rsort()` يعيدان فهرسة القيم.
- `asort()/arsort()` يحافظان على المفاتيح.
- `ksort()/krsort()` يرتبان المفاتيح.
- `usort()` يستخدم comparator يعيد سالبًا/صفرًا/موجبًا.

```
usort($orders, static fn ($a, $b) =>
    [$b['created_at'], $b['id']] <=> [$a['created_at'], $a['id']]
);
```

الذاكرة

PHP arrays مرنة لكنها أثقل من packed binary structures. لا تستخدم `array_map` و `fetchAll` على ملايين العناصر. استخدم Generator أو pagination أو processing streaming.

13. النصوص وUnicode وRegular Expressions

Bytes وUTF-8 وmbstring والتطبيع والمقارنة والRegex الآمن في PHP.

String في PHP

PHP string سلسلة bytes، وليست قائمة Unicode code points. لذلك:

```
$text = 'مرحبًا';
echo strlen($text);           // bytes
echo mb_strlen($text, 'UTF-8'); // characters تقريبًا
```

استخدم UTF-8 عبر التطبيق وقاعدة البيانات وHTTP:

```
header('Content-Type: text/html; charset=utf-8');
```

mb_strlen و mb_substr و mb_strtolower أنسب للنص متعدد اللغات من نسخ byte-oriented.

Graphemes والتطبيع

الحرف المرئي قد يتكون من أكثر من code point. امتداد intl يوفر أدوات grapheme و Normalizer:

```
$normalized = Normalizer::normalize($input, Normalizer::FORM_C);
```

طبّع عند الحاجة الواضحة مثل البحث أو uniqueness، ولا تغيّر النص الأصلي بلا متطلب. Case folding والقواعد اللغوية أعقد من strtolower.

Formatting آمن

- استخدم interpolation أو sprintf للعرض، لا لبناء SQL.
- قارن secrets بـ hash_equals() لا === عند الحاجة لمقارنة ثابتة الزمن.
- استخدم htmlspecialchars عند إخراج نص في HTML، و rawurlencode لمعامل URL.
- لا توجد "دالة تعقيم عامة" لكل السياقات.

Regular Expressions

```
$ok = preg_match('/\A[A-Z]{2}-\d{6}\z/D', $code) === 1;
```

- استخدم anchors واضحة.
- افحص `preg_last_error_msg()` عند الفشل.
- ضع حدودًا لطول input قبل regex معقدة.
- تجنب backtracking كارثي في patterns على نص غير موثوق.
- لا تستخدم regex لتحليل HTML أو JSON عندما يوجد parser.

مثال استخراج

```
if (preg_match('/\A(?<country>[A-Z]{2})-(?<number>\d{6})\z/D', $code, $m)) {  
    $country = $m['country'];  
    $number = $m['number'];  
}
```

بعد مطابقة الشكل طبق قواعد المجال؛ الشكل الصحيح لا يعني أن القيمة مسموحة.

14. التاريخ والوقت والمناطق الزمنية

DateTimeImmutable و UTC و timezones و DST و Parsing و Intervals والتخزين الصحيح.

لحظة أم وقت محلي؟

فرّق بين:

- **Instant**: نقطة عالمية على timeline؛ خزّنها غالبًا UTC.
 - **Local date/time**: مثل موعد متجر 09:00 في القاهرة.
 - **Timezone**: قواعد منطقة مثل `Africa/Cairo` وتشمل تغييرات DST التاريخية.
 - **Duration/Interval**: مدة وليست تاريخًا.
- Offset مثل `+02:00` ليس بديلًا عن اسم timezone؛ القواعد قد تتغير.

DateTimeImmutable

```
$now = new DateTimeImmutable('now', new DateTimeZone('UTC'));
$cairo = $now->setTimezone(new DateTimeZone('Africa/Cairo'));

echo $now->format(DateTimeInterface::ATOM);
echo $cairo->format('Y-m-d H:i:s P');
```

فضّل `DateTimeImmutable` حتى تعيد العمليات object جديدة ولا تتغيّر قيمة يشاركها كود آخر.

Parsing صارم

```
$date = DateTimeImmutable::createFromFormat(
    '!Y-m-d',
    $input,
    new DateTimeZone('Africa/Cairo'),
);
$errors = DateTimeImmutable::getLastErrors();

if ($date === false || ($errors !== false &&
    ($errors['warning_count'] > 0 || $errors['error_count'] > 0))) {
    throw new InvalidArgumentException('Invalid date');
}
```

لا تعتمد على parser المرن لمدخل مستخدم يحتاج format محددًا؛ قد "يصحح" تاريخًا غير موجود.

```
$tomorrow = $now->add(new DateInterval('P1D'));  
$after24Hours = $now->add(new DateInterval('PT24H'));
```

“اليوم التالي في الساعة نفسها” قد يختلف عن “بعد 24 ساعة” حول DST. حدد معنى المجال.

التخزين والعرض

- خزن instant بصيغة/نوع يحفظ UTC بدقة.
 - خزن timezone الأصلية إذا كان الموعد المستقبلي مرتبطًا بالوقت المحلي.
 - حوّل إلى منطقة المستخدم عند العرض.
 - لا تستخدم timezone الافتراضية الضمنية في business logic.
 - اختبر نهاية الشهر والسنة و leap day وتغييرات DST.
- استخدم clock قابلة للحقن في الاختبارات بدل استدعاء “الآن” داخل كل class.

15. من HTTP Request إلى Router وResponse

Front Controller وJSON وRouting وMiddleware وبناء استجابة HTTP صحيحة دون Framework.

Front Controller

اجعل خادم الويب يمرر الطلبات الديناميكية إلى `public/index.php`:

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
```

لا تجعل document root هو جذر المشروع؛ يجب ألا يصل العميل إلى `vendor/` أو `.env` أو source files.

قراءة body

```
$contentType = strtolower(trim(explode('; ', $_SERVER['CONTENT_TYPE'] ?? '')[0]));

if ($contentType !== 'application/json') {
    respond(['error' => 'Unsupported media type'], 415);
}

try {
    $payload = json_decode(
        file_get_contents('php://input'),
        true,
        64,
        JSON_THROW_ON_ERROR,
    );
} catch (JsonException) {
    respond(['error' => 'Invalid JSON'], 400);
}
```

ضع حدًا لحجم body في Web Server والتطبيق. Parsing لا يغني عن validation.

```
$handler = match ([$method, $path]) {  
    ['GET', '/health'] => static fn () => respond(['status' => 'ok']),  
    ['POST', '/api/orders'] => $createOrder,  
    default => null,  
};  
  
if ($handler === null) {  
    respond(['error' => 'Not found'], 404);  
}  
  
$handler();
```

Router حقيقية تحتاج parameters و method mismatch و URL decoding. الهدف فهم المسؤوليات قبل framework.

Response

```
function respond(array $body, int $status = 200): never  
{  
    http_response_code($status);  
    header('Content-Type: application/json; charset=utf-8');  
    header('Cache-Control: no-store');  
    echo json_encode($body, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);  
    exit;  
}
```

يجب إرسال headers قبل body. لا تخط `echo` عشوائية مع response object.

Middleware pipeline

```
request ID -> trusted proxy -> body limit -> routing  
-> authentication -> authorization -> validation  
-> handler -> error mapping -> response
```

كل middleware يجب أن تكون مسؤوليتها محددة. Logging و CORS ومعالجة الأخطاء قد تحتاج تغليف المسار كله.

16. ميزات PHP الحديثة 8.4 و 8.5

.Clone With و URI و Pipe و Lazy Objects و Asymmetric Visibility و Property Hooks و Reflection و Attributes

اكتب الحد الأدنى للإصدار

لا تستخدم ميزة جديدة بلا إعلان requirement في `composer.json` واختبار بيئة النشر:

```
{
  "require": {
    "php": "^8.4 || ^8.5"
  }
}
```

استخدم `composer check-platform-reqs` أثناء النشر. الأمثلة التالية مميزة بالإصدار وليست كلها متاحة في PHP الأقدم.

+Reflection — PHP 8 و Attributes

```
#[Attribute(Attribute::TARGET_METHOD)]
final readonly class RequiresRole
{
    public function __construct(public string $role) {}
}

#[RequiresRole('admin')]
function deleteUser(int $id): void {}

$attribute = (new ReflectionFunction('deleteUser'))
->getAttributes(RequiresRole::class)[0] ?? null;
```

Attributes metadata منظمة: لا تنفذ الحماية وحدها. framework أو كودك يجب أن يقرأها ويطبقها.

Asymmetric Visibility — PHP 8.4 و Property Hooks

```
final class User
{
    public private(set) string $email {
        set => filter_var($value, FILTER_VALIDATE_EMAIL)
            ? strtolower($value)
            : throw new InvalidArgumentException('Invalid email');
    }
}
```

hook تضيف سلوك get/set، والasymmetric visibility تحدد من يقرأ ومن يكتب. لا تحول كل property إلى منطق مخفي؛ method مسماة أفضل للعملية المعقدة.

Lazy Objects — PHP 8.4

Reflection تدعم lazy ghost و lazy proxy لتأخير initialization حتى ملاحظة الحالة. الاستخدام الأساسي داخل DI containers وORMs؛ لا تبن proxy خاصة قبل الحاجة وفهم identity و serialization.

Pipe Operator — PHP 8.5

```
$slug = $title
|> trim(...)
|> mb_strtolower(...)
|> (fn (string $v): string => str_replace(' ', '-', $v));
```

كل مرحلة callable تستقبل نتيجة السابقة كوسيط واحد. لا تستخدم pipe لسلسلة side effects غامضة.

Clone With — PHP 8.5 و URI Extension

URI extension توفر parsing وفق RFC 3986 و WHATWG بدل حلول string يدوية. Clone With تسهل نسخ value object مع تعديل properties:

```
$published = clone($draft, ['status' => Status::Published]);
```

حافظ على invariants عبر hooks/constructors؛ سهولة النسخ لا تبرر حالة غير صالحة.

إضافات 8.5

- `array_first()` و `array_last()`.
- `#[NoDiscard]` للتنبيه عند تجاهل return value.
- attributes على constants.
- تحسينات cloning و asymmetric visibility.
- `setcookie()` تدعم خيار `partitioned`.

راجع migration guide قبل الترقية، شغل الاختبارات والتحليل الساكن، ولا تعتمد على رقم الإصدار وحده.

مراجع

• PHP 8.4

تشغيل PHP والإنتاج

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

تشغيل PHP وأدوات الإنتاج

Composer و cURL و OPcache وإدارة الذاكرة و PHP-FPM و خوادم الويب ومتغيرات البيئة.

يحوّل هذا القسم الصفحات 78-86 من الملاحظات إلى مسار متكامل يشرح الأدوات والطبقات التي تحيط بتطبيق PHP بعد كتابة الكود. صحننا المعلومات المرتبطة بمحرك PHP الحديث، وفصلنا بين مسؤولية مدير الحزم و عميل HTTP و خادم الويب و PHP-FPM.

لماذا هو قسم مستقل؟

هذه الموضوعات ليست جزءًا من OOP، وليست مجرد صياغة للغة PHP. إنها طبقة **Runtime & Production**: تثبيت الاعتماديات، تحميل الأصناف، تنفيذ PHP بكفاءة، الاتصال بخدمات خارجية، إدارة العمليات، وتمرير الإعدادات بأمان.

خريطة الدروس

1. Composer وإدارة الاعتماديات — `composer.json` و `composer.lock` و `install` و `update` و PSR-4.
2. cURL وعملاء libcurl — HTTP وامتداد PHP و Guzzle والمهلات والتحقق من TLS.
3. OPcache و Opcode cache — Preloading، الإعداد للإنتاج، القياس، وحدود Preloading.
4. ذاكرة PHP و zval — Garbage Collection و Reference Counting و Copy-on-write والدورات المرجعية.
5. PHP-FPM وإدارة العمليات — Master و Workers و Pools وأنماط `static` و `dynamic` و `ondemand`.
6. Apache و Nginx و FastCGI — مسار الطلب و MPM و Event Loop والاتصال بـ FPM.
7. متغيرات البيئة وإدارة الإعدادات — `getenv()` و `$_ENV` و `.env` والأسرار.
8. PHP CLI و `php.ini` والامتدادات — SAPIs واكتشاف الإعداد و `extensions`.
9. Logging و PSR-3 و Observability و structured logs و metrics و traces.
10. الاختبارات وجودة الكود — PHPUnit و doubles و coverage والتحليل الساكن.
11. Data Caching و Cache-aside و Redis و TTL و invalidation و stampede.
12. Queues و Idempotency و Workers و retry و DLQ و scheduling.
13. Deployment و CI/CD و artifacts و Containers و health checks و rollout و rollback.
14. أمان الاعتماديات — Composer audit والسياسات و plugins و CI.



الخط الذي يربط الدروس

المتصفح يتصل بخادم الويب، والخادم يمرر طلب PHP إلى FPM، والـ Worker يشغل كودًا حملته Composer ويستفيد من OPcache، وقد يستدعي خدمة خارجية عبر cURL. الإعدادات والأسرار تصل من البيئة، والذاكرة تُدار داخل كل Worker.

Client

```
-> Nginx / Apache (TLS, static files, routing)
-> FastCGI
-> PHP-FPM pool
-> Composer autoloader + OPcache
-> Application / Database / External API
<- HTTP response
```

الأمثلة تستهدف PHP 8.x. إعدادات الخادم أمثلة تعليمية ويجب تكييف المستخدمين والمسارات والسعة مع نظام التشغيل والموارد الفعلية.

مراجع رسمية

- الاستخدام الأساسي لـ Composer وتحسين Autoloader
- توثيق cURL و libcurl
- OPcache و Preloading
- PHP Garbage Collection في PHP
- PHP-FPM
- Nginx FastCGI و Apache MPM
- `getenv()` و `$_ENV`

1. Composer وإدارة الاعتماديات

composer.json و composer.lock و install و update و require-dev و PSR-4 وتجهيز الإنتاج.

Composer أداة تدير مكتبات المشروع وإصداراتها وتولد Autoloader. هو لا يثبت "PHP نفسها"، بل يحل القيود الموجودة في `composer.json` ويضع الحزم غالبًا داخل `vendor/`.

الملفان الأساسيان

- `composer.json`: ما يحتاجه المشروع والقيود المسموحة والإعدادات و `scripts`.
- `composer.lock`: الإصدارات الدقيقة التي حُلّت بالفعل، مع `metadata` و `hashes` تجعل التثبيت قابلاً للتكرار.

```
{
  "require": {
    "php": "^8.3",
    "guzzlehttp/guzzle": "^7.9"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```



تصحيح مهم

`composer.lock` لا "يتابع ما حدث" فقط؛ في التطبيقات يجب رفعه إلى Git لأنه يثبت الإصدارات الدقيقة. أما المكتبة المنشورة للآخرين فالمستهلك يحل اعتمادياتها ضمن مشروعه، لذلك `lock` الخاص بالمكتبة لا يتحكم في تثبيتته.

install أم update؟

الأمر	مع وجود lock	النتيجة
<code>composer install</code>	يقرأ الإصدارات الدقيقة	تثبيت متكرر ومتوقع، وهو المناسب للـ CI والإنتاج
<code>composer update</code>	يعيد حل القيود	يغيّر <code>composer.lock</code> إلى أحدث إصدارات مسموحة
<code>composer update vendor/package</code>	تحديث محدود	يقلل نطاق التغيير لكنه قد يحدّث اعتماديات مرتبطة

```
composer install
composer require monolog/monolog
composer require --dev phpunit/phpunit
composer update guzzlehttp/guzzle --with-all-dependencies
```

`composer require` يعدّل `composer.json` ويحدّث الـ lock ويثبت الحزمة عادة. الخيار `--dev` يضعها في `require-dev`.

Autoloading وPSR-4

بعد تحديد Namespace إلى directory يشغّل Composer:

```
composer dump-autoload
```

ثم يكفي تحميل ملف واحد عند نقطة الدخول:

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$service = new App\Billing\InvoiceService();
```

في PSR-4 يجب أن يتوافق الاسم `App\Billing\InvoiceService` عادة مع:

```
src/Billing/InvoiceService.php
```

Linux حساس لحالة الأحرف؛ خطأً مثل `invoiceService.php` قد يعمل محلياً على Windows ثم يفشل في الإنتاج.

```
composer validate --strict
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
composer audit
```

- `--no-dev` يستبعد أدوات التطوير، لكنه لا يعني أن `require-dev` غير مهم أثناء الاختبار.
- `--optimize-autoloader` يحوّل قواعد PSR إلى class map محسّنة؛ استخدمه في الإنتاج لا أثناء التطوير المتغير بسرعة.
- يمكن استعمال `--classmap-authoritative` بعد الاختبار، لكنه يكسر الأصناف التي تُولّد وقت التشغيل إذا لم تدخل الخريطة.
- لا تستخدم `--ignore-platform-reqs` كحل دائم؛ قد تثبّت كودًا لا يدعمه إصدار PHP أو Extension على الخادم.
- راجع Composer scripts/plugins لأنها كود قابل للتنفيذ أثناء التثبيت. لا تثبّت مشروعًا غير موثوق بصلاحيات عالية.

سير عمل موصى به

1. أضف أو حدّث الحزم في فرع تطوير.
2. راجع فرق `composer.json` و `composer.lock`.
3. شغّل الاختبارات و `composer audit`.
4. ارفع الملفين إلى Git، ولا ترفع `vendor/` غالبًا.
5. في النشر شغّل `composer install` لا `update`.

بهذا تكون عملية اختيار الإصدارات منفصلة عن عملية نشرها، فلا يفاجئك إصدار جديد أثناء الإنتاج.

2. cURL وعملاء HTTP

libcurl وامتداد PHP Guzzle والمهلات و TLS وإرسال JSON ومعالجة الأخطاء.

مشروع cURL ينتج أداتين مختلفتين:

- **curl**: برنامج لسطر الأوامر.
- **libcurl**: مكتبة لنقل البيانات عبر بروتوكولات متعددة.

امتداد **ext-curl** في PHP يربط PHP بـ libcurl. وهو **عميل** يبدأ اتصالات بخوادم أخرى؛ ليس خادم ويب ولا Socket server.

دورة الطلب في PHP

```
<?php
declare(strict_types=1);

$handle = curl_init('https://api.example.com/v1/users/42');

curl_setopt_array($handle, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Accept: application/json',
        'User-Agent: OmarDevNotes/1.0',
    ],
    CURLOPT_CONNECTTIMEOUT => 3,
    CURLOPT_TIMEOUT => 10,
    CURLOPT_FOLLOWLOCATION => false,
]);

$body = curl_exec($handle);

if ($body === false) {
    $message = curl_error($handle);
    $code = curl_errno($handle);
    curl_close($handle);
    throw new RuntimeException("Network error {$code}: {$message}");
}

$status = curl_getinfo($handle, CURLINFO_RESPONSE_CODE);
curl_close($handle);

if ($status < 200 || $status >= 300) {
    throw new RuntimeException("Unexpected HTTP status: {$status}");
}

$data = json_decode($body, true, flags: JSON_THROW_ON_ERROR);
```

افصل دائماً بين:

1. خطأ النقل: DNS أو اتصال أو timeout أو TLS؛ يظهر من **curl_exec()**.
2. استجابة HTTP فاشلة: مثل 404 أو 500؛ الاتصال نجح لكن حالة الاستجابة ليست نجاحاً.

3. محتوى غير صالح: مثل JSON تالف؛ عالجّه عند فك الترميز.

إرسال JSON

```
$payload = json_encode(
    ['email' => 'user@example.com', 'active' => true],
    JSON_THROW_ON_ERROR
);

$handle = curl_init('https://api.example.com/v1/users');
curl_setopt_array($handle, [
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => $payload,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/json',
        'Accept: application/json',
        'Authorization: Bearer ' . $token,
    ],
    CURLOPT_CONNECTTIMEOUT_MS => 1000,
    CURLOPT_TIMEOUT_MS => 5000,
]);
```

لا تسجّل **Authorization** أو Cookies أو أجسامًا تحتوي بيانات شخصية.

Redirects و TLS



لا تعطّل التحقق

لا تستخدم **CURLOPT_SSL_VERIFYPEER => false** أو **CURLOPT_SSL_VERIFYHOST => 0** في الإنتاج. أصلح CA bundle أو إعداد النظام؛ تعطيل التحقق يجعل الاتصال عرضة لـ Man-in-the-Middle.

CURLOPT_FOLLOWLOCATION قد يرسل الطلب إلى وجهة أخرى. ضع حدًا بـ **CURLOPT_MAXREDIRS**، ولا تسمح للمستخدم بتحديد URL حُرًا في خدمة داخلية وإلا قد تنشئ SSRF. تحقّق من scheme والـ host، وامنع عناوين الشبكة الداخلية و metadata endpoints حسب بيئتك.

PSR و Guzzle

Guzzle يقدم API أعلى مستوى ويدعم middleware و promises و pooling. لكنه ليس "cURL بواجهة جميلة" بشكل مطلق؛ يختار Handler مناسبًا وقد يستخدم cURL أو PHP streams حسب البيئة.

```

use GuzzleHttp\Client;

$client = new Client([
    'base_uri' => 'https://api.example.com/',
    'connect_timeout' => 3,
    'timeout' => 10,
]);

$response = $client->get('v1/users/42', [
    'headers' => ['Accept' => 'application/json'],
]);

$data = json_decode(
    (string) $response->getBody(),
    true,
    flags: JSON_THROW_ON_ERROR
);

```

عقود PSR-7 تمثل Request/Response، وPSR-18 يعرّف واجهة عميل HTTP. الاعتماد على Interface يجعل اختبار الكود وتبديل العميل أسهل.

المهلات وإعادة المحاولة

- اضبط مهلة اتصال ومدة كلية؛ الافتراضات غير المحدودة قد تحتجز FPM worker.
- أعد المحاولة فقط للأخطاء المؤقتة، مع exponential backoff و jitter.
- لا تُعد POST غير idempotent تلقائيًا إلا بوجود idempotency key يدعمه الخادم.
- احترم **Retry-After** مع 429/503.
- ضع حدًا لحجم الاستجابة واستخدم streaming للملفات الكبيرة.

cURL Multi

`curl_multi_*` يسمح بعدة عمليات نقل متزامنة دون thread لكل طلب. يفيد عندما تكون الاستدعاءات مستقلة، لكن يجب ضبط عدد الاتصالات وحدود API؛ التزامن غير المحدود ينقل الاختناق إلى الطرف الآخر.

3. Preloading و OPcache

تخزين PHP bytecode وضبط OPcache ومراقبته والفرق بينه وبين cache البيانات وحدود Preloading.

بدون OPcache يقرأ PHP الملف ويحلله ويحوّله إلى Opcodes عند التنفيذ. OPcache يحتفظ بالـ bytecode المترجم في ذاكرة مشتركة لتستفيد منه الطلبات التالية، فيقل وقت parsing و compilation.

```
PHP source -> tokenize/parse -> compile to opcodes -> execute
|
+--> OPcache shared memory
```



ما لا يفعله OPcache

OPcache لا يخزن HTML النهائي أو نتائج SQL أو استجابة API. هذه مسؤولية HTTP cache أو application cache مثل Redis. كما أنه لا يجعل الخوارزمية البطيئة سريعة بعد بدء التنفيذ.

إعداد إنتاجي مبدئي

```
; php.ini / conf.d/opcache.ini
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
```

القيم ليست وصفة ثابتة. راقب عدد الملفات والذاكرة و hit rate و restarts قبل تعديلها.

- مع `validate_timestamps=1` يفحص OPcache تغيير الملفات حسب `revalidate_freq`.
- مع `validate_timestamps=0` لا يرى النشر الجديد حتى تعيد تشغيل FPM أو تستدعي reset بطريقة منضبطة.
- CLI له إعداد مستقل عادة عبر `opcache.enable_cli`، وتشغيله لسكربت قصير قد يضيف تكلفة بلا فائدة.

```
$status = opcache_get_status(false);

if ($status !== false) {
    printf(
        "hit-rate=%.2f%% used=%d free=%d\n",
        $status['opcache_statistics']['opcache_hit_rate'],
        $status['memory_usage']['used_memory'],
        $status['memory_usage']['free_memory'],
    );
}
```

لا تعرض endpoint يحوي `opcache_get_status()` للعامّة؛ قد يكشف paths ومعلومات تشغيلية. اجعله داخليًا ومحميًا. مؤشرات تحتاج انتباهًا:

- `cache_full` أو امتلاء الذاكرة.
- `oom_restarts` و `hash_restarts`.
- انخفاض hit rate بعد warm-up.
- عدد scripts قريب من `max_cached_keys`.

Preloading

من PHP 7.4 يمكن تشغيل ملف مرة عند بدء عملية الخادم لتحميل functions/classes/interfaces/traits في ذاكرة دائمة:

```
opcache.preload=/var/www/app/config/preload.php
opcache.preload_user=www-data
```

```
<?php
declare(strict_types=1);

$files = [
    __DIR__ . '/../src/Domain/Money.php',
    __DIR__ . '/../src/Domain/Order.php',
];

foreach ($files as $file) {
    opcache_compile_file($file);
}
```

التكلفة والمخاطر:

- يرفع الذاكرة الأساسية لكل بيئة تشغيل.
- أي تغيير في الكود المحمّل يحتاج restart؛ لا يكفي invalidation عادي.
- ترتيب الاعتماديات قد يهم إذا استخدمت `require` بدل `opcache_compile_file()`.
- لا يفيد تلقائيًا كل تطبيق؛ قس قبل وبعد.
- Preloading غير مدعوم على Windows وفق دليل PHP، ومناسب أساسًا للعمليات المستمرة مثل FPM.

1. انشر ملفات release جديدة في directory منفصل.
 2. ثبّت الاعتماديات بـ Composer وولّد autoloader محسّنًا.
 3. شغّل الاختبارات وhealth checks.
 4. بدّل symlink/release atomically.
 5. أعد تحميل FPM gracefully حتى ترى workers الكود وpreload الجديد.
- لا تخط بين OPcache وComposer class map: الأول يخزّن bytecode، والثاني يسرّع العثور على الملف الذي يحتوي الصنف. استخدامهما معًا يعالج مرحلتين مختلفتين.

4. ذاكرة PHP و Garbage Collection

zval و Reference Counting و Copy-on-write والمراجع والكائنات والدورات المرجعية وقياس الذاكرة.

محرك Zend يمثل قيمة PHP ببنية داخلية تسمى **zval** تحمل نوع القيمة وبياناتها. بعض البيانات المركبة مثل strings و arrays و objects لها هياكل إضافية محسوبة المراجع.



نصائح للملاحظات القديمة

وصف **zval** بأنه دائمًا **type + value + is_ref + refcount** مناسب تاريخيًا لـ PHP 5، لكنه تبسيط غير دقيق لـ PHP 7/8. في المحرك الحديث قد يكون Reference نفسه **zval** منفصلًا، وتوجد تفاصيل **refcount** داخل البنية المشار إليها بحسب النوع. تعلم السلوك المرئي في PHP ولا تعتمد على layout داخلي ثابت.

Reference Counting

عندما لا يعود شيء يشير إلى قيمة محسوبة المراجع، يمكن تحريرها فورًا عادة. لكن **refcount** ليس "عدد المتغيرات" بصورة مطلقة؛ قد تتدخل temporaries و interning وتحسينات المحرك، لذلك مخرجات debugging قد تبدو أعلى من المتوقع.

```
$first = ['name' => 'Omar'];
$second = $first;

unset($first);
// يستخدمها $second ما زالت البيانات موجودة لأن
```

Copy-on-write

عند الإسناد لا ينسخ PHP array أو string كبيرًا فورًا عادة؛ يشترك المتغيران في البيانات حتى يحاول أحدهما تعديلها:

```
$original = range(1, 100_000);
$copy = $original; // مشاركة مؤقتة
$copy[] = 100_001; // هنا يلزم فصل البيانات المعدلة
```

هذا يؤخر الذاكرة والوقت، لكنه يعني أن تعديل نسخة كبيرة قد يسبب memory spike لحظيًا.

الكائنات تختلف:

```
$a = new stdClass();
$a->count = 1;
$b = $a;
$b->count++;

echo $a->count; // 2
```

`$a` و `$b` يحملان handle للكائن نفسه. للحصول على كائن مستقل استخدم `clone` ونفذ `__clone()` للنسخ العميق عند الحاجة.

References & علامة

```
$value = 10;
$alias =& $value;
$alias = 20;

echo $value; // 20
```

المرجع في PHP alias إلى نفس متغير التخزين، وليس مؤشر C عام يمكن إجراء pointer arithmetic عليه. لا تستخدم `&` لتحسين أداء؛ غالبًا يزيد التعقيد وقد يمنع تحسينات Copy-on-write.

لماذا نحتاج Garbage Collector؟

Reference counting وحده لا يحرر دورة تشير إلى نفسها:

```
final class Node
{
    public ?Node $next = null;
}

$a = new Node();
$b = new Node();
$a->next = $b;
$b->next = $a;

unset($a, $b); // بقيت دورة لا يصل إليها كود المستخدم
```

يسجل GC الحاويات المرشحة ويفحص الدورات دوريًا. يمكن طلب دورة جمع يدويًا:

```
$cycles = gc_collect_cycles();
printf("Collected %d cycles\n", $cycles);
```

لا تستدعها بعد كل عملية. استخدمها في workers طويلة العمر فقط بعد القياس، مثل queue consumer ينشئ graphs كبيرة بين jobs.

الذاكرة في PHP-FPM

في نموذج request/response تُحرر ذاكرة الطلب عادة عند انتهائه، لكن الـ Worker نفسه يبقى حيًا وقد تحتفظ Extensions أو caches أو fragmentation بذاكرة. لذلك `pm.max_requests` مفيد لإعادة تدوير workers دوريًا، لكنه لا يصلح تسريًا منطقيًا داخل عملية طويلة.

القياس

```
$before = memory_get_usage(true);

$rows = loadReportRows();

printf(
    "current=%s peak=%s delta=%s\n",
    number_format(memory_get_usage(true)),
    number_format(memory_get_peak_usage(true)),
    number_format(memory_get_usage(true) - $before),
);
```

`true` يعرض الذاكرة المخصصة من النظام لمحرك PHP، وقد تكون أكبر من الحجم الفعلي المستخدم. قس داخل workload واقعي واستخدم profiler عند الحاجة.

تقليل الاستهلاك

- استخدم Generator أو pagination بدل تحميل ملايين الصفوف.
- حرر المراجع الكبيرة داخل loops طويلة بـ `unset()` عند ثبوت الحاجة.
- تجنب `fetchAll()` للنتائج الضخمة.
- انتبه إلى closures التي تلتقط كائنات كبيرة.
- افصل cache مشتركة مثل OPcache/Redis عن ذاكرة heap الخاصة بالطلب.
- لا ترفع `memory_limit` قبل معرفة سبب الاستهلاك.

5. PHP-FPM وإدارة العمليات

FastCGI و Master و Workers و Pools وأنماط static و dynamic و ondemand وحساب السعة والمراقبة.

PHP-FPM اختصار **FastCGI Process Manager**، وهو التنفيذ الأساسي لـ FastCGI في PHP. يستقبل طلبات FastCGI من خادم موثوق مثل Nginx أو Apache، يدير مجموعة عمليات PHP، ثم يعيد الاستجابة.

Nginx/Apache -> FastCGI socket -> FPM master -> worker -> PHP response



تحذير

لا تعرّض منفذ PHP-FPM لشبكة غير موثوقة. عميل FastCGI يستطيع تمرير إعدادات تؤثر في تنفيذ الطلب؛ اربطه بـ Unix socket أو interface داخلي واضبط الصلاحيات و `listen.allowed_clients` عند TCP.

Workers و Master

- Master process يقرأ إعدادات pools ويدير الإنشاء والإيقاف وإعادة التحميل.
 - Worker process يعالج طلب PHP واحدًا في اللحظة في النموذج المعتاد.
 - Pool يعزل مجموعة workers بإعدادات مثل user/group و socket و php.ini values والحدود.
- يمكن أن تخدم تطبيقين بمستخدمين و sockets منفصلة لتقليل أثر الاختراق، بدل وضع كل المواقع في pool واحدة.

أوضاع Process Manager

static

ينشئ بالضبط workers `pm.max_children` ويحتفظ بها. أداء متوقع ولا يوجد spawn عند الضغط، لكنه يحجز الذاكرة حتى في الهدوء.

```
pm = static
pm.max_children = 20
```

dynamic

يبدأ بعدد workers ويحافظ على spare capacity بين حدين:

```
pm = dynamic
pm.max_children = 40
pm.start_servers = 8
pm.min_spare_servers = 4
pm.max_spare_servers = 12
```

غالبًا اختيار متوازن لخدمة نشطة.

ondemand

ينشئ workers عند وصول الطلبات ويقتل الخامل بعد مدة:

```
pm = ondemand
pm.max_children = 20
pm.process_idle_timeout = 10s
```

يوفر ذاكرة للمواقع قليلة المرور مقابل cold-start وتكلفة spawn.

حساب pm.max_children

ابدأ بقياس RSS الفعلي بعد warm-up:

```
available RAM for FPM / realistic high-percentile worker RSS
```

مثال: إذا خصصت 2 GiB FPMJ وكان worker عند الحمل يستهلك 80 MiB، فالحد النظري 25. اترك هامشًا للنظام وخادم الويب وOPcache وقاعدة البيانات؛ قد تبدأ مثلًا بـ 18-20 ثم تختبر.

لا تقسم `memory_limit` على RAM مباشرة؛ `memory_limit` حد request تقريبي وليس RSS العامل المعتاد، وقد توجد ذاكرة Extensions أو shared memory خارجه.

إعدادات تشغيل مفيدة

```
pm.max_requests = 500
request_terminate_timeout = 30s
request_slowlog_timeout = 3s
slowlog = /var/log/php-fpm/app-slow.log
catch_workers_output = yes
ping.path = /fpm-ping
pm.status_path = /fpm-status
```

- `pm.max_requests` يعيد تدوير worker بعد عدد طلبات، مفيد للحد من نمو طويل الأمد.
- `slowlog` يسجل backtrace للطلبات البطيئة، وليس مجرد URL.
- `status` يعرض `active/idle processes` و `queue` و `max children reached`.
- لا تنشر `ping/status` للعامة؛ اسمح بهما داخليًا فقط.

Little's Law و queue

إذا وصل 100 طلب/ثانية ومتوسط خدمة PHP هو 200ms، فالتزامن المتوسط يقارب:

```
concurrency = throughput × latency = 100 × 0.2 = 20 workers
```

هذا متوسط لا يغطي القمم أو tail latency. إذا كان `max children reached` أو `listen queue` يرتفع، فإما أن السعة قليلة أو التطبيق/قاعدة البيانات بطيئة. زيادة `workers` بلا حدود قد تغرق قاعدة البيانات.

Graceful reload و fastcgi_finish_request

استخدم `reload` من مدير الخدمة في النشر حتى تكمل العمليات الحالية حيث أمكن. وتسمح `fastcgi_finish_request()` بإرسال الاستجابة ثم متابعة عمل قصير، لكنها تبقى `worker` مشغولاً؛ الأعمال الثقيلة أو الموثوقة مكانها `queue worker` مستقل.

6. Apache وNginx وFastCGI

مسؤولية خادم الويب وApache MPM وNginx event loop وربط PHP-FPM عبر Unix أو TCP socket.

خادم الويب يستقبل HTTP/TLS، يقدّم الملفات الثابتة، يطبق routing وحدود الطلب، ويرسل ملفات PHP إلى PHP-FPM runtime. PHP ينفذ؛ لا تجعل كل طلب، بما فيه الصور وCSS، يمر عبر التطبيق بلا داع.

تصحيح مقارنة Apache وNginx

قول "Apache process لكل connection" غير دقيق كقاعدة عامة. Apache يستخدم **MPM** واحدًا:

- **prefork**: عمليات بلا threads؛ مناسب لبعض التوافقات القديمة.
- **worker**: processes: تحتوي threads.
- **event**: threaded: ويعالج keep-alive بكفاءة أكبر.

Nginx يعتمد workers تقود event loop غير متزامنة، فيخدم اتصالات كثيرة بعدد محدود من العمليات. لكن هذا لا يعني أن PHP نفسها تصبح async؛ كل طلب PHP يشغل FPM worker حتى ينتهي.

Apache

يمكن تشغيل PHP تاريخيًا داخل Apache module، لكن فصل Apache عن PHP-FPM عبر FastCGI يعطي عزلاً وإدارة عمليات أوضح.

```
<VirtualHost *:443>
  ServerName example.com
  DocumentRoot /var/www/app/public

  <Directory /var/www/app/public>
    AllowOverride None
    Require all granted
    FallbackResource /index.php
  </Directory>

  <FilesMatch \.php$>
    SetHandler "proxy:unix:/run/php/app.sock|fcgi://localhost/"
  </FilesMatch>
</VirtualHost>
```

.htaccess يسمح بإعدادات موزعة إذا كان **AllowOverride** مفعلاً، لكنه يسبب filesystem checks ويجعل السياسة موزعة. في خادم تتحكم به، ضع القواعد في VirtualHost واضبط **AllowOverride None**. في shared hosting قد يكون **.htaccess** هو الخيار المتاح.

```
server {
    listen 443 ssl;
    server_name example.com;
    root /var/www/app/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        try_files $uri =404;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param HTTP_PROXY "";
        fastcgi_pass unix:/run/php/app.sock;
        fastcgi_read_timeout 30s;
    }

    location ~ /\. {
        deny all;
    }
}
```

النمط الآمن لتطبيق Front Controller هو تمرير `/index.php` فقط عندما لا تحتاج تنفيذ ملفات PHP أخرى. يقلل هذا مساحة الخطأ:

```
location = /index.php {
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root/index.php;
    fastcgi_pass unix:/run/php/app.sock;
}
```

لا تضع `.env` أو `vendor/` أو `uploads` القابلة للتنفيذ داخل `public root`.

Unix socket أم TCP؟

```
fastcgi_pass unix:/run/php/app.sock;
# أو
fastcgi_pass 127.0.0.1:9000;
```

- Unix socket مناسب عندما يكون Nginx و FPM على الجهاز نفسه؛ راجع `owner/group/mode`.
- TCP مطلوب عادة عبر `containers/hosts` منفصلة؛ اربطه بشبكة خاصة وجدار ناري.
- الفرق الأدائي غالبًا أقل أهمية من صحة الإعداد والمراقبة، فلا تختَر بالشعارات.

`SCRIPT_FILENAME` يخبر PHP بالملف المطلوب. خطأ فيه ينتج "Primary script unknown" أو قد يفتح مسارًا غير مقصود. مَرَّر كذلك method و query/content parameters عبر الملف القياسي الملائم لتوزيعتك.

لا تتق في headers يرسلها العميل مثل `X-Forwarded-For` إلا إذا جاءت من reverse proxy موثوق ومحدد. اضبط real-IP/trusted proxies بوضوح حتى لا يزور المستخدم IP أو scheme.

حدود ومسؤوليات

- Web server: TLS, static files, request-size limits, timeouts, buffering
- PHP-FPM: عدد workers, PHP settings, slowlog, status
- Application: validation, authorization, business logic, response
- CDN/load balancer: edge caching, health routing, وقد ينهي TLS.

اجعل timeouts متناسقة: مهلة upstream في خادم الويب يجب ألا تخفي عملية PHP عالقَة بلا حد، و `request_terminate_timeout` في FPM يجب أن يعكس SLA وطبيعة endpoint.

7. متغيرات البيئة وإدارة الإعدادات

getenv و ENV_\$ و putenv و env وتحويل الإعدادات والتحقق منها وحماية الأسرار في PHP-FPM.

متغيرات البيئة قيم يوفّرها نظام التشغيل أو container أو مدير الخدمة للعملية. تستخدم لتغيير إعدادات البيئة دون تعديل source code، مثل DSN وقيمة debug واسم البيئة.

```
OS / container / service manager
-> PHP-FPM master
-> worker process
-> getenv() / $_SERVER / $_ENV
```

getenv و ENV_\$ و putenv

```
$dsn = getenv('DATABASE_DSN');

if ($dsn === false || $dsn === '') {
    throw new RuntimeException('DATABASE_DSN is required');
}
```

افحص `=== false` لأن النص `"0"` قيمة صحيحة لكنه `falsey`.

- `getenv('NAME')` يقرأ متغيرًا ويعيد `false` عند غيابه.
- `$_ENV` نسخة superglobal من القيم التي استوردتها PHP، وقد تكون فارغة إذا كان `variables_order` لا يحتوي `E`.
- `$_SERVER` قد يحتوي بعض قيم البيئة/CGI حسب SAPI.
- `putenv('NAME=value')` يغيّر بيئة العملية للطلب الحالي، ولا يعني أن `$_ENV` سيتزامن تلقائيًا معه.



تصحيح مهم

PHP لا تقرأ ملف `.env` تلقائيًا. تحتاج framework أو مكتبة مثل `vlucas/phpdotenv` أو bootstrap خاص. في الإنتاج قد تُحقن المتغيرات مباشرة من orchestrator أو secret manager دون وجود ملف.

تحميل env. في التطوير


```
composer require vlucas/phpdotenv
```

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(dirname(__DIR__));
$dotenv->safeLoad();
$dotenv->required(['APP_ENV', 'DATABASE_DSN']);
```

```
# .env.example - أسماء وأمثلة غير سرية
APP_ENV=local
APP_DEBUG=false
DATABASE_DSN=mysql:host=127.0.0.1;dbname=app;charset=utf8mb4
```

- ارفع `.env.example` بلا أسرار.
- ضع `.env` في `.gitignore` وخارج `document root`.
- لا تضع قيم `production` الحقيقية في `README` أو `logs` أو `frontend bundle`.
- إذا تسرب سر، احذفه من السجل ودوّره؛ حذف `commit` وحده لا يبطل السر.

طبقة Config واحدة

لا تستدع `getenv()` في كل `class`. حوّل النصوص وتحقق منها عند بدء التطبيق:

```

final readonly class AppConfig
{
    public function __construct(
        public string $environment,
        public bool $debug,
        public string $databaseDsn,
        public int $httpTimeoutMs,
    ) {}

    public static function fromEnvironment(): self
    {
        $dsn = getenv('DATABASE_DSN');
        if ($dsn === false || $dsn === '') {
            throw new RuntimeException('DATABASE_DSN is required');
        }

        $debug = filter_var(
            getenv('APP_DEBUG') ?: 'false',
            FILTER_VALIDATE_BOOL,
            FILTER_NULL_ON_FAILURE,
        );

        if ($debug === null) {
            throw new RuntimeException('APP_DEBUG must be true or false');
        }

        $timeout = filter_var(
            getenv('HTTP_TIMEOUT_MS') ?: '5000',
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 100, 'max_range' => 60_000]],
        );

        if ($timeout === false) {
            throw new RuntimeException('HTTP_TIMEOUT_MS is invalid');
        }

        return new self(
            environment: getenv('APP_ENV') ?: 'production',
            debug: $debug,
            databaseDsn: $dsn,
            httpTimeoutMs: $timeout,
        );
    }
}

```

بهذا يفشل التطبيق مبكرًا بدل اكتشاف إعداد ناقص وسط طلب حقيقي، وتتعامل بقية الطبقات مع أنواع صحيحة.

PHP-FPM والبيئة

FPM قد ينظف البيئة افتراضيًا عبر `clear_env`. مرر قائمة صريحة في إعداد pool أو عبر مدير الخدمة:

```

; app-pool.conf
clear_env = yes
env[APP_ENV] = $APP_ENV
env[DATABASE_DSN] = $DATABASE_DSN

```

الأفضل عدم نسخ البيئة كاملة بلا حاجة. امنح كل pool الحد الأدنى من الأسرار والصلاحيات.

قواعد الأسرار

- Environment variables ليست تشفيرًا؛ يمكن أن تظهر لأدوات مراقبة أو child processes حسب النظام.
- استخدم secret manager عندما تحتاج rotation و auditing وسياسات وصول.
- لا تعرض `phpinfo()` أو dump كاملاً لـ `$_ENV` في الإنتاج.
- افصل secrets عن الإعدادات العامة، ودور كلمات المرور والمفاتيح دوريًا.
- لا ترسل سرًا إلى browser؛ ما يصل إلى JavaScript لم يعد server secret.

ترتيب الأولوية

حدد سياسة واحدة واضحة، مثل:

```
hard-coded safe defaults
<- committed non-secret config
<- environment-specific values
<- secret manager
```

لا تجعل ترتيب الدمج ضمنيًا؛ سجّل مصدر الإعداد وأسماء المفاتيح عند التشخيص، دون تسجيل القيم السرية.

8. PHP CLI و php.ini والامتدادات

اختلاف SAPIs واكتشاف الإعدادات وإدارة extensions وضبط التطوير والإنتاج.

SAPI تحدد بيئة التشغيل

PHP قد تعمل عبر CLI أو FPM/FastCGI أو Apache module. نفس الكود قد يقرأ `php.ini` مختلفة ويملك extensions وإعدادات مختلفة.

```
php -v
php --ini
php -m
php -i
php -r "echo PHP_SAPI, PHP_EOL;"
```

عند ظهور "يعمل في terminal ولا يعمل في الموقع"، قارن executable و SAPI وملف الإعداد والمستخدم والبيئة.

ترتيب الإعداد

`php --ini` يعرض الملف الأساسي ومجلد `conf.d`. الملفات الإضافية قد تعدّل قيمة سابقة. داخل التطبيق:

```
printf(
    "sapi=%s ini=%s memory=%s\n",
    PHP_SAPI,
    php_ini_loaded_file() ?: 'none',
    ini_get('memory_limit'),
);
```

بعض الإعدادات `PHP_INI_SYSTEM` ولا يمكن تغييرها بـ `ini_set()` أثناء الطلب. لا تجعل bootstrap يخفي configuration drift.

Extensions

```
php --ri opcache
php --ri pdo_mysql
composer check-platform-reqs
```

`extension_loaded()` يفيد في فحص واضح، لكن `declare requirements` في Composer أفضل لمنع نشر بيئة ناقصة.

Production مقابل Development

في التطوير: أخطاء ظاهرة، Xdebug عند الحاجة، وOPcache بسياسة مناسبة للتغييرات. في الإنتاج: لا تعرض الأخطاء، فَعّل logs وOPcache، اضبط limits و timeouts، وأزل extensions غير اللازمة.

لا تنسخ `php.ini-development` أو `php.ini-production` بلا مراجعة؛ هما baseline وليس إعداد تطبيقك النهائي.

CLI scripts

```
#!/usr/bin/env php
<?php
declare(strict_types=1);

set_time_limit(0);
$options = getopt('', ['dry-run', 'limit:']);
```

أضف exit codes صحيحة، signal handling للـ lock، logging، workers لمنع نسختين، و timeouts لأي `set_time_limit(0)` I/O. لا يعني أن النظام أو orchestrator لن يقتل العملية.

9. Observability و Logging

PSR-3 و structured logs و request IDs و metrics و traces والتنقية والمراقبة.

ثلاثة أنواع من الإشارة

- **Logs**: أحداث مفصلة قابلة للبحث.
- **Metrics**: أرقام مجمعة عبر الزمن مثل latency و error rate.
- **Traces**: رحلة request عبر services و database و queues.

لا تعالج observability بإضافة `error_log` في كل مكان؛ صمم schema وسياسة retention وتنبيهات.

Structured Logging و PSR-3

```
use Psr\Log\LoggerInterface;

final class Checkout
{
    public function __construct(private LoggerInterface $logger) {}

    public function run(Order $order, string $requestId): void
    {
        $this->logger->info('checkout.started', [
            'request_id' => $requestId,
            'order_id' => $order->id(),
        ]);
    }
}
```

مُزّر البيانات كـ context بدل تركيب نص يصعب تحليله. Monolog تنفيذ شائع لـ PSR-3.

Correlation

أنشئ request ID عند edge أو اقبلها فقط من proxy موثوق، ثم مَزرّها إلى logs و outgoing HTTP و Trace ID و queue metadata. ليست بالضرورة user-visible request ID.

ما لا نسجله

- كلمات المرور و session IDs و access/refresh tokens.
- Authorization/Cookie headers.
- مفاتيح التشفير والأسرار.

- bodies كاملة أو بيانات شخصية بلا حاجة.

طبّق allow-list أو redaction واختبرها. logs نفسها بيانات حساسة وتحتاج access control و integrity و retention.

Metrics مفيدة

استخدم latency histograms و request rate و error rate و saturation. راقب PHP-FPM queue و، `max children reached`، database pool، external API latency، queue lag، cache hit ratio.

تنبيه عملي

التنبيه يجب أن يعكس أثرًا قابلاً للتصرف، مثل ارتفاع نسبة `5xx` أو p95 latency، لا كل exception منفردة. اربط deployment version بالـ logs و metrics لتحديد regression.

مرجع

- [PSR-3 Logger Interface](#)

10. الاختبارات وجودة الكود

PHPUnit و Unit/Integration/Feature tests و Data Providers و Doubles و Coverage والتحليل الساكن والتنسيق.

هرم عملي

- **Unit**: منطق صغير بلا شبكة أو قاعدة بيانات.
 - **Integration**: تعاون حقيقي مع database/filesystem/client adapter.
 - **Feature/HTTP: request**: كاملة عبر التطبيق.
 - **End-to-end**: النظام من منظور المستخدم؛ أقل عددًا وأعلى كلفة.
- لا تحول كل شيء إلى unit test مع mocks. اختبر العقد عند الحدود والسلوك المهم للمستخدم.

PHPUnit

```
composer require --dev phpunit/phpunit
vendor/bin/phpunit
```

```
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class DiscountTest extends TestCase
{
    #[DataProvider('cases')]
    public function testDiscount(int $price, int $percent, int $expected): void
    {
        self::assertSame($expected, discount($price, $percent));
    }

    public static function cases(): iterable
    {
        yield 'none' => [1000, 0, 1000];
        yield 'ten percent' => [1000, 10, 900];
    }
}
```

اختبر exception والحدود والـ side effects، واجعل اسم الاختبار يشرح السلوك.

Test Doubles

- **Stub**: يعيد بيانات مجهزة.
- **Fake**: تنفيذ خفيف يعمل، مثل repository في الذاكرة.

- **Mock**: يتحقق من interaction متوقع.

فضّل fake/stub عند الإمكان. كثرة mocks تربط الاختبار بتفاصيل التنفيذ وتكسر refactoring.

Coverage

Coverage تكشف السطور غير المنفذة لكنها لا تثبت صحة assertions. راقب الفروع والسلوك الحرج، ولا تجعل الوصول إلى 100% هدفًا مستقلًا.

Static analysis و Style

```
vendor/bin/phpstan analyse
vendor/bin/phpunit
vendor/bin/phpcs
```

PHPStan/Psalm تكشف تناقض الأنواع والمسارات المستحيلة. PHPCS أو PHP-CS-Fixer توخّذ التنسيق. شغل الأدوات محليًا وفي CI بنفس الإعداد.

Test data

- اجعل الوقت والعشوائية قابلة للتحكم.
- استخدم transaction/reset لعزل database tests.
- لا تعتمد على ترتيب الاختبارات.
- لا تستخدم بيانات production حقيقية.
- اختبر migration و rollback/forward path حيث يلزم.

مرجع

- [PHPUnit Manual](#)

11. Redis و Data Caching

Cache-aside و TTL و invalidation و stamped و distributed locks والفرق عن Opcache و HTTP cache.

أي Cache؟

- **OPcache:** PHP bytecode
- **HTTP cache:** responses حسب HTTP semantics
- **Application/Data cache:** نتائج queries أو حسابات.
- **Local in-process:** سريع لكنه غير مشترك وقد يختلف بين workers.

Redis أداة شائعة للبيانات المشتركة، لكنه ليس مصدر الحقيقة تلقائيًا.

Cache-aside

```
$key = "product:{id}:v1";
$json = $redis->get($key);

if ($json === false) {
    $product = $repository->find($id);
    $json = json_encode($product, JSON_THROW_ON_ERROR);
    $redis->setex($key, 300, $json);
}

return json_decode($json, true, flags: JSON_THROW_ON_ERROR);
```

عند الكتابة حدث database أولًا ثم احذف/حدث cache وفق استراتيجية واضحة. توقع stale data خلال نافذة محددة.

المفاتيح وTTL

- ضع namespace/version في المفتاح.
- أضف jitter للTTL حتى لا تنتهي آلاف المفاتيح معًا.
- لا تجعل **KEYS *** جزءًا من request path.
- حدّد serialization format وحجمه.
- لا تخزن secret لمجرد أن Redis "داخلية".

Cache stampede

عند انتهاء key مشهورة قد تعيد عدة requests بناءها معًا. حلول:

- lock قصير مع timeout.
- stale-while-revalidate
- probabilistic early refresh
- single-flight داخل العملية.

الlock يجب أن تملك token فريدة وتحررها فقط إن كنت ما زلت المالك. لا تعتبر distributed lock حلاً بسيطاً لكل consistency.

الفشل

حدد هل **optimization** cache يمكن تجاوزها أم dependency أساسية مثل session store. ضع timeouts قصيرة وراقب hit ratio و memory و evictions و latency. لا تجعل سقوط Redis يحول كل traffic فجأة إلى database بلا load protection.

Invalidation

“هناك شيئان صعبان” ليست خطة. اكتب لكل key:

- من ينشئها؟
- ما source of truth؟
- متى تحذف أو تتغير؟
- ما أقصى stale time؟
- ماذا يحدث عند failure؟

12. Scheduling و Workers و Queues

Cron و graceful shutdown و DLQ و backoff و retries و idempotency و at-least-once delivery و Jobs

لماذا Queue؟

انقل العمل البطيء أو القابل لإعادة المحاولة خارج HTTP request: البريد، الصور، التقارير، مزامنة APIs. أعد الاستجابة بعد حفظ نية العمل بصورة موثوقة، لا بعد تشغيل background process عشوائي.

```
{"type": "SendReceipt", "job_id": "job_01J...", "order_id": 42, "attempt": 1}
```

أرسل IDs لا object graphs كاملة، وضع version للـ payload.

Idempotency و Delivery

أنظمة كثيرة تقدم **at-least-once**: قد تصل الرسالة مرتين. صمم handler آمنة:

```
if ($processedJobs->contains($jobId)) {
    return;
}

$handler->handle($orderId);
$processedJobs->record($jobId);
```

يجب أن يكون تسجيل النتيجة و idempotency atomic قدر الإمكان. Transactional Outbox تربط تغيير database بإنشاء الحدث.

Retry

أعد المحاولة للأخطاء المؤقتة فقط:

```
delay = min(cap, base * 2^attempt) + random_jitter
```

لا تعد Validation error أو credential مرفوضة بلا تغيير. بعد حد معين انقل الرسالة إلى Dead-Letter Queue مع سبب وتحقيق وتنبيه.

Worker lifecycle

- timeout لكل job و I/O.
- memory limit وإعادة تدوير العملية.

- SIGTERM يوقف استقبال الجديد وينتهي الحالي ضمن grace period.
 - acknowledge بعد النجاح لا قبله.
 - heartbeat و visibility timeout أطول من job مع renewal عند الحاجة.
- راقب queue depth و age of oldest و processing latency و retry/DLQ rate.

Schedulerg Cron

اجعل المهمة المجدولة idempotent، امنع overlap بقفل له expiry، واستخدم timezone واضحة. الـ scheduler يمكن أن ينشر jobs إلى queue بدل تنفيذ كل العمل في عملية واحدة.

13. Containers و CI/CD و Deployment

containers و rollback و zero-downtime و migrations و health checks و pipeline و Build artifacts.

Artifact قابلة للتكرار

ابن نسخة واحدة ثم رّفها بين البيئات بدل `composer install` مختلف على كل خادم:

```
composer install --no-dev --prefer-dist --no-interaction \
  --optimize-autoloader --classmap-authoritative
composer check-platform-reqs
```

احفظ commit SHA و build ID داخل metadata قابلة للمراقبة، ولا تضع secrets داخل image.

Pipeline

```
lint -> unit tests -> static analysis -> integration tests
-> dependency audit -> build artifact/image -> scan
-> deploy staging -> smoke test -> production -> verify
```

أوقف النشر عند فشل check. لا تجعل approval يدويًا بدلاً عن اختبارات قابلة للتكرار.

Container

- image صغيرة بإصدار PHP و extensions مثبت.
- process رئيسية واضحة و non-root user.
- filesystem read-only حيث يمكن.
- config/secrets وقت التشغيل.
- logs إلى stdout/stderr.
- graceful shutdown و limits.

افصل web و worker لأن لهما lifecycle وتوسّعًا مختلفين.

Health

- **Liveness**: هل العملية عالقة وتحتاج restart؟
- **Readiness**: هل تستطيع استقبال traffic الآن؟

• **Startup:** هل تحتاج وقت warm-up؟

لا تجعل liveness تعتمد على كل external service فتسبب restart storm. readiness يمكن أن تمنع traffic عند dependency أساسية.

Migrations و Zero-downtime

استخدم rolling/blue-green حسب المنصة. اجعل schema changes backward-compatible بنمط Expand/Contract:

1. أضف البنية الجديدة.
2. انشر كودًا يدعم القديم والجديد.
3. نفذ backfill.
4. انقل القراءة.
5. احذف القديم لاحقًا.

أعد تحميل FPM gracefully، ونسق queue workers مع نسخة payload.

Rollback

Rollback ليست فقط إعادة image migration: قد تكون غير قابلة للعكس. جهز feature flags، backups، roll-forward، ومقاييس نجاح تلقائية. اختبر الإجراء قبل incident.

14. أمان الاعتماديات و Supply Chain

Composer audit والسياسات و platform checks و lock files و plugins و scripts والتحقق في CI.

composer.lock عقد البناء

ارفع `composer.lock` للتطبيقات، واستخدم `composer install` في CI والإنتاج. `update` يقرر نسخًا جديدة ويجب أن يحدث في تغيير مراجع ومختبر، لا أثناء النشر.

Audit وسياسة الحزم

```
composer validate --strict
composer audit --locked
composer check-platform-reqs
composer outdated --direct
```

`composer audit` يفحص advisories والحزم المهجورة والسياسات المدعومة. لا تتجاهل advisory بلا سبب موثق ومدة ومراجعة للـ `reachability`.

Scripts و Plugins

Composer plugins و scripts تنفذ code بصلاحيّة المستخدم الذي يشغل Composer:

```
{
  "config": {
    "allow-plugins": {
      "trusted/package": true,
      "**": false
    }
  }
}
```

لا تشغل Composer كـ root على حزم غير موثوقة. `--no-plugins --no-scripts` يقلل التنفيذ لكنه قد يمنع build مطلوبة؛ افهم المشروع.

Version policy

- ضع SemVer constraints مقصودة.
- اختبر أقل وأعلى نسخة مدعومة عندما تكون مكتبة.

- راجع transitive dependencies لا المباشرة فقط.
- حدّث دوريًا بدل قفزة سنوية ضخمة.
- احذف الحزم غير المستخدمة.

CI

احم tokens، ثبّت permissions للworkflow، لا تطبع environment، وثبّت actions/images حيث يمكن. أنشئ SBOM إذا تتطلب البيئة، واحتفظ بسجل artifact ومن بناها.

استجابة لثغرة

حدد هل الكود المتأثر reachable، طبّق update أو mitigation، شغل suite، انشر، وراقب. إذا تسرب secret عبر package/script فدوره؛ إزالة الحزمة لا تبطل السر.

مراجع

04

المسار · 8 درسا

البرمجة كائنية التوجه

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

البرمجة كائنية التوجه

مسار PHP شامل للأصناف والتغليف والوراثة والتجريد والواجهات و Traits و Magic Methods.

يحوّل هذا القسم الصفحات 71-77 من الملاحظات إلى مسار عملي، مع تصحيح التفاصيل المرتبطة بإصدارات PHP وإضافة أمثلة تصميم قابلة للاختبار.

خريطة الدروس

1. الأصناف والكائنات والتغليف — Invariants و Visibility و Methods و Properties.
2. الوراثة وتعدد الأشكال — is-a و Override و final و Composition.
3. Traits وإعادة الاستخدام الأفقي — use و insteadof و as وحل التعارض.
4. التجريد والواجهات — Abstract Classes و Interfaces والعقود.
5. الثوابت و Static و const — Late Static Binding و self:: و static::.
6. Constructors و Readonly و Dependency Injection — حقن الاعتماديات وكائنات قيمة ثابتة.
7. Magic Methods و __get — Cloning و __call و __invoke و __clone و __debugInfo.
8. قائمة مراجعة OOP — أسئلة عملية وتمارين شامل لمراجعة التصميم.



نصيحة

OOP ليست تحويل كل شيء إلى class. الهدف حماية القواعد وتجميع الحالة مع السلوك وتقليل الاعتماد بين الأجزاء.

1. الأصناف والكائنات والتغليف

شرح Class و Object و Properties و Methods و Visibility مع حماية قواعد الكائن.

Object و Class

الـ Class قالب يحدد الحالة والسلوك، والـ Object instance مستقلة منه:

```
final class Product
{
    public function __construct(
        public readonly int $id,
        public string $name,
        private int $priceCents,
    ) {
        if ($priceCents < 0) {
            throw new InvalidArgumentException('Price cannot be negative');
        }
    }

    public function priceCents(): int
    {
        return $this->priceCents;
    }

    public function changePrice(int $newPriceCents): void
    {
        if ($newPriceCents < 0) {
            throw new InvalidArgumentException('Price cannot be negative');
        }

        $this->priceCents = $newPriceCents;
    }
}

$product = new Product(1, 'Keyboard', 150000);
$product->changePrice(145000);
```

كل object له حالته، بينما تعريف methods مشترك منطقياً بين instances.

Encapsulation

التغليف ليس مجرد كتابة `private`. معناه أن الكائن:

- يخفي تفاصيل تمثيله الداخلية.
- يعرض عمليات ذات معنى.
- يمنع الحالة غير الصالحة.
- يستطيع تغيير تنفيذه دون كسر المستدعين.

```
final class BankAccount
{
    public function __construct(private int $balanceCents = 0)
    {
        if ($balanceCents < 0) {
            throw new InvalidArgumentException('Invalid opening balance');
        }
    }

    public function deposit(int $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException('Amount must be positive');
        }
        $this->balanceCents += $amount;
    }

    public function withdraw(int $amount): void
    {
        if ($amount <= 0 || $amount > $this->balanceCents) {
            throw new DomainException('Invalid withdrawal');
        }
        $this->balanceCents -= $amount;
    }

    public function balanceCents(): int
    {
        return $this->balanceCents;
    }
}
```

Setter عام لكل property قد يهدم التغليف؛ سمّ العمليات حسب المجال مثل `withdraw()` بدل `setBalance()`.

Visibility

modifier	داخل class	child class	الخارج
<code>public</code>	نعم	نعم	نعم
<code>protected</code>	نعم	نعم	لا
<code>private</code>	نعم	لا مباشرة	لا

اجعل أقل visibility ممكنة. `protected` يربط الأبناء بتفاصيل الأب، لذلك لا تستخدمه تلقائيًا بدل `private`.

Object operator و this\$

داخل instance method يشير `$this` إلى object الحالي:

```
$this->name;    // property
$this->rename(); // method
```

استخدم `->` للأعضاء غير static. لا يتوفر `$this` داخل static method.

Dynamic properties و Property types

اكتب الأنواع وابدأ الكائن صالحًا بعد constructor. إنشاء properties غير معلنه deprecated في PHP الحديثة لمعظم الأصناف؛ عزفها صراحة أو استخدم بنية مناسبة بدل `$object->unknown = ...`.

Equality مقابل Identity

```
$a = new Product(1, 'Keyboard', 100);
$b = new Product(1, 'Keyboard', 100);

var_dump($a == $b); // properties متساوية
var_dump($a === $b); // false: instance ليست
```

لـ Value Objects عرّف method مثل `equals()` بدل الاعتماد على مقارنة عامة قد تتغير عند إضافة property.

2. الوراثة وتعدد الأشكال

علاقة is-a و extends و override و final والتعامل مع عدة أنواع من خلال عقد واحد.

Inheritance: علاقة is-a

```
abstract class Notification
{
    public function __construct(public readonly string $recipient) {}

    abstract public function send(string $message): void;
}

final class EmailNotification extends Notification
{
    public function send(string $message): void
    {
        echo "Email to {$this->recipient}: {$message}";
    }
}

final class SmsNotification extends Notification
{
    public function send(string $message): void
    {
        echo "SMS to {$this->recipient}: {$message}";
    }
}
```

`EmailNotification is a Notification` علاقة منطقية. إذا لم يستطع child احترام عقد الأب فليست الوراثة مناسبة.

PHP تدعم class inheritance: single لها parent واحدة، لكنها تستطيع تنفيذ عدة interfaces واستخدام عدة traits.

Override

الchild يعيد تنفيذ method مورثة مع توقيع متوافق. لا تغيّر المعنى المتوقع أو تشدد الشروط بطريقة تكسر المستدعي.

```
function notify(Notification $notification, string $message): void
{
    $notification->send($message);
}

notify(new EmailNotification('omar@example.com'), 'Welcome');
notify(new SmsNotification('+201...'), 'Code: 1234');
```

هذه **Polymorphism**: الكود يتعامل مع contract مشترك، والتنفيذ الفعلي يتحدد حسب object.

يمكن استخدام attribute لتأكيد نية override في PHP الحديثة:

```
#[\Override]
public function send(string $message): void
{
    // ...
}
```

final

- **final class** : لا يمكن تمديدتها.
- **final method** : لا يمكن للchild عمل override.
- **final const** : لا يمكن إعادة تعريفها في child.

استخدم **final** عندما يكون الامتداد سيكسر invariants أو لا يمثل extension point مصممًا.

protected و private أثناء الوراثة

الprivate في الأب موجود في جزء الأب من object لكن child لا يصل إليه مباشرة. اعرض protected/public method بدل تحويل كل شيء إلى protected.

Composition بدل inheritance

```
final class OrderService
{
    public function __construct(
        private PaymentGateway $payments,
        private Notifier $notifier,
    ) {}
}
```

Composition تعني **has-a** وتسمح بتبديل التعاون عبر interfaces دون شجرة وراثة صلبة. استخدم inheritance عندما توجد علاقة is-a ثابتة و substitutability صحيحة؛ وإلا فضل composition.

أخطاء تصميم

- Base class ضخمة فيها hooks كثيرة.
- child يرمي exceptions للعمليات الأساسية التي يعد بها الأب.
- الوراثة لإعادة استخدام سطرين فقط.
- فحص **instanceof** طويل بدل polymorphic method.
- تعديل signature بطريقة غير متوافقة.

3. Traits وإعادة الاستخدام الأفقي

تجميع السلوك المشترك وحل التعارض باستخدام insteadof و as وضبط visibility.

لماذا Trait؟

PHP لا تدعم Trait multiple class inheritance. Trait تجميع methods/properties/constants لإدخالها في classes غير مرتبطة بوراثية واحدة. لا يمكن إنشاء instance من Trait.

```
trait HasTimestamps
{
    private ?DateTimeImmutable $updatedAt = null;

    public function touch(): void
    {
        $this->updatedAt = new DateTimeImmutable();
    }

    public function updatedAt(): ?DateTimeImmutable
    {
        return $this->updatedAt;
    }
}

final class Article
{
    use HasTimestamps;
}
```

الـ Trait reuse أفقي أو composition وقت تعريف class، وليست type contract. إذا احتاج المستدعي معرفة capability فاستخدم interface أيضًا.

الأولية

بالنسبة للـ methods:

1. method داخل class الحالية تتغلب على Trait.
2. method من Trait تتغلب على method موروث.
3. تعارض method بين Traitين يحتاج حلًا صريحًا.


```

trait JsonLogger
{
    public function log(string $message): void
    {
        echo json_encode(['message' => $message]);
    }
}

trait TextLogger
{
    public function log(string $message): void
    {
        echo $message;
    }
}

final class Importer
{
    use JsonLogger, TextLogger {
        JsonLogger::log insteadof TextLogger;
        TextLogger::log as logText;
        JsonLogger::log as protected logJson;
    }
}

```

- `insteadof` يختار التنفيذ الفائز.
- `as` يضيف alias أو يغيّر visibility، ولا يحل التعارض وحده.
- إذا لم تحل تعارض الاسم يحدث fatal error.

Trait تفرض متطلبات

```

trait PublishesEvents
{
    abstract protected function aggregateId(): string;

    public function event(string $name): array
    {
        return ['id' => $this->aggregateId(), 'name' => $name];
    }
}

```

class المستخدمة يجب أن تنفذ التوقيع المتوافق.

حدود ومخاطر

- Trait كبيرة قد تخفي dependencies وحالة داخلية.
- تعارض properties/constants يجب أن يكون متوافقاً أو يفشل.
- الوصول إلى static members مباشرة على اسم Trait deprecated؛ استخدم class التي تستعملها.
- من PHP 8.3 يمكن جعل method المستوردة `final` عبر `as final`.
- PHP 8.5 غيّرت ترتيب ربط trait مع parent بالنسبة لتعارض property/constant؛ اختبر السلوك عند الترقية.

استخدم Trait لسلوك صغير متماسك. إذا احتاجت خدمات متعددة أو lifecycle معقدًا فغالبًا object composition أوضح.

4. التجريد والواجهات

Abstract Classes و Methods و Interfaces والعقود والفروق. ومتى نستخدم كل نوع.

Abstraction

التجريد يعرض ما يحتاجه المستدعي ويخفي تفاصيل التنفيذ. في PHP نستخدم abstract classes و interfaces و composition.

Abstract class

لا يمكن إنشاء instance منها. تستطيع الاحتفاظ بحالة و constructor و methods و منفذة و abstract methods:

```
abstract class PaymentMethod
{
    public function __construct(
        protected readonly string $accountId,
    ) {}

    final public function charge(int $amountCents): Receipt
    {
        if ($amountCents <= 0) {
            throw new InvalidArgumentException('Amount must be positive');
        }
        return $this->performCharge($amountCents);
    }

    abstract protected function performCharge(int $amountCents): Receipt;
}
```

الـ abstract method تعلن signature بلا body، وعلى concrete child تنفيذها بتوافق. وجود تنفيذ مشترك وحالة محمية قد يبرر abstract base، لكن لا تجعلها مخزن utilities غير مترابطة.

بدءًا من PHP 8.4 توجد abstract properties بشروط get/set، لكن استخدمها فقط عند استهداف ذلك الإصدار وبعد فهم property hooks؛ method contract أكثر توافقًا ووضوحًا في مكتبات متعددة الإصدارات.

Interface

Interface عقد type بلا فرض inheritance tree:

```

interface PaymentGateway
{
    public function charge(string $customerId, int $amountCents): Receipt;
    public function refund(string $paymentId): void;
}

final class StripeGateway implements PaymentGateway
{
    public function charge(string $customerId, int $amountCents): Receipt
    {
        return new Receipt('pay_123');
    }

    public function refund(string $paymentId): void {}
}

```

class تستطيع تنفيذ عدة Methods. Interfaces. public interface. يمكنها تعريف constants، لكن لا تحولها إلى حقيبة إعدادات.

Interface Segregation

```

interface ChargesPayments
{
    public function charge(string $customerId, int $amount): Receipt;
}

interface RefundsPayments
{
    public function refund(string $paymentId): void;
}

```

العميل يعتمد فقط على capability التي يحتاجها.

النقطة	Abstract class	Interface
constructor و State	نعم	لا كحالة instance
تنفيذ مشترك	نعم	signatures فقط عادة
العدد	extends واحدة	implements متعددة
العلاقة	عائلة قريبة	capability/contract

استخدم interface عند حدود التطبيق وحقن الاعتماديات. استخدم abstract class عندما توجد علاقة قوية وتنفيذ مشترك ثابت، لا لمجرد تجنب تكرار بسيط.

5. الثوابت وStatic وLate Static Binding

Class constants وstatic members والفرق بين self وstatic ومخاطر الحالة العامة.

Class constants

```
final class HttpStatus
{
    public const int OK = 200; // PHP 8.3+
    public const int NOT_FOUND = 404;
}
```

الثابت مخصص للـ class وليس لكل object. يمكن ضبط visibility وكتابة `final const`. إذا كان المشروع يستهدف PHP أقدم من 8.3 احذف نوع `int`.

Static properties وmethods

```
final class Id
{
    private static int $next = 1;

    public static function next(): int
    {
        return self::$next++;
    }
}
```

الـ static property مشتركة على مستوى class/process. لا يوجد `$this` داخل static method، واستدعاء non-static method بصورة static يرمي Error.



تنبيه

Static mutable state قد تصبح global state مخفية وتستمر داخل worker طويل العمر. استخدم object محققاً عندما توجد dependency أو lifecycle.

self مقابل static::

`self::` يرتبط بالـ class التي عُرِّفت فيها `static::` method. يستخدم Late Static Binding ويشير إلى called class.

```

class Document
{
    protected const TYPE = 'document';
    public static function early(): string { return self::TYPE; }
    public static function late(): string { return static::TYPE; }
}

class Invoice extends Document
{
    protected const TYPE = 'invoice';
}

echo Invoice::early(); // document
echo Invoice::late(); // invoice

```

استخدم `static::` عندما صُممت method للامتداد polymorphically، و `self::` عندما تقصد class المعرّفة.

Named constructors

```

class Money
{
    protected function __construct(
        public readonly int $cents,
        public readonly string $currency,
    ) {}

    public static function egp(int $cents): static
    {
        return new static($cents, 'EGP');
    }
}

```

النوع `static` يحافظ على `called class`، لكن `new static` يفرض توافق `constructors` في الأبناء. `Static` مناسبة للثوابت و `pure` `named constructors`، وليست بديلاً تلقائياً لـ `Dependency Injection`.

6. Dependency و Readonly و Constructors Injection

تهيئة object صالح و Constructor Promotion و Readonly و حقن الاعتماديات بدل إنشائها داخليًا.

Constructor يحمي بداية الكائن

```
final class EmailAddress
{
    public function __construct(public readonly string $value)
    {
        if (filter_var($value, FILTER_VALIDATE_EMAIL) === false) {
            throw new InvalidArgumentException('Invalid email');
        }
    }
}
```

بعد نجاح constructor يجب أن يكون object صالحًا. المثال يستخدم Constructor Property Promotion. وإذا عرّف child constructor خاصًا به فلن يستدعي parent constructor تلقائيًا؛ استدع `parent::__construct(...)` عند الحاجة.

readonly

```
final readonly class Money
{
    public function __construct(
        public int $cents,
        public string $currency,
    ) {
        if ($cents < 0) {
            throw new InvalidArgumentException();
        }
    }
}
```

Readonly class من PHP 8.2 تجعل properties المعلنة readonly وتمنع dynamic properties. لا يمكنها static أو untyped properties.



ليست Deep Immutability

إذا احتوت readonly property على object، لا يمكن تبديل المرجع لكن يمكن أن تتغير حالة object الداخلي إن كان mutable.

في PHP 8.4 أصبح نطاق التهيئة الافتراضي للـ readonly property هو `protected(set)` بدل `private-set` الضمني السابق؛ انتبه عند الوراثة والترقية.

Dependency Injection

```
final class OrderService
{
    public function __construct(
        private PaymentGateway $payments,
        private OrderRepository $orders,
        private Clock $clock,
    ) {}

    public function checkout(Order $order): Receipt
    {
        $receipt = $this->payments->charge(
            $order->customerId(),
            $order->totalCents(),
        );
        $this->orders->markPaid($order, $receipt, $this->clock->now());
        return $receipt;
    }
}
```

DI تجعل dependencies ظاهرة وتسمح بـ production و Dependency Injection. fake implementations ليست Container؛ يمكن توصيل objects يدويًا.

- Constructor injection للاعتماد المطلوب طوال عمر object.
- Method parameter لبيانات/اعتماد خاص بعملية واحدة.
- Setter injection قد تترك object ناقصًا.

تجنب Service Locator مثل `Container::get()` داخل منطق المجال لأنه يخفي dependencies.

7. Cloning و Magic Methods

سلوك PHP الخاص عبر `__get` و `__set` و `__call` و `__toString` و `__invoke` و `__clone` و `__debugInfo`.

ما Magic Methods ؟

أسماء تبدأ بـ `__` ويحجزها PHP لاعتراض أحداث معينة. لا تنشئ method مخصصة بهذا النمط. باستثناء `__construct` و `__destruct` و `__clone`، يجب إعلان magic methods كـ `public`.

`__get` و `__set` و `__isset` و `__unset`

```
final class AttributeBag
{
    public function __construct(private array $data = []) {}

    public function __get(string $name): mixed
    {
        if (!array_key_exists($name, $this->data)) {
            throw new OutOfBoundsException($name);
        }
        return $this->data[$name];
    }

    public function __set(string $name, mixed $value): void
    {
        $this->data[$name] = $value;
    }

    public function __isset(string $name): bool
    {
        return isset($this->data[$name]);
    }

    public function __unset(string $name): void
    {
        unset($this->data[$name]);
    }
}
```

تعمل عند property غير متاحة. المرونة تقلل static analysis وتخفي typos؛ الأعضاء الصريحة أفضل في domain models.

`__call` و `__callStatic`

يعملان عند method غير موجودة/غير متاحة:

```
public function __call(string $name, array $arguments): mixed
{
    throw new BadMethodCallException("Unknown method {$name}");
}
```

تُستخدم في proxies و fluent APIs لكن يجب أن تفشل بوضوح.

__toString و __invoke

```
final readonly class OrderId
{
    public function __construct(private string $value) {}
    public function __toString(): string { return $this->value; }
}

final class Slugify
{
    public function __invoke(string $value): string
    {
        return strtolower(trim(str_replace(' ', '-', $value)));
    }
}

$slugify = new Slugify();
echo $slugify(' Hello World ');
```

الكائن القابل للاستدعاء مناسب لـ strategy صغيرة قابلة للحقن.

clone و __clone

shallow copy `clone` افتراضياً؛ nested objects تظل مشتركة:

```
final class Order
{
    public function __construct(public DateTime $createdAt) {}

    public function __clone(): void
    {
        $this->createdAt = clone $this->createdAt;
    }
}
```

لا تستنسخ ORM Entity ذات identity بلا فهم Unit of Work؛ قد تحتاج factory لهوية جديدة.

```
public function __debugInfo(): array
{
    return ['id' => $this->id, 'token' => '[redacted]'];
}
```

يساعد على إخفاء secrets من `var_dump` ، لكنه لا يبرر logging للكائن كاملاً. فضل `__serialize()` و `__unserialize()` عند الحاجة، ولا تعمل `unserialize()` على بيانات غير موثوقة.

لا تعتمد على destructor لعمل business حرج؛ استخدم methods صريحة و `try/finally` .

8. قائمة مراجعة OOP

قائمة أسئلة وتمارين يجمع التغليف والواجهات والتركيب والحقن والاختبار.

قبل اعتماد تصميم اسأل:

- هل لكل class مسؤولية واضحة واسم من مجال المشروع؟
- هل constructor ينتج object صالحًا؟
- هل الـ properties محمية بأقل visibility؟
- هل الوراثة تعبّر عن is-a حقيقية، أم composition أوضح؟
- هل الواجهة صغيرة ويحتاجها المستدعي فعليًا؟
- هل الـ Trait صغيرة ولا تخفي dependencies؟
- هل الـ static state ستعقد الاختبار أو workers طويلة العمر؟
- هل الـ readonly تحمي المرجع فقط أم نحتاج deep immutability؟
- هل Magic Method تحسن API فعليًا أم تخفي typo؟
- هل الاعتماديات صريحة وقابلة للاستبدال في الاختبار؟

تمارين جامع

صمّم Checkout:

1. `Order` يحمي حالته ولا يسمح بالدفع مرتين.
2. `PaymentGateway` interface لها تنفيذ fake للاختبار.
3. `Receipt` و `Money` كـ readonly value objects.
4. `OrderService` يستقبل gateway و repository و clock بالـ DI.
5. أضف notification بالـ composition لا يجعل Order ترث Email.
6. اختبر نجاح الدفع، فشل المنفذ، والمطالب المدفوع مسبقًا.

05

المسار · 19 درسًا

الأمان والهوية والصلاحيات

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

الأمان والمصادقة والصلاحيات

مسار عملي لأمان PHP والجلسات وكلمات المرور والصلاحيات وJWT وOAuth 2.0 وOpenID Connect وSSO.

يغطي هذا القسم الصفحات 60-66 و87-103 من الملاحظات، مع تصحيح النقاط الأمنية وتوسيعها إلى تطبيقات عملية وفق الممارسات الحديثة.

خريطة الدروس

1. حماية الجلسات — Hijacking و Fixation وتغيير المعرّف والكوكي الآمنة.
2. التحقق من المدخلات والفلتر — Validation مقابل Sanitization ودوال Filter.
3. الحماية من Tokens — CSRF و SameSite وطرق التحقق.
4. الحماية من XSS — Output Encoding حسب السياق و HTML Sanitization.
5. كلمات المرور و bcrypt — Hashing و Argon2id والتحقق وإعادة hash.
6. Encryption و HTTP Authentication — المفاتيح و AEAD و Basic/Bearer.
7. المصادقة والصلاحيات عملياً — RBAC والملكية والسياسات والدفاع متعدد الطبقات.
8. JWT والتحقق الآمن — البنية والتوقيع والـ Claims والتحقق وإدارة المفاتيح.
9. Federated Identity و SSO و SAML و OIDC — الفصل بين البروتوكولات واختيار المناسب.
10. مفاهيم OAuth 2.0 والتوكينات — الأدوار والـ Clients و Scopes و Access/Refresh Tokens.
11. OAuth Flows الحديثة — Authorization Code + PKCE و Client Credentials و Device Flow.
12. أمان OAuth و State — OIDC و Nonce و Redirect URIs وتخزين التوكينات والإبطال.
13. دورة حياة الحساب — التسجيل والتأكيد والاسترجاع وإلغاء الجلسات.
14. MFA و TOTP و Passkeys — العوامل و WebAuthn و Recovery و Step-up.
15. API Keys وهوية الأنظمة — التخزين و Scopes والدوران و Service Accounts.
16. ABAC و ReBAC وعزل Tenants — السياسات المتقدمة ومنع IDOR.
17. Rate Limiting ومقاومة الإساءة — الخوارزميات وحماية login وسياسة الفشل.
18. Audit Trail و Security Logging — الأحداث والتنقية والسلامة والتنبيهات.
19. إدارة الأسرار والمفاتيح — Secret managers و rotation و envelope encryption.



قاعدة أساسية

كل ما يأتي من المتصفح أو API أو Cookie أو Header أو قاعدة بيانات قديمة يُعامل كبيانات غير موثوقة حتى يُتحقق منه، ثم يُرمز عند موضع الإخراج المناسب.

- [RFC 7519 - JWT وأفضل ممارساته - RFC 8725](#)
- [OAuth 2.0 - RFC 6749](#) و [Security BCP - RFC 9700](#)
- [PKCE - RFC 7636](#) و [Device Authorization - RFC 8628](#)
- [OpenID Connect Core](#)
- [SAML 2.0 Technical Overview](#)

1. حماية الجلسات

دورة الجلسة الآمنة ومنع Session Fixation و Hijacking وإدارة المعرف والكوكي.

كيف تعمل الجلسة؟

يحفظ الخادم البيانات، ويحمل المتصفح عادةً Cookie تحتوي Session ID عشوائيًا. المعرف بيانات اعتماد؛ من يسرقه قد ينتحل الجلسة حتى لو لم يعرف كلمة المرور.

أهم المخاطر:

- **Session Fixation**: يجبر المهاجم الضحية على استخدام ID يعرفه، ثم ينتظر تسجيل الدخول.
- **Session Hijacking**: يسرق ID فغالبًا عبر XSS أو شبكة غير محمية أو Logs أو جهاز مخترق.
- **Brute force**: محاولة تخمين IDs ضعيفة؛ يجب أن يولدها PHP عشوائيًا ولا تنشئها بـ `uniqid()`.
- **Race conditions**: حذف الجلسة القديمة فورًا أثناء طلبات متزامنة قد يفقد الحالة.

إعداد البداية

```
<?php
declare(strict_types=1);

session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_secure' => true,
    'cookie_httponly' => true,
    'cookie_samesite' => 'Lax',
]);
```

- **Secure**: لا ترسل الكوكي إلا عبر HTTPS.
 - **HttpOnly**: يمنع JavaScript من قراءتها، لكنه لا يمنع XSS من تنفيذ أفعال باسم المستخدم.
 - **SameSite**: يقلل الإرسال cross-site، وليس بديلًا مطلقًا عن CSRF token.
 - **use_strict_mode**: يرفض Session ID غير مهياً من الخادم.
- استدع `session_start()` قبل الإخراج. لا تمرر Session ID في URL لأنه يتسرب في History و Logs و Referer.

تسجيل الدخول وتغيير الصلاحية

بعد إثبات كلمة المرور غير المعرف قبل تثبيت حالة المصادقة الجديدة:


```
if (password_verify($password, $user['password_hash'])) {
    session_regenerate_id();
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['auth_time'] = time();
    $_SESSION['last_seen'] = time();
}
```

غير المعرف عند login، ورفع الصلاحية، وتغيير كلمة المرور أو حدث أمني مهم. لا تغيّره في كل request دون تصميم؛ الطلبات المتوازية قد تتسابق.



تنبيه

استخدام `session_regenerate_id(true)` لحذف التخزين القديم فورًا قد يسبب فقدًا على شبكات غير مستقرة أو مع AJAX المتزامن. الأنظمة الحساسة تستخدم انتقالًا زمنيًا وتمنع استخدام القديم بعد مهلة قصيرة ثم تترك GC يزيله.

انتهاء خمول وعمر كلي

```
$now = time();
$idleLimit = 15 * 60;
$absoluteLimit = 8 * 60 * 60;

if (
    isset($_SESSION['last_seen'], $_SESSION['auth_time'])
    && ($now - $_SESSION['last_seen'] > $idleLimit
        || $now - $_SESSION['auth_time'] > $absoluteLimit)
) {
    logout();
}

$_SESSION['last_seen'] = $now;
```

لا تعتمد على `session.gc_maxlifetime` وحده؛ GC تنظيف تخزين وليس سياسة وصول دقيقة.

Fingerprinting بحذر

يمكن تسجيل User-Agent وتغييرات IP كإشارة مخاطرة، لكن لا تربط الجلسة تطابقًا كاملاً بـ IP؛ شبكات الهاتف وVPN وNAT تتغير. استخدم الإشارات لطلب إعادة المصادقة أو تسجيل تنبيه، لا لطرد المستخدم عشوائيًا.

إغلاق القفل والخروج

```
// بعد تعديل الجلسة، احفظ وأغلق القفل مبكراً  
session_write_close();
```

لتسجيل الخروج: ابدأ الجلسة، امسح `$_SESSION`، احذف Cookie بنفس `path/domain/options`، ثم `session_destroy()`. أبطّل كذلك refresh tokens والجلسات المسجلة في قاعدة البيانات إن وجدت.

2. التحقق من المدخلات والفلتر

الفرق بين Validation و Sanitization واستخدام filter_var و filter_input وقواعد النطاق.

Validation ليست Sanitization

- **Validation**: هل القيمة تطابق شروطنا؟ لا يفترض أن يغيّرها.
- **Normalization**: توحيد تمثيل مشروع، مثل trim أو تحويل case عند الحاجة.
- **Sanitization**: حذف/تغيير محارف لصنع قيمة أخرى؛ قد يخفي الخطأ ويفسد البيانات.
- **Output Encoding**: تحويل القيمة عند إخراجها لسياق HTML/URL/JS، وليس عند الإدخال.

المسار المقترح:

```
Parse → Validate shape/type/range → Authorize action → Use safely
      ↓
      Encode at output sink
```

لا تحاول «تنظيف كل المدخلات» مرة واحدة ثم تعتبرها آمنة لكل مكان.

filter_var

```
$email = filter_var($rawEmail, FILTER_VALIDATE_EMAIL);
if ($email === false) {
    throw new InvalidArgumentException('Invalid email');
}

$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 18, 'max_range' => 120],
]);
if ($age === false) {
    throw new InvalidArgumentException('Age must be 18-120');
}

$ip = filter_var($rawIp, FILTER_VALIDATE_IP);
$url = filter_var($rawUrl, FILTER_VALIDATE_URL);
```

تحقق باستخدام `=== false`؛ القيمة الصحيحة قد تكون `0` و `FILTER_DEFAULT` يعني `FILTER_UNSAFE_RAW`، فلا يفعل تنظيفًا سحريًا.

filter_input و filter_input_array

```
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT, [
    'options' => ['default' => 1, 'min_range' => 1],
]);

$input = filter_input_array(INPUT_POST, [
    'email' => FILTER_VALIDATE_EMAIL,
    'age' => [
        'filter' => FILTER_VALIDATE_INT,
        'options' => ['min_range' => 18, 'max_range' => 120],
    ],
]);
```

`filter_input()` يقرأ المصدر الخارجي الأصلي حسب SAPI، وقد يختلف عن قيمة عدلتها داخل `$_GET` أو `$_POST`. اختبر بيئة التشغيل.

قواعد المجال أهم من شكل النوع

بريد صالح نحوياً لا يعني أنه يملك الحساب. و Order ID صحيح كعدد لا يعني أن المستخدم يملك الطلب.

```
$orderId = filter_input(INPUT_POST, 'order_id', FILTER_VALIDATE_INT);
if ($orderId === false || $orderId < 1) {
    throw new InvalidArgumentException('Invalid order');
}

$order = $orders->findForUser($orderId, $currentUserId);
if ($order === null) {
    throw new RuntimeException('Not found or forbidden');
}
```

طبّق allow-list للenums والفرز وأسماء الحقول. لا يمكن حماية SQL أو HTML بمجرد Sanitization عام؛ استخدم Prepared Statements و Output Encoding.

3. الحماية من CSRF

كيف يستغل CSRF جلسة المستخدم وكيف نمنعه بالـ SameSite tokens والتحقق من المصدر.

ما الهجوم؟

في CSRF يخدع موقع خبيث متصفح مستخدم مسجل الدخول ليرسل request غير مرغوب إلى موقع موثوق. المتصفح قد يرفق Cookies تلقائيًا، فيرى الخادم جلسة صحيحة لكنه لا يعرف أن المستخدم لم يقصد الفعل.

مثال خطر: Endpoint يغيّر البريد عبر GET أو يقبل POST بلا إثبات لنية المستخدم.

Synchronizer Token Pattern

أنشئ token عشوائيًا server-side، خزنه في Session، وضعه داخل النموذج:

```
session_start();

if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

```
<form method="post" action="/account/email">
  <input type="hidden" name="csrf_token"
    value="<?= htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8') ?>">
  <input type="email" name="email" required>
  <button>تحديث</button>
</form>
```

تحقق قبل تنفيذ أي تغيير:

```
$sent = $_POST['csrf_token'] ?? '';

if (
    !is_string($sent)
    || !isset($_SESSION['csrf_token'])
    || !hash_equals($_SESSION['csrf_token'], $sent)
) {
    http_response_code(403);
    exit('Invalid CSRF token');
}
```

مقارنة ثابتة الزمن نسبيًا. لا تضع token في URL أو Logs، واجعله مرتبطًا بالجلسة أو الفعل وفق نموذجك. `hash_equals()`

دفاع متعدد الطبقات

- لا تستخدم GET لتغيير الحالة.
- استخدم `Strict` أو `SameSite=Lax` حيث يناسب.
- تحقق من `Origin` للطلبات الحساسة، واستعمل `Referer` كخيار ثانٍ مدروس.
- في APIs التي لا تعتمد على Header، Cookies مخصص مع Token قد يمنع simple cross-site forms.
- أعد طلب كلمة المرور أو MFA للفعل شديد الحساسية.
- حدّد Content-Type المتوقع ولا تقبل أشكالاً أكثر من الحاجة.



تنبيه

CORS ليست بديلاً عن CSRF protection؛ وبعض الطلبات «البسيطة» يمكن إرسالها دون preflight. كذلك XSS في موقعك يستطيع غالباً قراءة token أو تنفيذ الطلب نفسه، لذلك إصلاح XSS أساسي.

Double-submit Cookie

يمكن إرسال قيمة في Cookie وقيمة في header/body ثم مقارنتهما، لكن الأفضل أن تكون القيمة موقعة ومرتبطة بالجلسة لمنع cookie injection. لا تخترع البروتوكول إن كان framework يوفر حماية مدققة.

Token lifecycle

لا يلزم دائماً تدوير token بعد كل request؛ ذلك قد يكسر tabs والطلبات المتزامنة. Token لكل جلسة أو لكل نموذج كلاهما صحيح حسب الخطر وتجربة الاستخدام. دوره عند تغيير الجلسة أو تسجيل الدخول، وحدد فشلاً واضحاً دون تنفيذ جزئي.

4. الحماية من XSS

Stored و Reflected و DOM XSS والترميز حسب سياق HTML و Attribute و URL و JavaScript.

ما XSS؟

تحدث Cross-Site Scripting عندما تُفسر بيانات غير موثوقة ككود داخل المتصفح.

- **Reflected**: ترجع payload مباشرة في نفس الاستجابة، مثل قيمة بحث.
- **Stored**: تُحفظ في قاعدة البيانات ثم تُعرض لضحايا آخرين.
- **DOM-based: JavaScript** في الصفحة ينقل بيانات إلى sink خطير مثل `innerHTML`.

النتائج قد تشمل سرقة بيانات متاحة للصفحة، تنفيذ أفعال بحساب المستخدم، تعديل الواجهة أو التصيد. `HttpOnly` يحمي قراءة Cookie فقط، ولا يمنع تنفيذ request من الصفحة المصابة.

HTML text و attributes

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

```
<h1><?= e($title) ?></h1>
<input value="<?= e($displayName) ?>">
```

ضع attribute بين quotes. لا تسمح للمستخدم بتحديد اسم attribute أو event handler.

URL context

رمز قيمة المعامل بـ `rawurlencode` ثم رمز الرابط كـ HTML عند إدخاله في attribute:

```
$url = '/search?q=' . rawurlencode($query);
?>
<a href="<?= e($url) ?>">بحث</a>
```

لا يكفي encoding لمنع scheme: `javascript`. حلّ URL وطبّق allow-list للبروتوكول والمضيف عند الروابط الخارجية.

الأفضل عدم إدخال نصوص مباشرة في inline JavaScript. إذا لزم نقل بيانات، استخدم JSON بخصائص hex:

```
<script>
const profile = <?= json_encode(
    $profile,
    JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT
) ?>;
</script>
```

في DOM استخدم `textContent` بدل `innerHTML` للنص:

```
message.textContent = untrustedValue;
```

عندما تريد السماح بـ HTML

`htmlspecialchars` سيعرض HTML كنص. إذا كان المنتج يسمح بمحتوى غني، استخدم HTML sanitizer موثوقًا بسياسة `allow-list`، وحذّته باستمرار. Regex ليست HTML parser.



أماكن خطيرة

تجنب وضع غير الموثوق داخل `<script>` أو `<style>` أو HTML comments أو أسماء tags/attributes أو event handlers. وتجنب `eval` و `document.write`.

طبقات إضافية

- Auto-escaping في template engine، مع مراجعة raw/unsafe escape hatches.
- Content Security Policy قوية، ويفضل nonce/hash بدل `unsafe-inline`.
- Trusted Types لتطبيقات DOM الكبيرة حيث يناسب.
- Cookies بـ HttpOnly/Secure/SameSite.
- MIME صحيح؛ JSON يجب أن يخرج `application/json`.
- CSP طبقة تقليل ضرر وليست بديلًا عن encoding و sanitization.

5. كلمات المرور و Hashing

تخزين كلمات المرور باستخدام password_hash و password_verify و bcrypt وإعادة hash.

Encryption ليس Hashing

كلمة المرور لا نحتاج استعادتها؛ نحتاج التحقق منها. لذلك نخزن password hash بطيئًا ومملحًا، لا تشفيرًا قابلاً للفك ولا SHA-256 سريعًا وحده.

دوال PHP تدير salt والصيغة والمعاملات داخل hash:

```
$hash = password_hash($password, PASSWORD_DEFAULT);
if ($hash === false) {
    throw new RuntimeException('Password hashing failed');
}
```

اجعل عمود قاعدة البيانات VARCHAR(255) لأن PASSWORD_DEFAULT قد تتغير خوارزميته وطول ناتجه.

إذا كان Argon2id متاحًا في البناء:

```
$hash = password_hash($password, PASSWORD_ARGON2ID, [
    'memory_cost' => 64 * 1024,
    'time_cost' => 3,
    'threads' => 2,
]);
```

القيم مثال وليست وصفة عالمية؛ benchmark على خادم الإنتاج واختبر تكلفة تؤخر المهاجم دون تعطيل المستخدمين.

التحقق وإعادة hash

```
if (!password_verify($password, $user['password_hash'])) {
    // رسالة عامة لا تكشف هل البريد موجود
    throw new AuthenticationException('Invalid credentials');
}

if (password_needs_rehash($user['password_hash'], PASSWORD_DEFAULT)) {
    $newHash = password_hash($password, PASSWORD_DEFAULT);
    $users->updatePasswordHash($user['id'], $newHash);
}
```

password_verify() يستخرج salt/options من hash. لا تقارن hashes يدويًا، ولا تنشئ salt بنفسك.

- اسمح بعبارات مرور طويلة، ولا تفرض قواعد تركيب مزعجة بلا سبب.
- لا تقص كلمة المرور بصمت. راع حد الخوارزمية؛ bcrypt يتعامل تاريخيًا مع أول 72 bytes.
- لا تغيّر case أو trim لكلمة مرور المستخدم دون سياسة معلنة.
- افحص كلمات المرور المسربة إن كانت لديك خدمة مناسبة تحفظ الخصوصية.
- أضف Rate Limiting وتأخيرًا تدريجيًا ومراقبة، وMFA للحسابات الحساسة.
- لا تسجل كلمة المرور أو تضعها في URL.

Pepper اختياري

يمكن إضافة secret server-side منفصل عن قاعدة البيانات باستخدام HMAC قبل `password_hash`، لكنه يزيد تعقيد التدوير والاسترجاع. احفظه في secret manager لا في Git، وخطط لتغيير المفاتيح. Salt ليس سرًا وموجود داخل Pepper:hash سر مختلف.

تغيير واسترجاع كلمة المرور

- اطلب إعادة المصادقة للتغيير الحساس.
- Reset token عشوائي، قصير العمر، single-use، وخزن hash له لا قيمته الخام.
- بعد التغيير ألغ الجلسات/refresh tokens الأخرى حسب سياسة المنتج.
- لا ترسل كلمة المرور القديمة أو الجديدة عبر البريد.

6. HTTP Authentication و Encryption

التشفير الآمن وإدارة المفاتيح و HTTP Basic و Bearer Tokens والفرق عن Hashing.

متى نستخدم Encryption؟

نستخدمه عندما يجب استعادة النص الأصلي، مثل سر تكامل خارجي مخزن. كلمة المرور لا تحتاج فگا، لذلك تستخدم Hashing.

- **Symmetric**: المفتاح نفسه للتشفير وفك التشفير؛ سريع للبيانات.
- **Asymmetric**: Public/Private Key: للتوقيع وتبادل الأسرار وحالات محددة.
- **Authenticated Encryption (AEAD)**: يحمي السرية ويكشف التعديل؛ فضله على «تشفير فقط».

مثال Sodium

```
function encrypt(string $plainText, string $key): string
{
    $nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
    $cipher = sodium_crypto_secretbox($plainText, $nonce, $key);
    return base64_encode($nonce . $cipher);
}

function decrypt(string $encoded, string $key): string
{
    $payload = base64_decode($encoded, true);
    if ($payload === false) {
        throw new RuntimeException('Invalid encoding');
    }

    $nonceLength = SODIUM_CRYPTO_SECRETBOX_NONCEBYTES;
    $nonce = substr($payload, 0, $nonceLength);
    $cipher = substr($payload, $nonceLength);
    $plain = sodium_crypto_secretbox_open($cipher, $nonce, $key);

    if ($plain === false) {
        throw new RuntimeException('Authentication failed');
    }

    return $plain;
}
```

المفتاح يجب أن يكون بطول `SODIUM_CRYPTO_SECRETBOX_KEYBYTES` ومن مصدر عشوائي آمن، ويُحفظ في secret manager. Nonce ليس سرًا لكنه يجب ألا يعاد استخدامه مع المفتاح وفق متطلبات الخوارزمية.



تنبيه

Base64 encoding وليس encryption. ولا تخرع خوارزمية أو تجمع AES/CBC و MAC يدويًا؛ استخدم API عالي المستوى يدعم authentication.

إدارة المفاتيح

- افصل المفتاح عن ciphertext وقاعدة البيانات قدر الإمكان.
- ضع Key ID/version مع البيانات لتسهيل rotation.
- حدّد من يستطيع decrypt وراقب العمليات.
- خطط للنسخ الاحتياطي والتدوير والإلغاء.
- لا تطبع المفتاح في logs أو exceptions.

HTTP Basic

يرسل العميل:

```
Authorization: Basic dXNlcjpwYXNz
```

القيمة Base64 ل `username:password` ويمكن فكها بسهولة؛ الحماية تأتي من HTTPS.

في PHP تحت بعض إعدادات الخادم:

```
$user = $_SERVER['PHP_AUTH_USER'] ?? null;
$pass = $_SERVER['PHP_AUTH_PW'] ?? null;

if ($user === null || !verifyCredentials($user, $pass ?? '')) {
    header('WWW-Authenticate: Basic realm="Admin"');
    http_response_code(401);
    exit('Authentication required');
}
```

Basic مناسب لأدوات داخلية بسيطة مع TLS و rate limiting، لكنه يرسل credential مع كل request ولا يملك logout/token lifecycle متقدمًا.

```
Authorization: Bearer opaque-random-token
```

من يحمل Bearer token يستطيع استخدامه؛ خزنه وانقله كسر. استخدم expiry و scope و rotation و revocation حيث يلزم. ليس كل JWT، token، JWT لا يعني أنه مشفر أو قابل للإلغاء تلقائيًا.

7. المصادقة والصلاحيات عمليًا

Login آمن و RBAC والسياسات وملكية الموارد ومنع IDOR وأقل قدر من الصلاحيات.

ثلاثة أسئلة مختلفة

1. **Authentication**: من المستخدم؟

2. **Authorization**: هل يُسمح له بهذا الفعل على هذا المورد؟

3. **Accounting/Auditing**: ماذا حدث، ومتى، ومن أي جلسة؟

نجاح تسجيل الدخول لا يعني السماح بكل شيء. وإخفاء الزر في الواجهة ليس Authorization.

Login flow

```
Validate input
→ Load account by normalized identifier
→ Verify password
→ Apply rate/risk controls
→ Regenerate session ID
→ Store minimal identity state
→ Redirect
```

أعد رسالة عامة مثل «بيانات الدخول غير صحيحة» لتقليل كشف وجود الحساب. لا تجعل كل الردود متطابقة زمنيًا بصورة مثالية على حساب DoS، لكن تجنب الفروق الفاضحة. سجّل الأحداث دون passwords أو tokens.

RBAC

```
enum Role: string
{
    case Admin = 'admin';
    case Editor = 'editor';
    case Viewer = 'viewer';
}

function canPublish(array $user): bool
{
    return in_array($user['role'], [
        Role::Admin->value,
        Role::Editor->value,
    ], true);
}
```

RBAC مناسب للصلاحيات العامة، لكنه لا يحل ملكية الموارد.

```
function canUpdatePost(array $user, array $post): bool
{
    return $user['role'] === 'admin'
        || (
            $user['role'] === 'editor'
            && $post['author_id'] === $user['id']
        );
}
```

الأفضل تحميل المورد مقيّدًا بالمستخدم عندما يناسب:

```
SELECT * FROM posts
WHERE id = :post_id AND author_id = :user_id
```

هذا يقلل IDOR/BOLA حيث يغيّر المستخدم ID في URL للوصول إلى مورد غيره.

Middleware لا يلغي الفحص داخل المجال

Middleware يتحقق من وجود مستخدم أو صلاحية عامة. أما قاعدة مثل «لا يمكن رد الطلب بعد الشحن» فتتنتمي لخدمة المجال/Policy ويجب تطبيقها مهما كان المدخل HTTP أو CLI أو Queue.

حالات HTTP

- **401 Unauthorized**: المصادقة مفقودة/غير صالحة، رغم أن الاسم التاريخي مبرك.
- **403 Forbidden**: الهوية معروفة لكن الفعل غير مسموح.
- **404 Not Found**: قد يُستخدم لإخفاء وجود مورد لا يحق للمستخدم معرفته.

Defense in depth

- Least privilege لحسابات DB والخدمات.
- Deny by default.
- تحقق server-side في كل request.
- MFA و step-up auth للعمليات الحساسة.
- Audit logs محمية من التعديل.
- إبطال الجلسات عند تغيير كلمة المرور أو الدور.
- اختبارات Authorization: مستخدم صحيح، دور خاطئ، مورد يملكه غيره، ID غير موجود.

8. JWT والتحقق الآمن

بنية JWT والتوقيع والـ Claims والتحقق من issuer و audience والوقت وإدارة المفاتيح.

JSON Web Token صيغة compact لنقل Claims بين أطراف. الشكل الشائع JWS مكوّن من ثلاثة أجزاء Base64url مفصولة بنقطة:

```
BASE64URL(header).BASE64URL(payload).BASE64URL(signature)
```

- Header: نوع التوكين والخوارزمية و Key ID اختياري.
- Payload: Claims مثل المستخدم والمصدر والجمهور ووقت الانتهاء.
- Signature: تكشف تعديل header أو payload إذا تم التحقق منها بالمفتاح الصحيح.



Base64url لا يخفي البيانات

يمكن لأي شخص يحمل JWT قراءة header و payload. التوقيع يوفر السلامة والأصالة، وليس السرية. لا تضع كلمة مرور أو Secret أو بيانات شخصية غير لازمة. التشفير يحتاج JWE أو تصميمًا آخر، وليس JWT موقعة فقط.

Claims قياسية مهمة

Claim	المعنى	التحقق
iss	الجهة المصدرة	تطابق issuer موثوقًا بالضبط
sub	موضوع/هوية التوكين	لا تفترض أنه email
aud	الجمهور المقصود	يجب أن تتضمن API الحالية
exp	انتهاء الصلاحية	ارفض المنتهي
nbf	غير صالح قبل	ارفض الاستخدام المبكر
iat	وقت الإصدار	افحص المعقولة حسب السياسة
jti	معرف فريد	يفيد في التتبع/منع إعادة الاستخدام

الـ Payload قد يحتوي أدوارًا أو scopes، لكن تغيّر صلاحية المستخدم بعد إصدار token لن يظهر حتى انتهاء التوكين أو فحص مصدر مركزي. لذلك اجعل access tokens قصيرة العمر ولا تعتبر JWT بديلًا دائمًا لسياسة Authorization.

التوقيع المتناظر وغير المتناظر

- HMAC: **HS256** بمفتاح سري مشترك؛ كل جهة تتحقق تستطيع أيضًا إصدار token.
- **RS256**/أمثاله: Private key للتوقيع و Public key للتحقق؛ أنسب عندما تتحقق خدمات كثيرة.
- **EdDSA** بخوارزمية مدعومة ومكتبة موثوقة خيار حديث في البيئات المتوافقة.

لا تختار الخوارزمية من قيمة **alg** داخل token وحدها. ثبتّ allowlist في verifier ولا تقبل **none**. لا تستخدم كلمة مرور بشرية كمفتاح HMAC.

مثال PHP بمكتبة

```
composer require firebase/php-jwt
```

```
<?php
declare(strict_types=1);

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$payload = [
    'iss' => 'https://id.example.com',
    'sub' => 'user_123',
    'aud' => 'https://api.example.com',
    'iat' => time(),
    'nbf' => time(),
    'exp' => time() + 300,
    'scope' => 'orders:read',
];

$token = JWT::encode($payload, $privateKey, 'RS256', 'key-2026-09');
$claims = (array) JWT::decode($token, new Key($publicKey, 'RS256'));

if (($claims['iss'] ?? null) !== 'https://id.example.com') {
    throw new RuntimeException('Invalid issuer');
}

$audiences = (array) ($claims['aud'] ?? []);
if (!in_array('https://api.example.com', $audiences, true)) {
    throw new RuntimeException('Invalid audience');
}
```

المكتبة تتعامل مع التوقيع وبعض claims الزمنية، لكن التطبيق ما زال مسؤولًا عن issuer و audience ونوع token والسياسة. استخدم مكتبة ناضجة بدل بناء Base64/signature يدويًا.

JWKS و Key ID

kid يسهّل rotation، لكنه مدخل غير موثوق:

1. اربط issuer بقائمة مفاتيح/JWKS URL موثوقة مسبقًا.

2. لا تستخدم `kid` كاسم ملف أو SQL بلا تحقق.
3. لا تتبع `jku` أو URL يرسله التوكين عشوائيًا؛ هذا قد يخلق SSRF أو مفتاح مهاجم.
4. خزّن JWKS مؤقتًا مع refresh محدود، واحتفظ بالمفتاح القديم حتى تنتهي التوكينات الموقعة به.

قائمة تحقق

- HTTPS دائمًا.
- حدد allowed algorithms و `typ` token.
- تحقق من signature و `iss` و `aud` و `exp` و `nbf`.
- امنح clock skew صغيرًا ومحددًا، لا ساعات.
- افصل مفاتيح وأنواع ID tokens عن access tokens لمنع substitution.
- لا تسجل التوكين كاملاً.
- خطط للإلغاء: مدة قصيرة، denylist لحالات محددة، أو opaque/reference token عندما تحتاج revocation فورًا.

9. Federated Identity و SSO و SAML و OIDC

الهوية الاتحادية وتسجيل الدخول الموحد وأدوار IdP و SP ومقارنة SAML مع OpenID Connect.

Federated Identity علاقة ثقة تسمح لتطبيق بالاعتماد على جهة خارجية لإثبات هوية المستخدم. **Single Sign-On (SSO)** تجربة يستطيع فيها المستخدم الوصول إلى عدة تطبيقات بعد عملية دخول مركزية، لكنه قد يُنفذ داخل مؤسسة أو عبر اتحاد هويات.

```
User -> Application / Service Provider
      -> Identity Provider
      <- signed identity result
      <- local session
```

التطبيق لا يشارك كلمة مرور المستخدم مع كل خدمة؛ يثق في نتيجة موقعة ومقيدة بجمهور ووقت محدد.

SAML و OIDC و OAuth

التقنية	الهدف الأساسي	النتيجة المعتادة
OAuth 2.0	تفويض وصول عميل إلى Resource Server	Access token
OpenID Connect	مصادقة مبنية فوق OAuth 2.0	ID token + معلومات هوية
SAML 2.0	تبادل Assertions للهوية/الصفات، شائع في Enterprise SSO	SAML Response/Assertion XML



التصحيح الأهم

OAuth وحده ليس بروتوكول Login. استخدام access token و endpoint عشوائي لاستنتاج هوية المستخدم يسمى أحياناً pseudo-authentication وقد يؤدي لخلط المستخدم أو الجمهور. استخدم OIDC للمصادقة.

SAML Web Browser SSO

الأطراف الرئيسية:

- **Identity Provider (IdP)**: يصادق المستخدم ويصدر assertion.
- **Service Provider (SP)**: التطبيق الذي يستهلك النتيجة وينشئ جلسة محلية.
- **Assertion: XML**: موقع يحوي subject و conditions و attributes.

تدفق SP-initiated مبسط:

```
Browser -> SP: protected page
SP -> Browser -> IdP: AuthnRequest
IdP authenticates user
IdP -> Browser -> SP ACS: SAMLResponse
SP validates response/assertion
SP creates local session
```

التحقق يجب أن يشمل XML signature من شهادة IdP الموثوقة، `Issuer`، `AudienceRestriction`، `Recipient/Destination`، `InResponseTo` عند استخدام request. امنع replay وسجل ID assertion. استخدم مكتبة SAML ناضجة وعطّل external entities؛ XML Signature معقدة ولا تُنفذ يدويًا.

OpenID Connect

OIDC يضيف طبقة هوية إلى OAuth. وجود `openid` داخل `scope` يحول الطلب إلى طلب OIDC:

```
scope=openid profile email
```

النواتج:

- **ID token**: موجه إلى Client ويخبره بنتيجة المصادقة.
 - **Access token**: موجه إلى API/Resource Server؛ لا تستخدم ID token بدلًا منه.
 - **Userinfo endpoint**: معلومات إضافية اختيارية باستخدام access token.
- Claims شائعة: `sub` و `iss` و `aud` و `exp` و `auth_time`، ومع scopes مناسبة قد تظهر `name` و `email`. الـ `scope` طلب لمجموعة claims/صلاحيات وليس ضمانيًا أن كل قيمة موجودة.

تحقق ID token

1. اكتشف metadata من issuer موثوق مسبقًا.
 2. تحقق من التوقيع عبر JWKS لهذا issuer.
 3. طابق `iss` حرفيًا و `aud` مع `client_id`.
 4. افحص `exp` و `iat` و `nbf` إن وُجد.
 5. طابق `nonce` مع المعاملة التي بدأت في المتصفح.
 6. طبق قواعد `azp` عندما توجد جماهير متعددة.
 7. إذا استخدمت Userinfo، يجب أن يطابق `sub` قيمة ID token.
- `email` قابل للتغيير وقد لا يكون verified؛ استخدم `iss + sub` كمفتاح هوية خارجي ثابت.

متى أختار ماذا؟

- تكامل حديث لتطبيقات Web/Mobile وOIDC APIs غالبًا.
- Enterprise قديم/مؤسسات تعتمد XML وSAML 2.0 metadata.

- منح تطبيق حق الوصول إلى API دون الحاجة لهوية مستخدم: OAuth 2.0.

ليس SAML "غير آمن لأنه قديم" ولا OIDC آمنًا تلقائيًا. الخطر في validation ناقص، trust غير مضبوط، redirect غير دقيق، أو مكتبة غير محدثة.

10. مفاهيم OAuth 2.0 والتوكينات

التفويض المفوض والأدوار والScopes والConsent وأنواع Clients وAccess Tokens وRefresh Tokens.

OAuth 2.0 إطار تفويض مفوض: يمنح المستخدم أو النظام Client وصولًا محدودًا إلى Resource Server دون تسليم كلمة مرور المستخدم للClient.

مثال: تطبيق صور يطلب قراءة ملفات محددة من خدمة تخزين. المستخدم يصادق نفسه لدى خدمة التخزين، يوافق على النطاق، ويحصل التطبيق على access token محدود بدل كلمة المرور.

الأدوار الأربعة

1. **Resource Owner**: الجهة القادرة على منح الوصول، وغالبًا المستخدم.
2. **Client**: التطبيق الذي يطلب الوصول؛ ليس "المستخدم".
3. **Authorization Server**: يصادق الأطراف حسب الحاجة ويصدر التوكينات بعد التفويض.
4. **Resource Server**: الAPI التي تستقبل access token وتحمي الموارد.

```
Resource Owner -> Authorization Server: approve
Client -> Authorization Server: grant/code
Authorization Server -> Client: access token
Client -> Resource Server: token + API request
```

قد تكون Authorization Server وResource Server جزءًا من منتج واحد أو خدمتين مختلفتين.

Client registration

يسجل العميل عادة:

- **client_id**: معرف عام، ليس سرًا.
- Redirect URIs دقيقة.
- نوع العميل وطرق authentication المسموحة.
- scopes/grants المسموحة وسياسات consent.

Confidential وPublic

- **Public client**: لا يستطيع حفظ secret، مثل native app أو JavaScript يعمل في المتصفح. تضمين **client_secret** في bundle لا يجعله سرًا.
- **Confidential client: backend**: قادر على حماية credentials ويصادق نفسه عند token endpoint.

اسم النوع يصف قدرة حفظ السري، وليس ثقة أخلاقية أو مستوى صلاحية تلقائيًا.

Consent و Scope

scope اسم صلاحية يطلبها Client:

```
orders:read orders:write profile
```

صمم scopes حول capabilities واضحة وبأقل صلاحية. لا تجعل scope `admin` عامًا يختصر عشرات العمليات الحساسة بلا سبب. Consent واجهة/قرار يمنح فيه Resource Owner وصولًا. لكنه ليس بديلًا عن سياسة Authorization Server؛ قد ترفض المؤسسة scope حتى لو وافق المستخدم، وقد يكون consent سابقًا أو إداريًا في machine-to-machine.

Access token

Credential يقدمها Client إلى Resource Server:

```
Authorization: Bearer eyJ...
```

- قد يكون opaque string أو JWT؛ OAuth لا يفرض صيغة واحدة.
- يجب تقييده بالمدة والجمهور والموارد والإجراءات.
- Bearer يعني أن من يحمله يمكنه استخدامه، لذلك خزنه وانقله كسر.
- API يجب أن تتحقق أنه موجه إليها وأن scope يسمح بالفعل، لا أن signature صحيحة فقط.

Refresh token

Credential طويل الحساسية يستخدم عند token endpoint للحصول على access token جديد. لا يُرسل إلى Resource Server.

```
Client --refresh token--> Authorization Server
Client <--new access token (+ rotated refresh token)--
```

public clients استخدم refresh-token rotation مع reuse detection أو sender-constraining حسب النظام. عند اكتشاف إعادة استخدام token قديم، ألغ family كاملة واطلب تسجيل دخول جديدًا.

API key ليست OAuth token

API key غالبًا تعرّف project/client وتستخدم للحصة أو خدمة بسيطة، ولا تمثل بالضرورة مستخدمًا أو consent أو scopes ديناميكية. تعامل معها كسر، لكن لا تسمها access token ولا تستخدمها لتسجيل دخول مستخدم.

Token endpoint response

```
{
  "access_token": "opaque-or-structured-value",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_token": "long-lived-secret",
  "scope": "orders:read"
}
```

الاستجابة حساسة: استخدم TLS، أرسل `Cache-Control: no-store`، ولا تسجل الجسم. `expires_in` مدة من لحظة الاستلام؛ لا تفترض أن JWT access token أو تحاول parse opaque token.

أخطاء مفاهيمية

- OAuth لا يثبت الهوية للتطبيق؛ OIDC يفعل.
- JWT صيغة token وليست بديلًا عن OAuth.
- Client ID ليس secret.
- Scope ليس Role داخليًا بالضرورة.
- Access token لا يذهب إلى صفحة login، و Refresh token لا يذهب إلى API.
- امتلاك token صالح لا يلغي فحص ملكية المورد وقواعد المجال.

11. OAuth Flows الحديثة

Authorization Code مع PKCE و Client Credentials و Device Authorization و Refresh Tokens والتدفقات القديمة.

Browser/Web/Mobile + مستخدم -> Authorization Code + PKCE
 خدمة تتحدث مع خدمة -> Client Credentials
 TV/CLI بلا متصفح مناسب -> Device Authorization
 تجديد وصول سابق -> Refresh Token

Authorization Code + PKCE

هذا هو الاختيار الأساسي لتطبيقات الويب و SPA و Native Apps. الكود يمر عبر الـ Front Channel لكنه قصير العمر وأحادي الاستخدام، أما التوكينات فتعود من Token Endpoint عبر اتصال مباشر.

Browser	Client	Authorization Server
--- start --->		
	-- authorize + challenge	
<----- redirect to login ----->		
<----- login/consent ----->		
<---- redirect: code + state ----->		
-- callback -->		
	-- code + verifier ----->	
	<-- access/refresh token-->	

إنشاء PKCE

```

<?php
declare(strict_types=1);

function base64Url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

$codeVerifier = base64Url(random_bytes(32)); // 43 chars
$codeChallenge = base64Url(hash('sha256', $codeVerifier, true));
$state = base64Url(random_bytes(32));
$nonce = base64Url(random_bytes(32)); // عند استخدام OIDC

$_SESSION['oauth'] = [
    'state' => hash('sha256', $state),
    'verifier' => $codeVerifier,
    'nonce' => $nonce,
    'created_at' => time(),
];
  
```

طلب التفويض:

```
response_type=code
client_id=...
redirect_uri=https://client.example.com/callback
scope=openid profile orders:read
state=...
nonce=...
code_challenge=...
code_challenge_method=S256
```

عند callback مطابق state بقيمة session مقارنة ثابتة الزمن واحذف المعاملة بعد الاستخدام. ثم أرسل code و `code_verifier` إلى `token endpoint` الـ Authorization Server بحسب: `redirect URI` نفسها

```
BASE64URL(SHA256(code_verifier)) == code_challenge
```

PKCE ليس تشفيرًا للكود؛ يربط الكود بعمل بدأ المعاملة. استخدم قيمة جديدة عالية entropy لكل محاولة و `S256` فقط.

Client Credentials

machine-to-machine عندما يعمل Client بصفته، لا بصفة مستخدم:

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Authorization: Basic BASE64(client_id:client_secret)

grant_type=client_credentials&scope=reports:write
```

- للـ Confidential clients فقط.
- لا يوجد user أو consent تفاعلي؛ لا تختبر `user_id` لهذا token.
- فضل مصادقة أقوى مثل mTLS أو `private_key_jwt` في الأنظمة الحساسة.
- اجعل كل workload بهوية وصلاحيات مستقلة بدل secret مشترك بين الخدمات كلها.

Device Authorization

مناسب لـ Smart TV أو CLI أو جهاز إدخاله محدود:

1. الجهاز يطلب `device_code` و `user_code` و `verification_uri`.
2. يعرض للمستخدم الرابط والكود، ويفضل QR لا يحتوي أسرارًا غير مطلوبة.
3. المستخدم يفتح الرابط على جهاز آخر ويسجل الدخول ويوافق.
4. الجهاز يعمل polling للـ `token endpoint`.

```
{
  "device_code": "secret-device-value",
  "user_code": "WDJB-MJHT",
  "verification_uri": "https://id.example.com/device",
  "expires_in": 600,
  "interval": 5
}
```

احترم `interval` ، ومع `slow_down` زد فترة polling خمس ثواني، وتوقف عند `access_denied` أو `expired_token` . اعرض اسم الجهاز والعميل للمستخدم لتقليل هجمات خداع الأكواد.

Refresh Token Grant

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=refresh_token&
refresh_token=...&
client_id=...
```

لا تطلب scope أوسع من الأصلي. استخدم rotation: كل استبدال يصدر refresh token جديدًا ويلغي السابق، مع كشف reuse.

Assertion Grants

يمكن استخدام SAML assertion كـ Authorization Grant أو لمصادقة Client وفق RFC 7522. هذا امتداد تكاملي بين اتحاد SAML و OAuth وليس التدفق الافتراضي لكل SAML SSO. تحقق من issuer و audience والوقت والتوقيع ومنع replay.

تدفقات لا تبدأ بها مشروعًا جديدًا



تحذير

- **Implicit Grant**: يعيد access token عبر front channel حيث يزداد التسريب وإعادة الاستخدام. استخدم Authorization Code + PKCE.
- **Resource Owner Password Credentials**: يعلّم المستخدم إدخال كلمة مروره في Client ويصعب MFA و WebAuthn؛ RFC 9700 يقول إنه يجب عدم استخدامه.

12. أمان OAuth وOIDC

State و Nonce و PKCE و Redirect URI وتخزين التوكينات و BFF والإبطال ومراقبة التكامل.

لا تستبدل `state` و `nonce` و PKCE ببعضها عشوائيًا:

الآلية	تحمي أساسًا من
<code>state</code> مرتبط بالجلسة	CSRF وتبديل معاملات callback
OIDC <code>nonce</code>	replay/substitution لنتيجة المصادقة
PKCE	سرقة أو حقن Authorization Code
Redirect URI exact match	إرسال النتيجة إلى وجهة مهاجم

وفق OAuth Security BCP، PKCE مطلوب للـ public clients وموصى به أيضًا للـ confidential clients. استخدم قيمًا فريدة لكل معاملة واحذفها عند النجاح أو انتهاء مهلة قصيرة.

```
<?php
declare(strict_types=1);

session_start();

$tx = $_SESSION['oauth'] ?? null;
unset($_SESSION['oauth']); // one-time transaction

if (!is_array($tx) || time() - $tx['created_at'] > 600) {
    throw new RuntimeException('OAuth transaction expired');
}

$returnedState = $_GET['state'] ?? '';
if (
    !is_string($returnedState)
    || !hash_equals($tx['state'], hash('sha256', $returnedState))
) {
    throw new RuntimeException('Invalid state');
}

if (isset($_GET['error'])) {
    // فقط دون التوكينات code اعرض رسالة آمنة، وسجل.
    throw new RuntimeException('Authorization was not completed');
}

$code = $_GET['code'] ?? null;
if (!is_string($code) || $code === '') {
    throw new RuntimeException('Missing authorization code');
}

// Exchange code once using tx['verifier'] and the exact redirect URI.
```

لا تسجل query كاملة لأن callback قد يحتوي code. ضع **Referrer-Policy: no-referrer** وتجنب third-party scripts في صفحة callback.

Redirect URIs

- سجل HTTPS URI كاملة وطابقها حرفيًا؛ لا تستخدم wildcard للإنتاج.
- اسمح URI loopback/custom وفق قواعد apps native الرسمية فقط.
- لا تجعل callback open redirect يأخذ **next=https://evil.example**.
- بعد callback استخدم destination مخزنة server-side أو allowlist لمسارات داخلية.
- اربط المعاملة بالissuer لتجنب mix-up عندما يدعم Client أكثر من Authorization Server.

مكان التوكينات في تطبيق Browser

Backend for Frontend

```
Browser --Secure HttpOnly session cookie--> BFF
BFF --access token--> APIs
```

هذا الاختيار يقلل وصول JavaScript إلى التوكينات. طَبَّق CSRF protection و SameSite مناسبًا و session rotation.

Browser-only client

إذا كان التوكين داخل المتصفح، فضّل memory على storage دائم طويل العمر. `localStorage` متاح لأي JavaScript داخل `origin`؛ XSS يستطيع سرقة. CSP و Trusted Types وتقليل third-party scripts دفاعات مهمة لكنها لا تجعل التخزين آمنًا مطلقًا. لا تضع token في URL أو history أو logs أو analytics. ولا تنقل client secret إلى SPA.

Bearer مقابل Sender-constrained

Bearer token قابل لإعادة الاستخدام من أي حامل. للأنظمة الأعلى حساسية يمكن ربط التوكين بعميل عبر mTLS أو DPoP، لكن ذلك يزيد التعقيد ولا يلغي التحقق من audience/scope أو حماية الجهاز.

Refresh lifecycle و Access

- Access token قصيرة العمر ومقيّدة بالـ `resource/audience`.
- Refresh token أطول عمرًا وتخزن بأقوى حماية متاحة.
- Rotate refresh tokens واكشف `reuse`.
- ألغ العائلة عند logout أو سرقة أو تغيير أمني.
- وفر revocation endpoint أو introspection للـ `opaque tokens` حيث يلزم.
- لا تفترض أن "JWT لا يمكن إلغاؤها" حقيقة مطلقة؛ يمكن `denylist` أو `version/session checks`، لكن لها تكلفة `state`.

OIDC login checklist

1. Discovery من issuer موثوق، لا من قيمة يرسلها المستخدم.
2. Authorization Code + PKCE S256.
3. `state` و `nonce` مرتبطان بالجلسة والمعاملة.
4. Redirect URI مسجلة بدقة.
5. تحقق signature و `iss` و `aud` و `exp` و `nonce`.
6. استخدم `iss + sub` لربط الحساب.
7. لا تمنح صلاحيات محلية من email/domain claim دون سياسة واضحة والتحقق المطلوب.

8. أنشئ جلسة تطبيق جديدة ثم غيّر session ID.

المراقبة دون تسريب

سجّل: flow، client ID، issuer، نتيجة عامة، correlation ID، error code، ووقت الاستجابة. لا تسجل token أو code أو verifier أو secret أو assertion. راقب:

- ارتفاع `invalid_grant`.
- reuse للRefresh Token.
- redirect/issuer mismatch.
- فشل signature أو audience.
- طلب scopes غير معتادة.
- معدل Device Flow polling وانتهاء الأكواد.



الخلاصة

استخدم مكتبة OIDC/OAuth مجزّبة وموفر هوية واضح. البروتوكول يبدو و redirects و HTTP parameters، لكن صحة الربط بين browser session و issuer و client و redirect والتوكين هي الجزء الأمني الصعب.

13. دورة حياة الحساب

التسجيل وتأكد البريد واسترجاع كلمة المرور وتغيير الهوية وإعادة المصادقة وإلغاء الجلسات.

التسجيل

عامل التسجيل كعملية هوية لا مجرد **INSERT** :

1. طبع البريد وفق سياسة موثقة دون تغيير عشوائي.
 2. تحقق من الشكل والنطاق وقواعد المنتج.
 3. طبق rate limit ومراقبة الإساءة.
 4. خزّن password بـ **password_hash()**.
 5. لا تمنح صلاحيات حساسة قبل إثبات البريد/الهاتف المطلوب.
- لتقليل account enumeration استخدم رسالة متقاربة مثل "إن كان العنوان صالحًا فستصل رسالة"، مع logging داخلي.

Token التأكيد

```
$token = bin2hex(random_bytes(32));
$tokenHash = hash('sha256', $token);
// خزّن hash + user_id + expires_at + used_at
// مرة واحدة HTTPS الخام في رابط token أرسل
```

عند الاستخدام: hash القيمة، افحص expiry وused_at، ثم علّمها مستخدمة داخل transaction. لا تخزن token الخام.

Password reset

- رسالة عامة سواء الحساب موجودًا أم لا.
- token عشوائية قصيرة العمر وأحادية الاستخدام.
- لا تغير password قبل إثبات token.
- بعد النجاح دور session ID وألغ جلسات/refresh tokens حسب السياسة.
- أرسل إشعارًا آمنًا.

لا تستخدم أسئلة أمان قابلة للتخمين.

العمليات الحساسة

اطلب re-authentication أو MFA قبل تغيير البريد أو password أو وسائل MFA أو تنفيذ تحويل مالي. لا تعتبر session قديمة دليلًا كافيًا؛ سجل `auth_time` وحدد max age.

تغيير البريد والحذف

تحقق من البريد الجديد، وأبلغ القديم، وتعامل مع uniqueness race داخل database. عوّف الفرق بين suspension و deactivation و deletion، وألغ credentials والجلسات وسجل actor/reason/time.

حالات السباق

استخدم transaction و unique constraints، وامتنع استخدام token نفسها مرتين. اختبر click مزدوج وطلب reset متزامن وتغيير البريد أثناء وجود جلسات أخرى.

14. MFA وTOTP وPasskeys

عوامل المصادقة وTOTP وRecovery Codes وWebAuthn/Passkeys وStep-up والاسترجاع الآمن.

العامل ليس مجرد خطوة ثانية

- شيء تعرفه: password/PIN.
- شيء تملكه: authenticator أو security key.
- شيء تكونه: biometric محلي يفتح credential.

عاملان من الفئة نفسها ليسا MFA قوية. SMS أفضل أحيانًا من password وحدها لكنه معرض لـ SIM swap و phishing.

TOTP

عند التسجيل أنشئ secret عشوائية، اعرض QR عبر قناة authenticated، واطلب code لإثبات الإعداد قبل التفعيل. خزّن السر مشفرًا بمفتاح مُدار، لا hashed فقط لأن الخادم يحتاجه للتحقق.

- نافذة زمنية صغيرة مع clock متزامن.
- امنع replay لنفس time step إن أمكن.
- rate limit للتحقق.
- لا تسجل secret أو code.

Recovery Codes

أنشئ codes عشوائية أحادية الاستخدام، اعرضها مرة، وخزّن hashes قوية لها. عند استخدام code علّمها مستخدمة وأخبر المستخدم، واسمح بتجديد المجموعة بعد re-authentication.

Passkeys وWebAuthn

```
server challenge -> browser/authenticator -> public-key credential
server stores credential ID + public key + metadata
```

في الدخول يتحقق الخادم من signature و challenge و origing و RP ID و user verification وسياسة counter. الخادم لا يخزن private key. Passkeys مقاومة للتصيد لأنها مرتبطة بالـ RP/origin.

اطلب assurance أعلى عند تغيير وسائل الاسترجاع، دفع كبير، عرض سر، أو سلوك عالي المخاطر. اربط النتيجة بالعملية ومدة قصيرة.

Recovery هي أضعف حلقة

- عدة قنوات/إثباتات حسب المخاطر.
- تأخير وإشعارات للتغيير الحساس.
- مراجعة دعم بشري بإجراءات وتدقيق.
- إلغاء credentials المفقودة وتسجيل الحدث.

مرجع

- [OWASP Authentication Cheat Sheet](#)

15. API Keys وهوية الأنظمة

إصدار مفاتيح API وتخزينها وتدويرها و Scopes و Service Accounts و mTLS والفرق عن جلسة المستخدم.

API key ليست هوية مستخدم

المفتاح يعرّف application أو integration غالبًا، ولا يثبت وحده المستخدم النهائي أو التفويض المفوض مثل OAuth. اربطه principal واضح مع owner و purpose و بيئة وصلاحيات.

شكل وتخزين

استخدم prefix يوضح النوع ومعرّفًا عامًا وسرًا عشوائيًا:

```
live_ak_7F2K.<random-secret>
```

ابحث بالمعرف، وخرّن hash للسر:

```
$secret = bin2hex(random_bytes(32));
$hash = hash('sha256', $secret);
// فقط hash اعرض السر مرة واحدة ثم خزن
```

الـ prefix يساعد secret scanners والدعم، لكنه ليس سرًا.

صلاحيات ودورة حياة

- scopes صغيرة و deny-by-default.
- expiry عند الإمكان.
- last-used metadata بلا body حساس.
- rotation بفترة overlap قصيرة.
- revoke فوري.
- مفاتيح منفصلة لكل نظام وبيئة.

أرسل key في header **Authorization** أو header مخصص عبر TLS، لا query string.

Service Accounts و Rate limits

طبّق quota لكل IP allow-list. key/tenant طبقة إضافية وليست هوية وحدها. امنح الحساب الآلي أقل صلاحية واربط actions به في audit trail. فضل credentials قصيرة العمر بدل ملفات keys ثابتة عند توفر workload identity.

Mutual TLS يسمح للطرفين بتقديم شهادات. يحتاج إصدار وتجديد وإلغاء وربط subject بصلاحيات؛ التشفير المتبادل لا يلغي authorization على العملية.

16. ABAC و ReBAC وعزل tenants

نماذج صلاحيات تتجاوز RBAC مع السياسات والعلاقات والعزل متعدد المستأجرين والاختبار.

RBAC ليست كل شيء

- **RBAC**: القرار مبني على roles.
- **ABAC**: attributes للمستخدم والمورد والسياق.
- **ReBAC**: علاقات مثل owner/member/manager.

```
allow if
  subject.tenant_id == resource.tenant_id
  AND subject.department == resource.department
  AND action == "read"
```

ابدأ بسياسة بسيطة، واستخدم نموذجًا أعقد فقط عندما يعبر عن قواعد حقيقية.

Policy decision

```
final class InvoicePolicy
{
  public function view(User $user, Invoice $invoice): bool
  {
    return $user->tenantId() === $invoice->tenantId()
      && ($user->id() === $invoice->ownerId() || $user->hasRole('auditor'));
  }
}
```

الـ API قد تخفي الزر، لكن الخادم يعيد القرار لكل request و queue/CLI path.

Multi-tenancy

مرر tenant من الهوية الموثقة لا من body فقط. طبق العزل في queries و cache keys و object storage و queue messages و search indexes و exports.

اختر database مشتركة أو schema منفصلة أو database لكل tenant بناءً على العزل والتكلفة والتشغيل، ولا تفترض أن نمطًا واحدًا آمن تلقائيًا.

Deny by default والاختبار

أي action غير معروفة تُرفض. راجع inheritance والتعارض بين allow/deny، وسجل نسخة policy والسبب العام داخليًا.

اختبر matrix تشمل owner في tenant نفسه، مستخدمًا بلا ID، role مطابقًا في tenant مختلف، auditor، و resource غير موجود.
اختبر IDOR بتبديل IDs وكل HTTP/CLI/queue path.

مرجع

• [OWASP Authorization Cheat Sheet](#)

17. Rate Limiting ومقاومة الإساءة

Fixed و Sliding Window و Token Bucket وحدود تسجيل الدخول و Credential Stuffing و CAPTCHA وسياسة الفشل.

ما الذي نحده؟

لا تعتمد على IP فقط؛ قد يشترك مستخدمون في NAT وقد يوزع المهاجم الطلبات. كَوْن مفاتيح حسب العملية:

- login: account + IP/network + device signals
- password reset: account/contact + IP
- API: key + tenant + endpoint
- expensive search: user + query cost

ضع حدودًا أقصى للفشل والعمليات المكلفة، مع حد عالمي يحمي السعة.

الخوارزميات

الأسلوب	الفكرة
Fixed window	عداد داخل فترة؛ بسيط وله burst عند الحدود
Sliding log/window	أدق وأعلى تكلفة
Token bucket	tokens تتجدد وتسمح burst مضبوط
Leaky bucket	يصقل معدل الخروج

يجب أن تكون العملية atomic في التخزين المشترك عند تعدد الخوادم.

استجابة HTTP

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
```

لا تكشف هل username موجود. أضف delay/backoff بحذر دون حجز workers طويلًا.

Credential Stuffing و Login

- لا تستخدم lockout دائمًا يسمح للمهاجم بقفل حساب الضحية.

- استخدم progressive delay وحدودًا متعددة.
- راقب passwords مسربة وفق سياسة الخصوصية.
- MFA/Passkeys تقللان أثر password المسروقة.
- أخطر المستخدم عند نشاط غير معتاد.

Risk و CAPTCHA

CAPTCHA friction وليست proof of humanity كاملة، ويمكن تجاوزها. استخدمها بعد إشارة خطر لا لكل المستخدمين، وراع accessibility والخصوصية.

الفشل والتشغيل

حدد fail-open أو fail-closed لكل عملية إذا تعطل مخزن login. limits. إداري حساس يختلف عن endpoint عامة. راقب allow/deny latency و top keys دون تخزين credentials.

Audit Trail و Security Logging .18

أحداث الأمان وبنية السجل والتنقية وaudit غير القابل للعبث والتنبيهات والتحقيق.

؟Audit أم Operational log

- operational log للتشخيص والأداء وقد يتغير retention.
 - security event للكشف والتنبيه.
 - audit trail يسجل من فعل ماذا ومتى وعلى أي مورد، مع متطلبات سلامة واحتفاظ أعلى.
- لا تستخدم نصوصًا غير منظمة لسجل تدقيق قانوني.

Schema مقترحة

```
{
  "event": "authorization.denied",
  "time": "2026-09-21T10:15:00Z",
  "request_id": "req_01J...",
  "actor_id": "usr_42",
  "tenant_id": "tn_7",
  "action": "invoice.delete",
  "resource_id": "inv_99",
  "result": "deny",
  "reason_code": "not_owner"
}
```

استخدم IDs لا أسماء/بيانات شخصية إن لم تلزم. وُحد UTC ونسخة schema ومصدر الخدمة.

أحداث مهمة

- نجاح/فشل login وMFA وrecovery.
 - تغيير password/email/MFA/roles.
 - إنشاء أو تدوير أو إلغاء API key.
 - authorization deny والعمليات الإدارية.
 - export/delete للبيانات.
 - secret access وpolicy changes.
- لا تسجل password أو token أو session ID أو key خام أو recovery code.

سلامة ووصول

أرسل logs إلى مخزن مركزي محدود الوصول، افصل صلاحية الكتابة عن الحذف، واستخدم retention و backup وتزامن وقت. راقب توقف ingestion ومحاولات العبث.

التنبيه والتحقيق

التنبيه على نمط: failures موزعة، impossible travel، رفع صلاحيات، key جديدة ثم export ضخ. ضع runbook و owner و severity. واحفظ IDs request/trace لربط الأدلة.

اختبر أن logging لا تفشل العملية الأساسية بلا سبب ولا تسرب أسرارًا عند exception.

مرجع

• [OWASP Logging Cheat Sheet](#)

19. إدارة الأسرار ومفاتيح التشفير

Secret managers و envelope encryption و rotation و revocation و access policy و break-glass والاستجابة للتسريب.

ما السر؟

كلمات مرور قواعد البيانات و API keys و private keys و encryption keys و signing secrets و أسرار. اسم البيئة و URL عامة ليست أسرارًا. التصنيف يحدد التخزين والوصول والدوران.

ملف `.env` وسيلة تحميل في التطوير وليس secret manager، و Base64 ليست تشفيرًا.

دورة الحياة

generate -> store -> distribute -> use -> rotate -> revoke -> destroy

لكل سر owner و purpose و consumers و created/expiry و rotation policy. لا تشارك سرًا واحدًا بين خدمات كثيرة؛ يصعب معرفة المتسبب وإلغاؤه.

Secret Manager

التطبيق يحصل على السر بهوية workload وبأقل صلاحية. فضل secrets قصيرة العمر أو Cache و dynamic credentials. في الذاكرة مدة محدودة وتوقع renewal/failure دون كتابتها إلى disk أو logs.

Envelope Encryption

KMS/HSM master key encrypts a data-encryption key (DEK)
DEK encrypts application data
store ciphertext + encrypted DEK + algorithm/version metadata

لا تستخدم master key مباشرة لكل record. استخدم authenticated encryption مثل Sodium AEAD، مع nonce صحيحة و associated data تربط ciphertext بالسياق.

Rotation

احفظ key ID/version مع البيانات أو token. أثناء الدوران:

1. أنشئ key جديدة.

2. ابدأ الكتابة بها.

3. استمر في قراءة النسخ القديمة.

4. أعد التشفير/التوقيع حسب الحاجة.

5. ألغ القديمة بعد نافذة آمنة.

تدوير signing key يختلف عن encryption key؛ حذف مفتاح تشفير قد يجعل البيانات غير قابلة للقراءة.

الوصول والتدقيق

- least privilege وفصل admin عن runtime.
- audit لكل قراءة/تغيير وإلغاء.
- approval أو break-glass للعمليات شديدة الحساسية.
- تنبيه على access غير معتاد.
- backups مشفرة واختبار recovery للمفاتيح المطلوبة.

عند التسريب

أوقف/قيّد الاستخدام، دوّر السر، حدّد النطاق من audit logs، أصلح المصدر، وافحص history وartifacts وlogs. حذف السر من Git لا يلغيه. لا تطبع قيم secrets في رسائل CI.

مرجع

06

المسار · 5 درسا

قواعد البيانات وPDO

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

قواعد البيانات و PDO

اتصال آمن، Prepared Statements، ORM، Transactions، ACID، الأقفال والفهارس.

يحوّل هذا القسم الصفحات 67-70 إلى مسار كامل يبدأ من اتصال PDO وينتهي بتصميم معاملات آمنة وقابلة للتوسع.

1. PDO والاتصال الآمن — DSN والإعدادات والاستعلامات المحضّرة.
2. ORM وأنماط الوصول للبيانات — Active Record و Data Mapper و N+1.
3. Transactions و Auto-commit — ACID و Commit و Rollback.
4. Isolation والأقفال و Deadlocks — مشاكل التزامن وإعادة المحاولة و Savepoints.
5. تصميم المخطط والفهارس و Migrations — القيود والفهارس و قياس الأداء.



نصيحة

ابدأ بـ PDO و SQL قبل ORM. الأداة لا تلغي ضرورة فهم الاستعلامات والفهارس والمعاملات.

1. PDO والاتصال الآمن

إنشاء اتصال PDO وضبط الأخطاء وتنفيذ Prepared Statements ومنع SQL Injection.

لماذا PDO؟

PHP Data Objects واجهة موحدة للتعامل مع عدة drivers مثل MySQL و PostgreSQL و SQLite. التوحيد لا يعني أن SQL وخصائص قواعد البيانات متطابقة؛ ارجع دائمًا لتوثيق driver.

الاتصال

```
<?php
declare(strict_types=1);

$dsn = 'mysql:host=127.0.0.1;port=3306;dbname=shop;charset=utf8mb4';

$pdo = new PDO(
    $dsn,
    getenv('DB_USER') ?: throw new RuntimeException('DB_USER missing'),
    getenv('DB_PASSWORD') ?: throw new RuntimeException('DB_PASSWORD missing'),
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

- لا تحفظ credentials داخل Git.
- استخدم charset في DSN، وليس query يدويًا بعد الاتصال.
- الاستثناءات تجعل الفشل واضحًا، لكن لا تعرض رسالتها للمستخدم.
- `ATTR_EMULATE_PREPARES=false` يطلب native prepares عندما يدعمها driver؛ اختبر SQL الفعلي وخصائص driver بدل اعتباره قانونًا عامًا.

Prepared Statements

```
$stmt = $pdo->prepare(
    'SELECT id, name, email
    FROM users
    WHERE email = :email AND status = :status'
);

$stmt->execute([
    'email' => $email,
    'status' => 'active',
]);

$user = $stmt->fetch(); // array|false
```

الplaceholder يفصل قيمة البيانات عن تركيب SQL، فيمنع تحويل quote داخل القيمة إلى أمر SQL.

```
$stmt = $pdo->prepare(
    'INSERT INTO products (name, price_cents)
    VALUES (:name, :price)'
);

$stmt->execute([
    'name' => $name,
    'price' => $priceCents,
]);

$id = (int) $pdo->lastInsertId();
```

ما لا يمكن ربطه؟

الparameter يمثل data literal كاملاً؛ لا يمثل اسم جدول/عمود أو keyword أو اتجاه ترتيب:

```
$allowedSort = ['name', 'created_at', 'price_cents'];
$sort = $_GET['sort'] ?? 'created_at';

if (!in_array($sort, $allowedSort, true)) {
    $sort = 'created_at';
}

$sql = "SELECT * FROM products ORDER BY {$sort} DESC";
$rows = $pdo->query($sql)->fetchAll();
```

استخدم allow-list للأسماء. ولا تربط قائمة **IN** كلها بplaceholder واحد:

```
$ids = [4, 7, 9];
$marks = implode(',', array_fill(0, count($ids), '?'));
$stmt = $pdo->prepare("SELECT * FROM products WHERE id IN ({$marks})");
$stmt->execute($ids);
```

تعامل مع القائمة الفارغة منفصلاً، وقيد عدد العناصر.

bindValue أم execute array؟

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = :id');
$stmt->bindValue('id', $id, PDO::PARAM_INT);
$stmt->execute();
```

`bindValue` يربط القيمة الآن. `bindParam` يربط المتغير بالمرجع وقد يفاجئك إذا تغير. تمرير array إلى `execute` أبسط غالبًا، واستخدم النوع الصريح عندما يؤثر في driver/query plan.

Fetching والذاكرة

- `fetch()` : صف واحد أو `false`.
 - `fetchAll()` : كل الصفوف؛ قد تستهلك ذاكرة كبيرة.
 - `fetchColumn()` : عمود واحد.
 - `rowCount()` : موثوق عادة لعمليات التعديل، وسلوكه مع SELECT يختلف بين drivers.
- مرّ على النتائج الكبيرة صفًا صفًا، وحدد الأعمدة بدل `SELECT *` في الكود الإنتاجي.

معالجة الخطأ

```
try {
    $user = $stmt->fetch();
} catch (PDOException $e) {
    error_log($e);
    throw new RuntimeException('Database operation failed', previous: $e);
}
```

في المنتج أظهر رسالة عامة وRequest ID، وسجل التفاصيل في مكان محمي دون credentials أو بيانات شخصية غير لازمة.

2. ORM وأنماط الوصول للبيانات

معنى ORM و Active Record و Data Mapper والعلاقات ومشكلة N+1 وحدود التجريد.

ما ORM؟

Object-Relational Mapping يربط صفوف الجداول بكائنات وخصائص وعلاقات. يقلل SQL المتكرر لكنه لا يلغي قاعدة البيانات ولا تكلفة الاستعلام.

```
Object/Entity ↔ Mapper/Model ↔ Table Row
User           ↔ ORM           ↔ users
```

Active Record

الكائن يمثل صفًا ويحتوي عمليات الحفظ والاستعلام. مثل Laravel Eloquent مشهور:

```
$user = User::findOrFail($id);
$user->name = 'Omar';
$user->save();

$orders = Order::query()
    ->where('status', 'paid')
    ->latest()
    ->get();
```

مميزاته: مباشر وسريع في CRUD. عيوبه: قد يخلط منطق المجال بالتخزين، ويصعب فصل الاختبارات في المجالات المعقدة.

Data Mapper

Entity تبقى أقرب إلى منطق المجال، و Mapper/Repository يدير التخزين. مثل Doctrine ORM:

```
$user = $entityManager->find(User::class, $id);
$user->rename('Omar');
$entityManager->flush();
```

مميزاته: فصل أقوى ونمذجة مجال أغنى. عيوبه: مفاهيم وحالة Unit of Work و Identity Map وتعقيد أكبر.



ملاحظة

Eloquent ORM مثل Django Active Record-like ORM، وليس مكتبة PHP. ذكره مفيد للمقارنة فقط؛ في PHP قارن Eloquent وDoctrine.

العلاقات

- One-to-one.
- One-to-many.
- Many-to-many عبر join table.
- Foreign keys تحمي التكامل في قاعدة البيانات، لا تعتمد على ORM وحده.

حدد cascade بعناية؛ حذف parent لا يجب أن يمسح بيانات كثيرة بالمفاجأة.

N+1 Queries

```

$orders = Order::all();

foreach ($orders as $order) {
    echo $order->customer->name; // query قد ينفذ
}

```

الحل يكون eager loading أو join/select مصممًا:

```

$orders = Order::with('customer')->get();

```

قيس عدد الاستعلامات وزمنها، ولا تجعل eager load كل العلاقات افتراضيًا لأنه قد ينقل بيانات ضخمة.

Serialization و Mass Assignment

لا تمرر `$_POST` كاملاً إلى model:

```

$user->fill([
    'name' => $validated['name'],
    'email' => $validated['email'],
]);

```

وإلا قد يمر حقل مثل `is_admin` . استخدم allow-list و DTO/validated input. كذلك لا تحول entity كاملة إلى JSON إن كانت تحمل password hash أو secrets.

متى نستخدم SQL مباشرًا؟

- تقارير وتجميعات معقدة.
- Bulk operations.
- استعلامات تعتمد على خصائص قوية في قاعدة محددة.
- مسار أداء حرج بعد القياس.

يمكن الجمع بين ORM و Query Builder و PDO. الهدف وضوح وصحة وأداء، لا الالتزام العقائدي بأداة واحدة.

3. Transactions و ACID

Auto-commit و beginTransaction و commit و rollBack وتطبيق ACID في عمليات متعددة الخطوات.

لماذا Transaction؟

بدون Transaction قد تنجح خطوة وتفشل أخرى، فنتترك بيانات نصف مكتملة. في وضع Auto-commit المعتاد يصبح كل statement مستقلاً ما لم تبدأ معاملة صريحة.

مثال تحويل رصيد:

1. خصم من الحساب A.

2. إضافة للحساب B.

3. تسجيل الحركة.

يجب أن تنجح كلها أو لا يحدث شيء.

ACID

- **Atomicity**: المعاملة وحدة واحدة؛ Commit كامل أو Rollback.
 - **Consistency**: تنتقل البيانات بين حالات تحترم القيود والقواعد. التطبيق والconstraints مسؤولان أيضاً.
 - **Isolation**: نتائج المعاملات المتزامنة لا تتداخل بطريقة تكسر المستوى المختار.
 - **Durability**: بعد Commit الناجح تبقى النتيجة رغم الأعطال ضمن ضمانات النظام.
- ACID ليس معناه «لا تحدث أخطاء» ولا أن كل قواعد البيانات والإعدادات تعطي المستوى نفسه.

```

function transfer(PDO $pdo, int $from, int $to, int $amount): void
{
    if ($amount <= 0 || $from === $to) {
        throw new InvalidArgumentException('Invalid transfer');
    }

    $pdo->beginTransaction();

    try {
        $lock = $pdo->prepare(
            'SELECT id, balance_cents
            FROM accounts
            WHERE id IN (?, ?)
            ORDER BY id
            FOR UPDATE'
        );
        $lock->execute([$from, $to]);
        $accounts = $lock->fetchAll();

        if (count($accounts) !== 2) {
            throw new RuntimeException('Account not found');
        }

        $balances = array_column($accounts, 'balance_cents', 'id');
        if ((int) $balances[$from] < $amount) {
            throw new RuntimeException('Insufficient balance');
        }

        $debit = $pdo->prepare(
            'UPDATE accounts
            SET balance_cents = balance_cents - :amount
            WHERE id = :id'
        );
        $credit = $pdo->prepare(
            'UPDATE accounts
            SET balance_cents = balance_cents + :amount
            WHERE id = :id'
        );

        $debit->execute(['amount' => $amount, 'id' => $from]);
        $credit->execute(['amount' => $amount, 'id' => $to]);

        $pdo->commit();
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) {
            $pdo->rollBack();
        }
        throw $e;
    }
}

```

ترتيب قفل الحسابات حسب ID يقلل احتمال Deadlock لكنه لا يلغيه. أسماء الصياغة مثل **FOR UPDATE** ودعمها تختلف حسب قاعدة البيانات.

حدود المعاملة

اجعلها قصيرة:

- لا تنتظر API أو بريدًا داخل lock إن أمكن.
- تحقق من المدخلات قبل البداية.
- ضع كل تغييرات قاعدة البيانات اللازمة داخلها.
- لا تُرجع نجاحًا قبل Commit.
- لا تمسك Transaction أثناء عرض صفحة للمستخدم.

لإرسال حدث/رسالة بعد نجاح البيانات استخدم **Transactional Outbox**: اكتب الحدث في جدول داخل المعاملة، ثم ينشره worker لاحقًا.

Implicit Commit و DDL

بعض الأنظمة، ومنها حالات في MySQL، تنفذ implicit commit عند أوامر مثل `CREATE TABLE` و `DROP TABLE`. لا تفترض أن كل DDL يمكن Rollback؛ افحص محرك قاعدة البيانات.

ينطبق التنبيه كذلك على Oracle: معاملات DML تدعم ACID، بينما أوامر DDL تنفذ عادة implicit commit قبل الأمر وبعده. لا تخلط تغيير schema مع business transaction وتتوقع التراجع عن الاثنين كوحدة واحدة.

أخطاء شائعة

- بدء المعاملة بعد أول UPDATE.
- نسيان Rollback في catch.
- ابتلاع exception وإكمال الطلب.
- الاعتماد على Transaction دون constraints.
- إجراء external side effect ثم Rollback؛ قاعدة البيانات لا تستطيع التراجع عن بريد أُرسِل.

4. Isolation والأقفال و Deadlocks

مشكلات التزامن ومستويات العزل والأقفال و Deadlocks و Savepoints وسياسة إعادة المحاولة.

لماذا التزامن صعب؟

قد يقرأ طلبان القيمة نفسها ثم يكتبان نتائج متعارضة. Transaction وحدها لا تعني أن كل المعاملات تعمل واحدًا بعد الآخر؛ **Isolation Level** يحدد ما يمكن لكل معاملة رؤيته.

مشكلات معروفة:

- **Dirty Read**: قراءة تغيير لم يُعمل له Commit.
- **Non-repeatable Read**: قراءة الصف مرتين والحصول على قيمتين بسبب Commit آخر.
- **Phantom Read**: إعادة query شرطية وظهور/اختفاء صفوف.
- **Lost Update**: تحديث يطغى على تحديث متزامن.
- **Write Skew**: معاملتان تقرأان حالة مشتركة وتكتبان صفوفًا مختلفة فتتكسر قاعدة.

مستويات العزل

المستوى	الفكرة
Read Uncommitted	أقل عزل؛ يسمح برؤية أوسع لتغييرات غير مثبتة حسب النظام
Read Committed	كل statement يرى بيانات committed
Repeatable Read	القراءات داخل المعاملة أكثر ثباتًا
Serializable	أقرب لتنفيذ متسلسل، وقد يزيد الانتظار/الفشل القابل لإعادة المحاولة

التفاصيل تختلف بشدة حسب MySQL/PostgreSQL/SQL Server وآلية MVCC. لا تحفظ جدولًا عامًا فقط؛ اختبر قاعدة بياناتك.

Pessimistic locking

```
SELECT id, stock
FROM products
WHERE id = :id
FOR UPDATE
```

يقفل الصفوف المختارة عادة حتى نهاية المعاملة. استخدم index ليصل إلى الصفوف المقصودة؛ query سيئة قد تقفل نطاقًا أوسع.

أضف `version`:

```
UPDATE products
SET stock = :new_stock, version = version + 1
WHERE id = :id AND version = :old_version
```

إذا كان `affected rows = 0` فقد عدّل طرف آخر الصف؛ أعد القراءة وقرر `retry` أو `409 conflict`.

Deadlock

يحدث عندما تمسك معاملة A قفلاً تحتاجه B، وتمسك B قفلاً تحتاجه A. قاعدة البيانات تقتل إحدى المعاملات لحل الحلقة.

قلل الاحتمال عبر:

- ترتيب الوصول للموارد دائماً بنفس الطريقة.
- معاملات قصيرة.
- فهارس مناسبة.
- عدم طلب تفاعل المستخدم داخل المعاملة.
- `Retry` محدود مع `backoff` و `jitter` للأخطاء القابلة للإعادة.

يجب أن تكون العملية `idempotent` أو تستخدم `idempotency key` حتى لا يتكرر `side effect` عند `retry`.

Savepoints والمعاملات المتداخلة

PDO لا يوفر `nested transactions` حقيقية بطريقة عامة. بعض قواعد البيانات تدعم `Savepoints`:

```
$pdo->beginTransaction();

try {
    $pdo->exec('SAVEPOINT before_optional_step');
    // optional database work
    $pdo->exec('RELEASE SAVEPOINT before_optional_step');
    $pdo->commit();
} catch (Throwable $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    throw $e;
}
```

للرجوع الجزئي استخدم `ROLLBACK TO SAVEPOINT ...` وفق `dialect`. انتبه: `rollback` إلى `savepoint` لا يلغي ضرورة `commit/rollback` للمعاملة الخارجية.

لا تجعل isolation دواءً سحريًا

رفع المستوى إلى Serializable قد يحسن الصحة لكنه يزيد conflicts والـretries. ابدأ من invariants: ما القاعدة التي لا يجوز كسرها؟
ثم استخدم constraints وatomic updates وlocks أو optimistic version بما يناسب.

5. تصميم المخطط والفهارس و Migrations

أنواع البيانات والقيود والعلاقات والفهارس المركبة و EXPLAIN وإدارة تغييرات المخطط.

اجعل قاعدة البيانات تحمي قواعدها

التطبيق قد يحتوي bug أو توجد أداة إدارية أو worker آخر. ضع القيود الأساسية في Schema أيضًا:

```
CREATE TABLE orders (
  id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  status VARCHAR(30) NOT NULL,
  total_cents BIGINT NOT NULL CHECK (total_cents >= 0),
  created_at TIMESTAMP NOT NULL,
  CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

استخدم:

- **NOT NULL** للقيم الإلزامية.
- **UNIQUE** للهوية الفريدة.
- **CHECK** للقواعد المحلية المدعومة.
- Foreign keys للتكامل المرجعي.
- أنواعًا مناسبة؛ المال غالبًا integer minor units أو DECIMAL، لا float.
- timestamps مع سياسة timezone واضحة.

الفهرس

الفهرس بنية إضافية تسرع lookup/sort/join مقابل مساحة وكلفة كتابة.

```
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at);
```

ترتيب الأعمدة مهم. يفيد هذا غالبًا queries تبدأ بـ **user_id** ثم **status** وتستفيد من **created_at**. ليس ضمانًا أن index لكل عمود أو كل query أفضل.

EXPLAIN والقياس

```
EXPLAIN
SELECT id, total_cents
FROM orders
WHERE user_id = 42 AND status = 'paid'
ORDER BY created_at DESC
LIMIT 20;
```

افحص plan والصفوف المقدرة والفعالية إن كانت الأداة تدعم analysis. اختبر بيانات بحجم قريب من الإنتاج، وراقب slow query log وp95/p99.

أسباب بطء شائعة:

- $N+1$.
- `SELECT *` ونقل بيانات غير لازمة.
- Pagination عميقة بـ OFFSET.
- Functions/casts تمنع استخدام index.
- Missing أو low-selectivity index.
- Query كثيرة بدل batch.

Pagination

Offset سهل:

```
SELECT * FROM orders
ORDER BY id DESC
LIMIT 20 OFFSET 10000
```

لكنه يزداد كلفة وقد يتغير مع إدخالات متزامنة. Keyset pagination:

```
SELECT *
FROM orders
WHERE id < :last_seen_id
ORDER BY id DESC
LIMIT 20
```

أفضل غالبًا للfeeds الكبيرة مع ترتيب ثابت.

Migrations

Migration تغيير versioned قابل للمراجعة والتطبيق آليًا:

```
20260921_001_create_orders
20260921_002_add_orders_status_index
```

قواعد مهمة:

1. خذ backup واختبر الاسترجاع.
2. جرّب على نسخة وحجم بيانات واقعي.
3. اجعل التغيير متوافقًا أثناء النشر المرحلي.
4. افصل schema change المكلف عن backfill الضخم.
5. راقب locks ووقت التنفيذ.
6. لا تفترض أن rollback آمن لكل DDL.

نمط Expand/Contract: أضف العمود/البنية الجديدة أولاً، انشر كودًا يدعم القديم والجديد، انقل البيانات، ثم احذف القديم في نشر لاحق.

Seed وبيانات الاختبار

Migrations للبنية والتغييرات الحتمية. Seed ينشئ بيانات مرجعية أو تطويرية. لا تضع بيانات شخصية إنتاجية في fixtures، واجعل seed قابلاً للتكرار دون duplication.

Checklist

- هل كل query تستخدم prepared values؟
- هل القيود تمنع الحالة المستحيلة؟
- هل transaction تغطي كل التغييرات المطلوبة فقط؟
- هل الفهرس يخدم query مقاسة؟

English edition

A print-ready edition containing every published track and lesson in the documentation.

01

TRACK · 13 LESSONS

Programming and web fundamentals

Progressive explanations, practical examples, and corrections that take each idea into real code.

Web and networking basics

A corrected, expanded track for pages 1–11, from entering a URL to PHP and the response.

This section turns pages **1–11** of the notes into a structured learning path. It retains every readable concept, corrects oversimplifications, and adds practical examples.

Learning path

1. The Web and request flow
2. DNS and IP addresses
3. TCP, UDP, and packets
4. URLs, ports, and HTTP
5. HTTP messages and state
6. HTTPS, TLS, and certificates
7. Inside the server
8. Problem solving and algorithms
9. How browsers render a page
10. HTTP caching and compression
11. Same-origin policy and CORS
12. Real-time communication and webhooks
13. API design

```
URL → DNS → IP + Port → TCP/QUIC + TLS → HTTP Request
      → CDN/WAF/Load Balancer → Web Server → PHP → Database
      → HTTP Response → Browser
```

This is a teaching map. Caches, reused connections, and application architecture can skip optional stages.

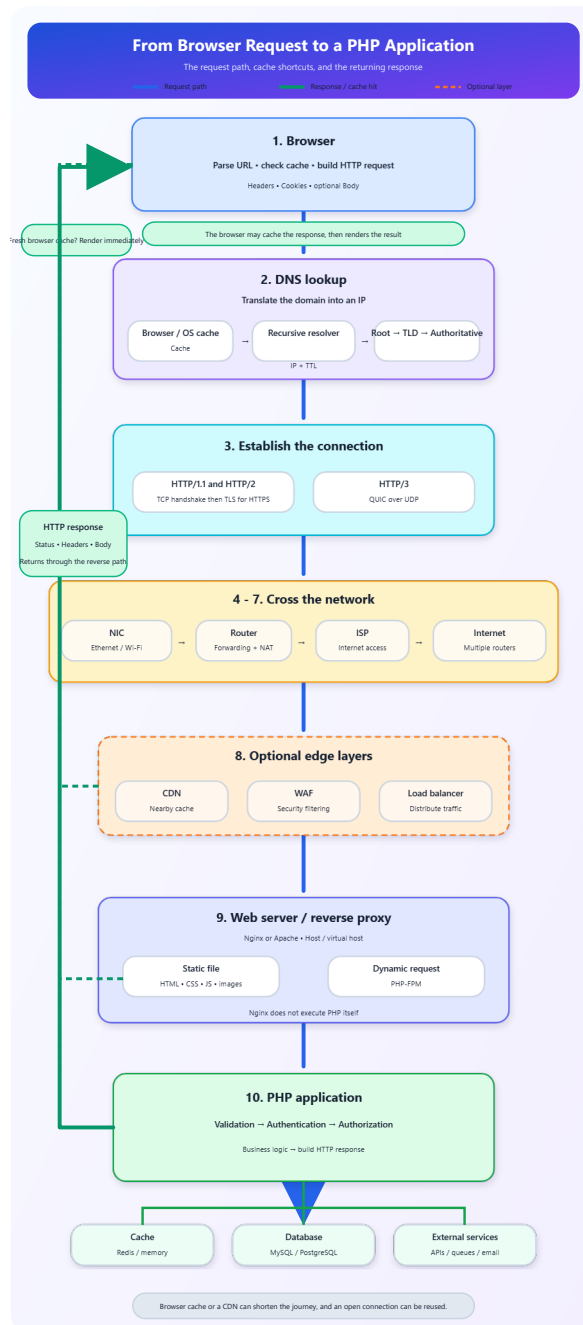
1. The Web and request flow

Internet vs Web, client/server, and the complete request-response journey.

The **Internet** is the global network infrastructure. The **Web** is one service on top of it, using HTTP/HTTPS and URLs. Email, file transfer, calls, and games also use the Internet.

A client requests a service; a server receives and processes requests. These are roles, so both may run on one development machine.

When a browser opens a URL it checks local cache, resolves DNS, connects through the NIC/router/ISP, establishes TCP or QUIC and TLS, sends HTTP, and may pass through a CDN, WAF, or load balancer. A web server serves static content or forwards dynamic work to PHP-FPM. The application may use cache/database services and returns an HTTP response.



[Open the diagram full size](#)

```
curl -i "https://example.com/"
curl -v "https://example.com/" -o NUL
```

The second command exposes DNS, connection, and TLS details. Use `/dev/null` instead of `NUL` on Linux/macOS.

This is a comprehensive teaching route, not a mandatory fixed path: browser/CDN cache can answer early, and a small app may have no load balancer or database.

2. DNS and IP addresses

Name resolution through caches, recursive resolvers, root, TLD, and authoritative servers.

An IP address identifies a network interface for routing. Private and public addresses differ, and a home router commonly performs NAT. `127.0.0.1` and `::1` are loopback addresses; `localhost` normally resolves to them.

DNS is a distributed record system. Common records include `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias), `MX` (mail), `TXT`, and `NS`.

Resolution usually checks browser and OS caches, then asks a recursive resolver. On a cache miss, that resolver follows referrals from root to TLD to an authoritative server and returns the answer with a TTL.

```
Browser/OS Cache → Recursive Resolver
    ↓ cache miss
    Root → TLD → Authoritative
```

The browser does not normally query every level itself, and the full chain is not repeated while a cached answer is valid. DNS may return several IPs for balancing or locality; it locates a server but does not deliver HTML or execute PHP.

```
nslookup example.com
nslookup -type=AAAA example.com
nslookup -type=MX example.com
```

3. TCP, UDP, and packets

Handshake, segments, packets, frames, ACKs, retransmission, flow, and congestion control.

Application data is encapsulated as a TCP segment or UDP datagram, then an IP packet, then a local Ethernet/Wi-Fi frame. Headers contain information such as ports, IPs, TCP sequence/flags, and IP TTL/hop limit. Large content is commonly split and reassembled.

TCP provides a reliable ordered byte stream and normally starts with:

```
Client                                Server
| ----- SYN -----> |
| <----- SYN + ACK ----- |
| ----- ACK -----> |
| ===== Application Data =====> |
```

Sequence numbers and acknowledgements let TCP reorder data and retransmit loss. An ACK confirms transport receipt, not completion of PHP business logic.

Flow control uses the receiver's advertised window to avoid overwhelming it. Congestion control changes the sender's congestion window to protect the network. Effective in-flight data is constrained by both.

UDP sends independent datagrams without a traditional connection, guaranteed delivery, or ordering. It has less protocol overhead and is used by DNS, live media, and games. TCP is common for HTTP/1.1 and HTTP/2. HTTP/3 uses QUIC over UDP; QUIC adds its own reliability, security, and congestion control.

```
ping example.com
tracert example.com
```

Use `tracert` instead of `tracert` on Linux/macOS. Ping uses ICMP, not TCP.

4. URLs, ports, and HTTP

URL anatomy, ports, sockets, ranges, and the role of HTTP.

HTTP is an application protocol defining request/response semantics for resources. It is not the network connection itself; it runs over transports such as TCP or QUIC. HTTP is stateless by default, so applications add cookies, sessions, or tokens for continuity.

```
https://example.com:443/products/42?currency=EGP#reviews
scheme  host      port path      query      fragment
```

The fragment is normally browser-local and is not sent in the HTTP request. Never place passwords, tokens, or other secrets in URLs because history, logs, analytics, and referrers may expose them.

A port is a 16-bit number from 0 to 65535 identifying a service in the operating system. Common values include FTP 21, SSH 22, SMTP 25, DNS 53, HTTP 80, HTTPS 443, MySQL 3306, and PostgreSQL 5432. A conventional port is configuration, not security.

A socket endpoint can be simplified as protocol + IP + port. A TCP connection is distinguished by client IP/port and server IP/port, allowing many clients to share server port 443.

```
php -S localhost:8000
```

In `http://localhost:8000/hello.php`, the scheme is HTTP, host is localhost, port is 8000, and path is `/hello.php`. Omitting a port means the scheme's default, not "no port".

5. HTTP messages and state

Methods, fields, content, status codes, cookies, sessions, and tokens.

An HTTP/1.1 request contains a request line, header fields, a blank line, and optional content:

```
POST /api/orders?notify=1 HTTP/1.1
Host: shop.example
Accept: application/json
Content-Type: application/json
Authorization: Bearer ey...

{"product_id":42,"count":2}
```

GET retrieves, HEAD is GET without response content, POST performs resource-specific processing, PUT replaces, PATCH partially updates, DELETE removes, and OPTIONS describes communication options. Safe/idempotent semantics are contracts, not authorization. Never implement destructive work through GET.

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/orders/901

{"id":901,"status":"pending"}
```

Response classes are 1xx informational, 2xx success, 3xx redirection/cache, 4xx request/client error, and 5xx server error. 401 generally indicates missing/invalid authentication; 403 indicates understood but unauthorized.

HTTP does not automatically remember an earlier request. Cookies are browser-stored values sent by matching rules; sessions are server-side state commonly linked by a cookie ID; tokens are client-sent credentials. Not every token is a JWT, and JWT does not automatically replace session design.

```
<?php
header('Content-Type: application/json; charset=utf-8');

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'GET') {
    http_response_code(405);
    header('Allow: GET');
    echo json_encode(['error' => 'Method not allowed']);
    exit;
}

$name = trim((string) ($_GET['name'] ?? 'Guest'));
echo json_encode(['message' => "Hello {$name}"], JSON_THROW_ON_ERROR);
```

6. HTTPS, TLS, and certificates

TLS guarantees, handshake, keys, certificates, and DV/OV/EV validation.

HTTPS is HTTP carried through TLS. TLS provides confidentiality, integrity, and authentication of the server identity represented by its certificate. It does not repair SQL injection, XSS, or broken authorization.

Asymmetric cryptography uses public/private keys for signatures and secure agreement; symmetric traffic keys efficiently protect application data. The note that a client simply encrypts a session key with the server public key describes old RSA key exchange. TLS 1.3 normally uses ephemeral key agreement such as ECDHE: both peers derive traffic keys rather than transmitting the final symmetric key, providing forward secrecy.

```
ClientHello (versions, ciphers, key share, name)
  ↓
ServerHello (choice, key share, certificate, signature)
  ↓
Certificate/signature validation
  ↓
Derived traffic keys → encrypted HTTP
```

A certificate binds a public key to domain names and chains to a trusted CA. Clients check names in SAN, validity, chain, key usage, and proof that the server holds the private key.

DV validates domain control; OV adds organisation checks; EV applies broader organisational policy. OV/EV do not create stronger encryption than DV; they differ in identity validation.

Never ignore production certificate errors, disable API TLS verification, commit a private key, or mistake Base64 for encryption.

```
curl -v https://example.com/ -o NUL
```

Use `/dev/null` on Linux/macOS.

7. Inside the server

Load balancers, web servers, virtual hosts, PHP-FPM, applications, and the response path.

```
Network → Load Balancer → Web Server
├─ Static file → Response
└─ PHP-FPM → App → Database/Cache/API → Response
```

A load balancer distributes traffic across healthy backends. Layer 4 works with transport connections; Layer 7 understands HTTP. It may terminate TLS, perform health checks, and add forwarding headers. Only trust forwarded client IP headers from configured trusted proxies.

Nginx or Apache selects a virtual host, serves static assets directly, reverse-proxies services, or forwards dynamic PHP through FastCGI. Nginx does not execute PHP inside its own process; PHP-FPM workers run the entry point and return headers/content.

The application routes the request, runs middleware, validates input/authentication/authorization, executes business logic, and may use a database, cache, queue, or external API.

```
<?php
header('Content-Type: application/json; charset=utf-8');

try {
    echo json_encode(['id' => 42, 'name' => 'Keyboard'], JSON_THROW_ON_ERROR);
} catch (Throwable $e) {
    error_log($e);
    http_response_code(500);
    echo json_encode(['error' => 'Internal server error']);
}
```

Do not expose stack traces or secrets. On the return path, layers may add compression, security headers, and caching. A browser/CDN cache hit skips the app; a static asset can stop at Nginx; a dynamic request reaches PHP-FPM. A proxy commonly returns 502 for an invalid/unavailable upstream and 504 for an upstream timeout, while 500 generally comes from the processing server/application.

Use request IDs and timing to trace the whole path rather than measuring PHP alone.

8. Problem solving and algorithms

Turn a problem into steps, write pseudocode, choose data structures, and reason about complexity.

From a problem to a program

Before coding, define inputs, outputs, rules, small testable steps, and boundary or invalid cases.

```
INPUT items, discountPercent
IF discountPercent < 0 OR discountPercent > 100
  RETURN error
total = 0
FOR EACH item IN items
  IF item.price < 0 OR item.quantity < 1
    RETURN error
  total = total + item.price * item.quantity
RETURN total * (1 - discountPercent / 100)
```

Pseudocode communicates logic without binding it to PHP, JavaScript, or another language.

Choose a data structure

Need	Useful structure
Ordered values with duplicates	List/array
Lookup by key	Map/associative array
Unique membership	Set
First in, first out	Queue
Last in, first out	Stack

A good structure improves clarity and cost. A map can replace a repeated nested search with direct lookup.

Big O

Big O describes how time or memory grows with input size, not exact seconds.

Complexity	Example
$O(1)$	Read a known key
$O(\log n)$	Binary search in sorted data
$O(n)$	Scan a list once
$O(n \log n)$	Common efficient sorts
$O(n^2)$	Compare every item with every item

Correctness and clarity come first. Measure before optimizing because constants and real workloads still matter.

Exercise

Given orders containing `customer_id` and `total`, design an algorithm that returns a total per customer. Define edge cases, use a map for $O(n)$ traversal, and write three test cases before the implementation.

9. How browsers render a page

Navigation, DOM, CSSOM, render trees, JavaScript execution, and practical DevTools analysis.

From a response to pixels

```
HTML bytes -> decode -> tokens -> DOM
CSS bytes  -> parse  -> CSSOM
DOM + CSSOM -> Render Tree -> Layout -> Paint -> Composite
```

- **DOM** represents elements and content.
- **CSSOM** contains parsed style rules.
- **Layout** calculates geometry.
- **Paint** draws text, color, and borders.
- **Composite** assembles layers into the final frame.

Render-critical CSS can delay paint. A classic script can pause HTML parsing while it downloads and executes.

JavaScript loading

```
<script src="/app.js" defer></script>
<script type="module" src="/main.js"></script>
```

`defer` preserves order and runs after DOM parsing. Modules are deferred by default. `async` runs as soon as it is ready and does not preserve ordering between independent scripts.

Repeated layout reads and writes can cause layout thrashing. Batch reads and writes, and prefer compositor-friendly properties such as `transform` where appropriate.

Event loop and responsiveness

Long JavaScript tasks block input and rendering. Inspect Largest Contentful Paint, Interaction to Next Paint, Cumulative Layout Shift, and main-thread long tasks.

DevTools workflow

1. Use **Network** with cache disabled for controlled tests.
2. Inspect DNS, connection, TLS, TTFB, and download phases.

3. Check headers, transferred size, and protocol.
4. Use **Performance** for tasks, layout, and paint.
5. Use **Elements** for DOM and computed styles.



Tip

A high TTFB may come from the network, CDN, PHP, or the database. Correlate browser traces with server logs and profiling.

10. HTTP caching and compression

Freshness, validation, Cache-Control, ETag, Vary, CDNs, gzip, and Brotli.

Why cache a response?

Caching reduces latency, bandwidth, PHP work, and database load. Copies may exist in a browser, proxy/CDN, or application, and each layer needs an explicit policy.

Freshness and validation

```
Cache-Control: public, max-age=60, s-maxage=300
ETag: "product-42-v7"
Vary: Accept-Encoding, Accept-Language
```

- `max-age` controls client freshness.
- `s-maxage` can target shared caches.
- `private` prevents shared reuse.
- `no-store` asks caches not to store.
- `no-cache` permits storage but requires validation before reuse.
- `Vary` adds selected request fields to the cache key.

An expired response can be revalidated with `If-None-Match`. If unchanged, the server returns `304 Not Modified` without a body. `Last-Modified` and `If-Modified-Since` are another validator pair.

PHP example

```
$etag = '' . hash('sha256', $json) . '';
header('Cache-Control: public, max-age=60');
header("ETag: {$etag}");

if (trim($_SERVER['HTTP_IF_NONE_MATCH'] ?? '') === $etag) {
    http_response_code(304);
    exit;
}

header('Content-Type: application/json; charset=utf-8');
echo $json;
```

Never mark personalized data `public`.

Compression

gzip and Brotli reduce text assets such as HTML, CSS, and JSON. Configure them in the web server or CDN, avoid recompressing already compressed images without evidence, and vary cached representations by `Accept-Encoding`.

Checklist

- Define who can store the response and for how long.
- Use fingerprinted filenames for immutable static assets.
- Test invalidation before choosing a long lifetime.
- Do not publicly cache personal responses or unsafe `Set-Cookie` responses.
- Monitor hit ratio, transferred size, and latency.

Reference

- [RFC 9111: HTTP Caching](#)

11. Same-origin policy and CORS

Origins, browser boundaries, simple requests, preflight, credentials, and safe configuration.

What is an origin?

An origin is **scheme + host + port**. `https://app.example` therefore differs from `http://app.example`, `https://api.example`, or a different port.

The same-origin policy stops JavaScript from reading many cross-origin resources without permission. It does not prevent every request and does not control server-to-server clients.

CORS response fields

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
Vary: Origin
```

Credentials cannot be combined with a wildcard origin. Match a strict allow-list and never blindly reflect an arbitrary `Origin`.

Preflight

```
OPTIONS /api/orders HTTP/1.1
Origin: https://app.example
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, Authorization
```

The server answers with allowed methods, fields, and an optional cache duration. Handle `OPTIONS` before middleware that requires credentials which preflight does not carry.

Minimal PHP handling

```
$allowed = ['https://app.example', 'https://admin.example'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';

if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: {$origin}");
    header('Vary: Origin');
    header('Access-Control-Allow-Credentials: true');
}

if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    header('Access-Control-Allow-Methods: GET, POST');
    header('Access-Control-Allow-Headers: Content-Type, Authorization');
    http_response_code(204);
    exit;
}
```



Danger

CORS is not authentication, authorization, or a replacement for CSRF protection. Non-browser clients can ignore it, so the server must validate every request.

12. Real-time communication and webhooks

Polling, long polling, SSE, WebSockets, webhooks, and choosing the smallest suitable model.

Communication models

Pattern	Direction	Good fit
Polling	Client asks periodically	Infrequent updates and simplicity
Long polling	A request waits for an event	Broad compatibility and lower latency
SSE	Server → browser	Text notifications and feeds
WebSocket	Persistent two-way	Chat, games, collaboration
Webhook	Server → server	Notify another system

WebSockets add connection state, heartbeat, backpressure, and scaling concerns. Choose the simplest pattern that meets the requirement.

Server-Sent Events

```
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
header('X-Accel-Buffering: no');

echo "event: order.updated\n";
echo 'data: ' . json_encode(['id' => 42, 'status' => 'paid']) . "\n\n";
flush();
```

Browsers reconnect automatically. Event IDs and `Last-Event-ID` can support resume. Long streams can occupy FPM workers, so validate the server architecture and timeouts.

Webhook receiver

1. Read the raw body.
2. Verify an HMAC signature and timestamp.
3. Reject replay outside a time window.
4. persist the event ID to deduplicate.
5. Acknowledge quickly and enqueue heavy work.

```
$expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);  
if (!hash_equals($expected, $signature)) {  
    http_response_code(401);  
    exit;  
}
```

Senders retry failed deliveries, so idempotency is required.

Operations

- Limit connection count, message rate, and size.
- Use heartbeat and detect dead peers.
- Authenticate connections and authorize every channel or event.
- Monitor reconnect rate, queue lag, and delivery failures.

13. API design

REST, RPC, GraphQL, resources, status codes, errors, pagination, versioning, and contracts.

Choose a style for the problem

- **REST** models resources with HTTP semantics.
- **RPC** exposes explicit operations such as `calculateShipping`.
- **GraphQL** lets clients select fields through a schema, but needs complexity controls and resolver-level authorization.

A label does not guarantee quality. Clear contracts, compatibility, security, and observability matter more.

Resources and HTTP

```
GET    /api/orders/42
POST   /api/orders
PATCH /api/orders/42
DELETE /api/orders/42
```

Use consistent nouns. Return `201` and `Location` for creation, `204` when no body is needed, and distinguish authentication, permission, absence, conflict, and validation failures instead of returning `200` for everything.

Validation and Problem Details

```
{
  "type": "https://docs.example/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": {"email": ["Invalid format"]},
  "request_id": "req_01j..."
}
```

Keep a stable error shape and never expose stack traces or SQL.

Pagination and filtering

```
GET /api/orders?status=paid&limit=20&cursor=eyJpZCI6OTAwfQ
```

Set a maximum page size. Cursor pagination usually behaves better for large changing datasets; offset pagination is simpler for limited navigation.

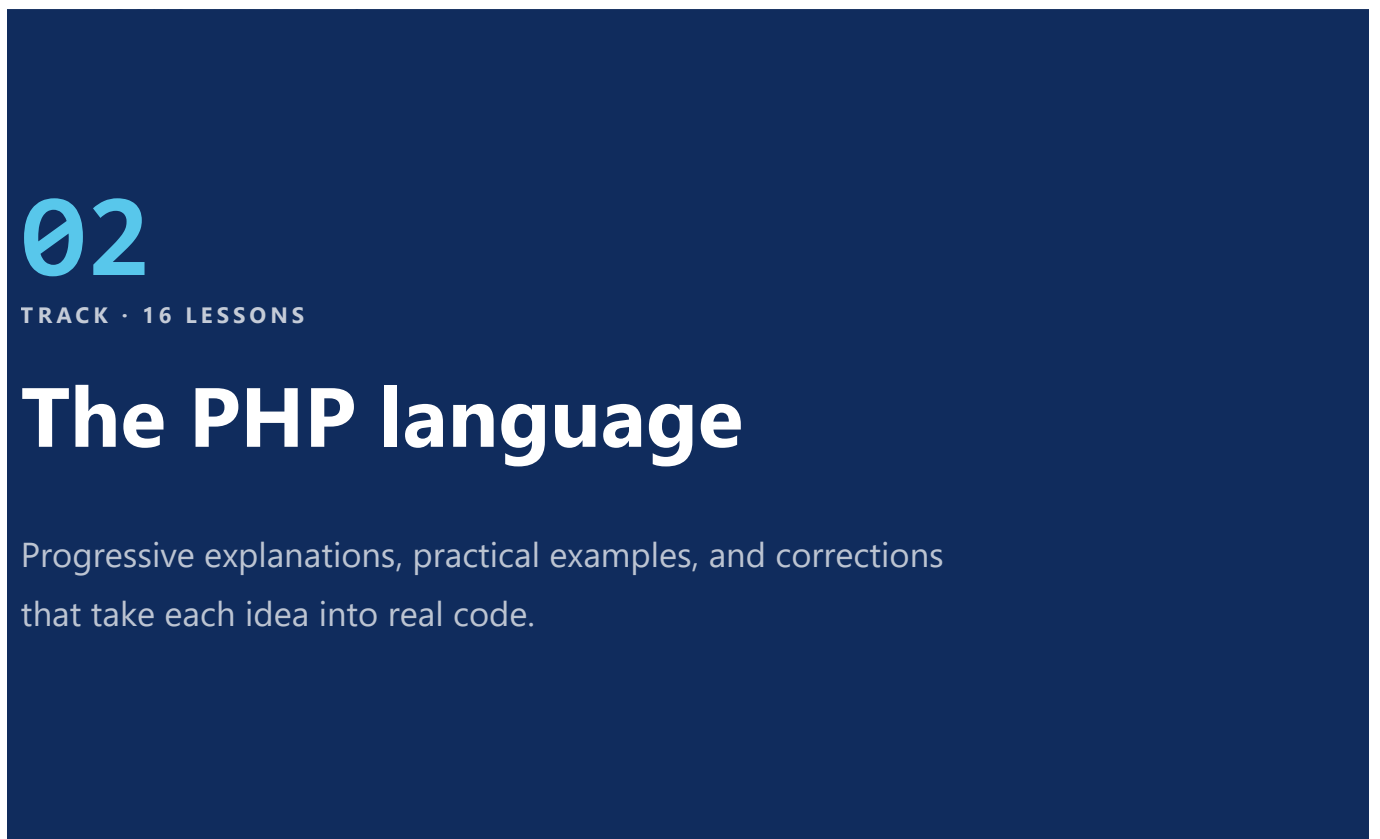
Compatibility and versions

- Prefer backward-compatible additions.
- Never silently change an existing field's meaning.
- Announce deprecation and removal dates.
- Contract-test consumers.
- Path or header versioning can work; consistency matters most.

Contract and operations

Publish a testable OpenAPI/schema contract. Include authentication, rate limits, idempotency, request IDs, timeouts, and observability. Design for partial failure and retries, not only the happy path.

References



02
TRACK · 16 LESSONS

The PHP language

Progressive explanations, practical examples, and corrections
that take each idea into real code.

PHP

A corrected, expanded PHP fundamentals track based on pages 12–60 of the notes.

This section turns pages **12–60** of the notes into a practical learning path. Every written concept is retained, unclear or outdated points are corrected, and each lesson adds runnable examples, edge cases, and security guidance.

Learning path

1. [Introduction and syntax](#)
2. [Variables, scope, and superglobals](#)
3. [Data types and the type system](#)
4. [Output, debugging, and constants](#)
5. [Conditions and loops](#)
6. [Expressions and operators](#)
7. [Functions, callbacks, and includes](#)
8. [Files, streams, JSON, and CSV](#)
9. [Uploads, cookies, and sessions](#)
10. [Namespaces and autoloading](#)
11. [Errors and exceptions](#)
12. [Arrays and transformation tools](#)
13. [Strings, Unicode, and regular expressions](#)
14. [Date, time, and timezones](#)
15. [From HTTP request to router and response](#)
16. [Modern PHP 8.4 and 8.5](#)



How to study

Run every example, change its inputs, and predict the result before running it again. The security notes are part of correct PHP usage, not optional extras.

```
<?php

declare(strict_types=1);

function greet(string $name): string
{
    return "Hello {$name}";
}

echo greet('Omar');
```

Examples target PHP 8.x. Features requiring a newer minor release are labelled explicitly.

1. Introduction and PHP syntax

What PHP is, request execution, tags, HTML templates, comments, and case sensitivity.

What PHP is

The name historically meant **Personal Home Page**; its recursive official expansion is **PHP: Hypertext Preprocessor**. PHP is an open-source, general-purpose language best known for server-side web development.

A request normally follows this path: browser → web server → PHP runtime → database/files/services when needed → HTML or JSON response → browser. PHP source code stays on the server.



Correction

“PHP runs line by line without compilation” is only a teaching shortcut. The engine compiles source to opcodes before execution, and modern versions can use JIT.

PHP can process forms, serve APIs, access databases and files, run CLI and cron jobs, send mail, create images/PDFs through libraries, and support both procedural and object-oriented programming. Desktop apps are possible through third-party tooling, but are not PHP's common use case.

Runtime and syntax

XAMPP, WAMP, MAMP, and LAMP bundle a web server, database, and PHP for local work. PHP also has a development server:

```
php -S localhost:8000 -t public
```

Do not use it as a production server.

```

<?php
$title = 'My shop';
?>
<!doctype html>
<html lang="en">
<head><title><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></title></head>
<body>
<?php if ($title !== ''): ?>
    <h1><?= htmlspecialchars($title) ?></h1>
<?php else: ?>
    <p>No title</p>
<?php endif; ?>
</body>
</html>

```

Statements end with `;`; compound blocks do not need a semicolon after `}`. In PHP-only files, omit the closing `?>` to avoid accidental output. Headers, cookies, and sessions must be started before output.

Comments use `//`, `#`, or `/* ... */`. Variable names are case-sensitive. Although keywords and function/class lookup are case-insensitive, always use the declared casing; constants are case-sensitive.

2. Variables, scope, and superglobals

Naming, assignment by value/reference, scope, static locals, and request superglobals.

PHP variables start with `$`; the name begins with a letter or underscore, may then contain digits, and is case-sensitive.

```
$name = 'Omar';
$count = 3;
echo "{$name}: {$count}";
```

Double-quoted strings interpolate variables; single-quoted strings normally do not. Prefer `{$name}` for clear interpolation.

```
$a = 10;
$b = $a; // value copy
$b++;

$x = 10;
$y =& $x; // both names refer to the same value container
$y++;
unset($y);
```

References are not manual C-style pointers; use them sparingly. Variable variables also exist:

```
$field = 'email';
$$field = 'omar@example.com';
echo $email;
```

Prefer arrays or objects when names may come from user input.

Scope

Global values are not automatically visible inside functions. `global $name` and `$GLOBALS['name']` expose them, but explicit arguments are easier to test.

```
function nextId(): int
{
    static $id = 0;
    return ++$id;
}
```

A local `static` value is initialized once and survives later calls in the same process. Included files inherit the scope at the include point.

Superglobals

`$_GET`, `$_POST`, `$_SERVER`, `$_FILES`, `$_COOKIE`, `$_SESSION`, and `$GLOBALS` are available in every scope.

```
$method = $_SERVER['REQUEST_METHOD'] ?? 'CLI';
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT) ?: 1;
$ids = $_GET['check_orders'] ?? []; // ?check_orders[]=10&check_orders[]=20
$ids = is_array($ids) ? array_map('intval', $ids) : [];
```

Input is never trusted merely because PHP parsed it. Validate it, escape output, and use prepared SQL statements.



`eval()`

`eval('echo "hello";');` executes a string as PHP code. Never feed it user input; ordinary applications should replace it with functions, callback maps, or classes.

3. Data types and the type system

Scalar, compound and special types, conversion, callables, iterables, and declarations.

PHP has scalar `bool`, `int`, `float`, and `string`; compound `array` and `object`; and special types including `null`, `resource`, `callable`, `iterable`, `mixed`, `void`, and `never`. Developers define classes, interfaces, and enums. Declarations may use unions such as `int|string` and intersections such as `Countable&Iterator`.

Falsy values are `false`, `0`, `0.0`, `""`, `"0"`, `[]`, and `null`. Prefer strict comparisons:

```
var_dump(0 == false); // true
var_dump(0 === false); // false
```

Integer literals may be decimal, `0o52` octal, `0x2A` hexadecimal, or `0b101010` binary. `/` performs division and `intdiv(7, 2)` returns `3`. Casting `(int) 3.9` truncates. Binary floating point is approximate, so use integer minor units or a decimal library for money.

Strings support single/double quotes, heredoc/nowdoc, indexing (including negative indexes), and `.` concatenation. Validate numeric strings before arithmetic instead of relying on juggling.

Arrays act as lists or maps. Objects combine state and behaviour. Enums model a finite set. A resource is a handle to an external resource such as a stream, though modern extensions often return objects.

- `void`: no useful returned value.
- `never`: execution never returns normally.
- `mixed`: any PHP value, including null.

```
$double = fn (int $n): int => $n * 2;

class Formatter
{
    public function upper(string $value): string { return strtoupper($value); }
    public function __invoke(string $value): string { return trim($value); }
}

$f = new Formatter();
$upper = $f->upper(...); // first-class callable
```

A callback is a callable passed for later execution. A Closure represents an anonymous function. An object with `__invoke()` is callable.

```
function numbers(int $max): iterable
{
    for ($i = 1; $i <= $max; $i++) {
        yield $i;
    }
}
```

`iterable` accepts arrays and `Traversable`. Every Generator is an Iterator, but not every Iterator is a Generator.

```
<?php
declare(strict_types=1);

function findUser(int $id, ?string $locale = null): array|null
{
    return $id > 0 ? ['id' => $id, 'locale' => $locale] : null;
}
```

Without strict mode PHP may coerce scalar arguments. Strictness belongs to the calling file and applies to scalar declarations; an integer is still accepted for a float declaration. Use `get_debug_type()` and `var_dump()` when inspecting values.

4. Output, debugging, and constants

echo, print, value inspection, regular, magic, and predefined constants.

`echo` and `print` are language constructs. `echo` has no return value and can accept multiple comma-separated arguments without parentheses; `print` accepts one value and returns `1`.

```
echo 'Hello', ' ', 'PHP', PHP_EOL;
$result = print 'Printed';
var_dump($result);
```

The performance difference is irrelevant. Escape untrusted HTML output with `htmlspecialchars`.

```
$user = ['id' => 7, 'active' => true];
var_dump($user);
print_r($user);
$text = print_r($user, true);
echo get_debug_type($user);
```

Framework helpers such as `dump()` and `dd()` are useful in development; do not expose debug data in production.

```
const APP_NAME = 'Omar Notes';
define('APP_VERSION', '1.0.0');

class HttpStatus
{
    public const OK = 200;
}
```

`define()` executes at runtime and may appear conditionally; it defines a global constant, not a class constant. `const` is a declaration and can define class constants, but cannot be placed inside a function or conditional block.

Magic constants include `__LINE__`, `__FILE__`, `__DIR__`, `__FUNCTION__`, and `__CLASS__`. Predefined constants include `PHP_VERSION`, `PHP_OS_FAMILY`, and `PHP_EOL`. Use `__DIR__` for reliable include paths.

5. Conditions and loops

if, switch, match, for, while, do-while, and foreach with important differences.

```
if ($score >= 90) {
    $grade = 'A';
} elseif ($score >= 75) {
    $grade = 'B';
} else {
    $grade = 'C';
}

$label = $active ? 'active' : 'inactive';
$username = $_GET['username'] ?? 'guest';
$postId = $user->latestPost()?->id;
```

Alternative `if: ... endif;` syntax is useful in templates. `??` checks existence/non-null like `isset`; the nullsafe operator is `?->`.

```
switch ($role) {
    case 'admin':
        $permissions = ['all'];
        break;
    default:
        $permissions = ['read'];
}
```

`switch` historically uses loose comparison and falls through without `break`.

```
$message = match ($status) {
    200, 201 => 'success',
    404 => 'not found',
    default => 'unexpected',
};

$category = match (true) {
    $age < 13 => 'child',
    $age < 18 => 'teen',
    default => 'adult',
};
```

`match` returns a value, compares strictly, has no fall-through, and throws `UnhandledMatchError` when no arm/default matches.

```

for ($i = 0; $i < 5; $i++) {
    echo $i;
}

while ($attempts < 3) {
    $attempts++;
}

do {
    $input = readline();
} while ($input === '');

```

A `while` body may never run; `do-while` runs once before checking.

```

foreach ($users as ['id' => $id, 'name' => $name]) {
    echo "{$id}: {$name}";
}

foreach ($prices as &$price) {
    $price *= 1.14;
}
unset($price);

```

Always unset a reference variable after a by-reference `foreach`. Use `continue` to skip an iteration, `break` to leave a loop, and `break 2` for two nested levels.

6. Expressions and operators

Values, precedence, arithmetic, assignment, comparison, logic, arrays, execution, and pipes.

Every PHP expression has a value; assignment itself evaluates to the assigned value.

```
$b = $a = 5;
$result = 1 + 5 * 3;    // 16
$grouped = (1 + 5) * 3; // 18

$ok = true && false;    // false
$ok = true and false;   // assignment happens first: true
```

Prefer parentheses over memorising precedence, and normally use `&&` and `||`.

Arithmetic operators are `+` `-` `*` `/` `%` `**`; assignments include `+=` `-=` `*=` `/=` `%=` `**=` `.=` `??=`. Prefix increment changes then returns; postfix returns then changes.

Use `===` / `!==` when type matters. The spaceship `<=>` returns -1, 0, or 1 and is convenient in sorting.

```
usort($numbers, fn (int $a, int $b): int => $a <=> $b);

$union = ['a' => 1, 'shared' => 'left']
        + ['b' => 2, 'shared' => 'right'];
```

Array `+` is key union: left-side duplicate keys win. It is not the same as `array_merge()`. Array `===` additionally requires identical types and order. Use `instanceof` for object type checks. Object assignment normally points both variables at the same object; use `clone` for another instance.

Avoid the `@` error-control operator because it hides diagnostics. Backticks, `exec`, `system`, `shell_exec`, and `proc_open` execute operating-system commands.



Danger

Never interpolate user input into a shell command. Prefer a safe API; when process execution is unavoidable, allow-list the command and arguments and run with minimal privileges.

Pipe operator — PHP 8.5+

```
$slug = ' PHP 8.5 Released '  
|> trim(...)  
|> (fn (string $s) => str_replace(' ', '-', $s))  
|> strtolower(...);
```

`|>` passes the left value as the single argument to the callable on the right. This syntax does not run on PHP 8.4 or earlier.

7. Functions, callbacks, and includes

Parameters, references, closures, arrow functions, callables, include, and require.

Functions package reusable behaviour. PHP does not support declaring several global functions with the same name merely to overload signatures.

```
function calculateTotal(float $price, int $quantity = 1): float
{
    return $price * $quantity;
}

echo calculateTotal(quantity: 3, price: 19.5);

function sum(int ...$numbers): int
{
    return array_sum($numbers);
}
```

Parameters are names in the declaration; arguments are supplied values. Prefer returning data over printing inside business functions. Passing by reference with `&` mutates the caller's variable, so use it deliberately.

```
function greet(string $name): string { return "Hello {$name}"; }
$functionName = 'greet';
echo $functionName('Omar');

$tax = 0.14;
$long = function (float $price) use ($tax): float {
    return $price * (1 + $tax);
};
$short = fn (float $price): float => $price * (1 + $tax);

$clean = array_map(trim(...), [' ali ', 'mona ']);
```

Closures explicitly capture with `use` and may capture by reference. Arrow functions capture outer variables automatically by value and contain one expression. Verify uncertain callbacks with `is_callable()`. Avoid nested named function declarations; use closures.

Including files

```
$config = require __DIR__ . '/../config/app.php';
include __DIR__ . '/partials/header.php';
```

`require` failure raises an `Error` on modern PHP; `include` emits `E_WARNING` and normally continues. The `_once` variants prevent duplicate inclusion. An included file may `return` a value and inherits the scope at the include point. Use `__DIR__` for stable paths, `require_once` for unique definitions, and Composer autoload for project classes.

`goto label;` can jump to a label in the same file/scope, but cannot jump into a loop or switch. Small functions, `break`, and `continue` are usually clearer.

8. Files, streams, JSON, and CSV

Stream wrappers and contexts, open modes, permissions, JSON, serialization, and CSV.

A stream is a common interface for flowing data from or to files, memory, networks, URLs, and processes.

- A **wrapper** selects a protocol such as `file://`, `http://`, `ftp://`, `php://`, `data://`, or `zlib://`.
- A **context** supplies options such as timeouts and headers.
- A **filter** transforms data while it is read or written.

```
$context = stream_context_create([
    'http' => ['timeout' => 3, 'header' => "Accept: application/json\r\n"],
]);
$body = file_get_contents('https://example.com/api', false, $context);
```

Do not enable `allow_url_include`. For production HTTP, use a client that handles TLS, status codes, and retries.

```
$handle = fopen(__DIR__ . '/data.txt', 'rb');
if ($handle === false) {
    throw new RuntimeException('Cannot open file');
}
try {
    while (($line = fgets($handle)) !== false) {
        echo rtrim($line), PHP_EOL;
    }
} finally {
    fclose($handle);
}
```

Modes: `r/r+` require an existing file; `w/w+` truncate or create; `a/a+` append or create; `x/x+` exclusively create; `c/c+` create if absent without truncating. Add `b` for binary portability.

Use `fwrite`, `fflush`, `flock`, and `fclose` deliberately. Check return values. On Unix, permissions use read=4, write=2, execute=1: `0644` is owner read/write and others read; `0755` adds execution; `0600` is owner-only read/write. Do not default to `0777`.

```
$json = json_encode(
    ['name' => 'Omar', 'active' => true],
    JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR
);
$data = json_decode($json, true, flags: JSON_THROW_ON_ERROR);
```

`json_validate()` is available from PHP 8.3. Never `unserialize()` untrusted data because object injection may result; use JSON or tightly restrict allowed classes.

Use `fputcsv()`, `fgetcsv()`, and `str_getcsv()` rather than splitting on commas, because CSV quoting may contain commas.

9. Uploads, cookies, and sessions

Secure file uploads, cookie options, CSRF, session lifecycle, and session security.

File-upload forms use POST and `enctype="multipart/form-data"`. PHP exposes metadata in `$_FILES` and initially stores data in `upload_tmp_dir`.

```
$file = $_FILES['avatar'] ?? null;
if (!is_array($file) || $file['error'] !== UPLOAD_ERR_OK) {
    throw new RuntimeException('Upload failed');
}
if ($file['size'] > 2 * 1024 * 1024) {
    throw new RuntimeException('Maximum size is 2MB');
}

$mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);
$extensions = ['image/jpeg' => 'jpg', 'image/png' => 'png'];
if (!isset($extensions[$mime])) {
    throw new RuntimeException('Unsupported type');
}

$name = bin2hex(random_bytes(16)) . '.' . $extensions[$mime];
if (!move_uploaded_file($file['tmp_name'], __DIR__ . '/../storage/uploads/' . $name)) {
    throw new RuntimeException('Could not store upload');
}
```

Check `UPLOAD_ERR_*`, size, and server-detected MIME; generate the filename, store outside the public root where possible, and never execute an upload.

```
setcookie('theme', 'dark', [
    'expires' => time() + 2592000,
    'path' => '/',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Lax',
]);
```

`Secure` requires HTTPS; `HttpOnly` blocks JavaScript access; `SameSite` controls cross-site sending. `SameSite=None` requires `Secure`. These controls do not replace CSRF tokens for sensitive actions.

Session data normally lives server-side while the browser holds an identifier cookie.

```
session_start([
    'use_strict_mode' => true,
    'cookie_httponly' => true,
    'cookie_secure' => true,
    'cookie_samesite' => 'Lax',
]);

session_regenerate_id();
$_SESSION['user_id'] = $user->id;
```

Regenerate after authentication/privilege changes, before setting the elevated authentication state. Immediately deleting old session data with `session_regenerate_id(true)` can lose sessions or race with concurrent requests; security-sensitive systems should use a documented timestamp-based transition.

For logout, clear `$_SESSION`, expire the session cookie using its existing parameters, then call `session_destroy()`. The latter does not itself clear the in-memory array or client cookie. Close locks early with `session_write_close()`; use `read_and_close` for read-only access. Implement custom storage with `SessionHandlerInterface` or `session_set_save_handler()`.

Continue with [Session security](#) for fixation, hijacking, and lifecycle defenses.

10. Namespaces and autoloading

Names, imports, aliases, resolution rules, and mapping a namespace to directories with Composer PSR-4.

Why namespaces?

Namespaces prevent name collisions and give code a logical identity independent of a filename.

```
<?php
declare(strict_types=1);

namespace App\Billing;

final class InvoiceService {}
```

Place `namespace` near the top after `declare`. A namespace does not load its file; an autoloader does.

Imports and aliases

```
namespace App\Http;

use App\Billing\InvoiceService;
use App\Contracts\Logger as LoggerContract;
use function App\Support\normalize_email;
use const App\Config\MAX_RETRIES;

$service = new InvoiceService();
```

A leading `\` is fully qualified. Imported names resolve through `use`; other qualified names resolve relative to the current namespace. Imports are file-level compile-time declarations, not dynamic function-local operations.

Composer and PSR-4

```
{
  "autoload": {
    "psr-4": {
      "App\\\\": "src/"
    }
  }
}
```

`App\Billing\InvoiceService` normally maps to `src/Billing/InvoiceService.php`. Run `composer dump-autoload` after changing mappings.

Organization rules

- Prefer one main class per file.
- Match filename case because Linux filesystems are usually case-sensitive.
- Keep domain namespaces independent of a framework where practical.
- Put test namespaces in `autoload-dev`.
- Avoid side effects when a class file is loaded.



Caution

PHP itself does not require a namespace to match a directory. PSR-4 and Composer define that mapping.

References

- [PHP Namespaces](#)
- [Composer PSR-4](#)

11. Errors and exceptions

Throwable, Error, Exception, try/catch/finally, custom exceptions, boundaries, and safe reporting.

Error or Exception?

Both `Exception` and `Error` implement `Throwable`. Exceptions often represent operational or domain failure; `Error` includes engine, type, and programming failures that should not all be converted into success.

```
try {
    $receipt = $payments->charge($order);
} catch (PaymentDeclined $e) {
    // Expected domain failure
} catch (Throwable $e) {
    // Application boundary: log and return a generic response
} finally {
    $lock?->release();
}
```

`finally` always runs and is useful for releasing resources. Never leave a catch block empty.

Domain exceptions

```
final class InsufficientStock extends DomainException
{
    public function __construct(public readonly int $productId)
    {
        parent::__construct('Insufficient stock');
    }
}
```

The type carries machine-readable meaning that an outer layer can map to `409` or another contract. Do not branch on message text.

Reporting policy

Enable `E_ALL` and visible errors in development. In production keep `display_errors=Off`, `log_errors=On`, and return a generic message plus request ID. A browser stack trace can expose paths, secrets, and SQL.

Global boundary

```
set_exception_handler(function (Throwable $e): void {
    $requestId = bin2hex(random_bytes(8));
    error_log("[{$requestId}] {$e}");

    if (!headers_sent()) {
        http_response_code(500);
        header('Content-Type: application/json');
    }

    echo json_encode(['error' => 'Internal error', 'request_id' => $requestId]);
});
```

The global handler is a safety net, not a replacement for handling expected failures close to their context.

Rules

- Do not hide failures with `@`.
- Never send raw exception messages to clients.
- Preserve `previous` when wrapping.
- Redact passwords, tokens, and sensitive bodies.
- Retry only transient failures and only with idempotent behavior.

12. Arrays and transformation tools

Lists, maps, destructuring, unpacking, map, filter, reduce, sorting, and memory trade-offs.

One type, several shapes

PHP `array` is an ordered map used as either a list or a dictionary. Validate its shape at boundaries.

```
$ids = [10, 20, 30];
$user = ['id' => 7, 'name' => 'Omar'];

[$first, $second] = $ids;
['id' => $id, 'name' => $name] = $user;
```

`array_is_list()` identifies keys `0..n-1`. Removing a list item does not reindex automatically; use `array_values()` when required, especially before JSON encoding.

map, filter, and reduce

```
$paidTotals = array_map(
    static fn (array $order): int => $order['total_cents'],
    array_filter($orders, static fn (array $order): bool =>
        $order['status'] === 'paid'
    ),
);

$sum = array_reduce(
    $paidTotals,
    static fn (int $carry, int $total): int => $carry + $total,
    0,
);
```

These functions suit clear transformations. A loop is often clearer when producing several results, stopping early, or avoiding intermediate arrays.

Unpacking and merge behavior

```
$config = [...$defaults, ...$environment];

function sum(int ...$numbers): int {
    return array_sum($numbers);
}

$total = sum(...[2, 4, 6]);
```

For string keys, later unpacked values win. Array union with `+` instead preserves the left value for an existing key.

Sorting

`sort` reindexes values, `asort` preserves keys, `ksort` sorts keys, and `usort` accepts a comparator returning a negative number, zero, or a positive number.

Memory

PHP arrays are flexible but relatively heavy. Avoid loading millions of items into chained transformations. Use generators, pagination, or streaming processing.

13. Strings, Unicode, and regular expressions

Bytes, UTF-8, mbstring, normalization, contextual encoding, and safe regular expressions.

PHP strings are bytes

```
$text = 'مرحبًا';  
echo strlen($text);           // bytes  
echo mb_strlen($text, 'UTF-8'); // characters, approximately
```

Use UTF-8 consistently across HTTP, PHP, and the database. Prefer `mb_strlen`, `mb_substr`, and `mb_strtolower` for multilingual text.

Graphemes and normalization

A visible symbol may contain multiple code points. The `intl` extension provides grapheme functions and `Normalizer`:

```
$normalized = Normalizer::normalize($input, Normalizer::FORM_C);
```

Normalize for a clear purpose such as search or uniqueness. Preserve original text when required; case folding is language-sensitive.

Contextual output

- Use interpolation or `sprintf` for display, never SQL.
- Compare secret values with `hash_equals()` where timing-safe equality matters.
- Use `htmlspecialchars` for HTML text and `rawurlencode` for URL parameters.
- No universal sanitization function works for every context.

Regular expressions

```
$ok = preg_match('/\A[A-Z]{2}-\d{6}\z/D', $code) === 1;
```

Use explicit anchors, inspect `preg_last_error_msg()`, limit input size before complex patterns, avoid catastrophic backtracking, and choose a real parser for HTML or JSON.

Pattern validity is not business validity. Apply domain rules after matching the shape.

14. Date, time, and timezones

DateTimeImmutable, UTC, named zones, DST, strict parsing, intervals, and storage policy.

Instant or local time?

Distinguish an **instant** on the global timeline, a **local date/time**, a named timezone such as `Africa/Cairo`, and a duration. A fixed offset such as `+02:00` is not a timezone because regional rules can change.

DateTimeImmutable

```
$now = new DateTimeImmutable('now', new DateTimeZone('UTC'));
$cairo = $now->setTimezone(new DateTimeZone('Africa/Cairo'));

echo $now->format(DateTimeInterface::ATOM);
echo $cairo->format('Y-m-d H:i:s P');
```

Prefer `DateTimeImmutable` so operations return new values instead of mutating a shared object.

Strict parsing

```
$date = DateTimeImmutable::createFromFormat(
    '!Y-m-d',
    $input,
    new DateTimeZone('Africa/Cairo'),
);
$errors = DateTimeImmutable::getLastErrors();

if ($date === false || ($errors !== false &&
    ($errors['warning_count'] > 0 || $errors['error_count'] > 0))) {
    throw new InvalidArgumentException('Invalid date');
}
```

Do not use the flexible parser for input that promises a specific format; it may normalize impossible dates.

Arithmetic and DST

“Tomorrow at the same local time” can differ from “24 hours later” around DST. Define the domain meaning, especially for appointments and billing.

Storage policy

- Store instants in UTC with sufficient precision.
- Preserve the named timezone for future local schedules.
- Convert at presentation boundaries.
- Avoid implicit default timezones in business logic.
- Test month/year boundaries, leap days, and DST transitions.

Inject a clock in tests instead of reading “now” throughout domain code.

15. From HTTP request to router and response

Front controllers, JSON bodies, routing, middleware, and correct HTTP responses without a framework.

Front controller

Route dynamic requests to `public/index.php` and start from one entry point:

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
```

Keep the project root outside the document root so clients cannot fetch `vendor/`, `.env`, or source files.

Read a JSON body

```
$contentType = strtolower(trim(explode('; ', $_SERVER['CONTENT_TYPE'] ?? '')[0]));

if ($contentType !== 'application/json') {
    respond(['error' => 'Unsupported media type'], 415);
}

try {
    $payload = json_decode(
        file_get_contents('php://input'),
        true,
        64,
        JSON_THROW_ON_ERROR,
    );
} catch (JsonException) {
    respond(['error' => 'Invalid JSON'], 400);
}
```

Enforce body size in both web server and application. Parsing is not validation.

Minimal router

```
$handler = match ([$method, $path]) {  
  ['GET', '/health'] => static fn () => respond(['status' => 'ok']),  
  ['POST', '/api/orders'] => $createOrder,  
  default => null,  
};  
  
if ($handler === null) {  
  respond(['error' => 'Not found'], 404);  
}  
  
$handler();
```

A production router also handles parameters, method mismatch, and decoding.

Response and middleware

Send status and headers before the body. Keep output in one response abstraction.

```
request ID -> trusted proxy -> body limit -> routing  
-> authentication -> authorization -> validation  
-> handler -> error mapping -> response
```

Each middleware should have one clear responsibility. Error mapping, logging, and CORS may need to wrap the whole pipeline.

16. Modern PHP 8.4 and 8.5

Attributes, reflection, property hooks, asymmetric visibility, lazy objects, pipe, URI, and clone-with.

Declare the minimum version

Do not use a feature without declaring the runtime requirement and testing deployment:

```
{
  "require": {
    "php": "^8.4 || ^8.5"
  }
}
```

Run `composer check-platform-reqs` during deployment.

Attributes and reflection — PHP 8+

```
#[Attribute(Attribute::TARGET_METHOD)]
final readonly class RequiresRole
{
    public function __construct(public string $role) {}
}

#[RequiresRole('admin')]
function deleteUser(int $id): void {}
```

Attributes are structured metadata; they do not enforce authorization until application or framework code reads and applies them.

Property hooks and asymmetric visibility — PHP 8.4

```
final class User
{
    public private(set) string $email {
        set => filter_var($value, FILTER_VALIDATE_EMAIL)
            ? strtolower($value)
            : throw new InvalidArgumentException('Invalid email');
    }
}
```

Hooks customize get/set behavior while asymmetric visibility controls who may read and write. Prefer named methods for complex domain operations.

Lazy objects — PHP 8.4

Reflection supports lazy ghosts and proxies that initialize when state is observed. They primarily support DI containers and ORMs; understand identity, initialization triggers, cloning, and serialization before direct use.

Pipe operator — PHP 8.5

```
$slug = $title
    |> trim(...)
    |> mb_strtolower(...)
    |> (fn (string $v): string => str_replace(' ', '-', $v));
```

Each callable receives the previous result as one argument. Avoid hiding side effects in a pipeline.

URI and clone-with — PHP 8.5

The URI extension provides standards-based parsing instead of manual string operations. Clone-with simplifies immutable “with” operations:

```
$published = clone($draft, ['status' => Status::Published]);
```

Preserve invariants with hooks, constructors, and tests.

Other additions include `array_first()`, `array_last()`, `#[NoDiscard]`, constant attributes, and partitioned cookies. Read migration guides and run tests plus static analysis before upgrading.

References

- [PHP 8.4](#)

03

TRACK · 14 LESSONS

PHP runtime and production

Progressive explanations, practical examples, and corrections that take each idea into real code.

PHP Runtime & Production

Composer, HTTP clients, OPcache, memory, PHP-FPM, web servers, and environment configuration.

This section turns pages **78–86** into a complete guide to the layers around PHP application code. It updates the notes for the modern Zend Engine and separates dependency management, HTTP clients, web serving, and FastCGI process management.

Why a separate section?

These subjects are neither OOP nor PHP syntax. They form the **runtime and production** layer: reproducible dependencies, autoloading, efficient execution, outbound HTTP, process capacity, and safe configuration.

Learning path

1. [Composer and dependency management](#)
2. [cURL and HTTP clients](#)
3. [OPcache and preloading](#)
4. [PHP memory and garbage collection](#)
5. [PHP-FPM and process management](#)
6. [Apache, Nginx, and FastCGI](#)
7. [Environment variables and configuration](#)
8. [PHP CLI, php.ini, and extensions](#)
9. [Logging and observability](#)
10. [Testing and code quality](#)
11. [Data caching and Redis](#)
12. [Queues, workers, and scheduling](#)
13. [Deployment, CI/CD, and containers](#)
14. [Dependency and supply-chain security](#)

```
Client
-> Nginx / Apache (TLS, static files, routing)
-> FastCGI
-> PHP-FPM pool
-> Composer autoloader + OPcache
-> Application / Database / External API
<- HTTP response
```

Examples target PHP 8.x. Server snippets are educational baselines; adapt users, paths, and capacity to the actual operating system and workload.

Official references

- [Composer basic usage](#) and [autoloader optimization](#)
- [cURL and libcurl documentation](#)
- [OPcache](#) and [preloading](#)
- [PHP garbage collection](#)
- [PHP-FPM](#)
- [Nginx FastCGI](#) and [Apache MPMs](#)
- [getenv\(\)](#) and `$_ENV`

1. Composer and dependency management

composer.json, composer.lock, install, update, require-dev, PSR-4, and production installs.

Composer resolves project libraries and versions and generates an autoloader. It does not install PHP itself; it resolves constraints in `composer.json` and normally installs packages under `vendor/`.

The two core files

- `composer.json` declares requirements, allowed version ranges, autoloading, and scripts.
- `composer.lock` records the exact resolved versions and metadata for reproducible installs.

```
{
  "require": {
    "php": "^8.3",
    "guzzlehttp/guzzle": "^7.9"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Commit the lock file for applications. A published library's consumers still resolve that library inside their own application dependency graph.

install versus update

Command	Purpose
<code>composer install</code>	installs exact locked versions; use in CI and production
<code>composer update</code>	resolves allowed constraints again and rewrites the lock
<code>composer update vendor/package</code>	narrows the requested update, with related dependency changes possible

```
composer install
composer require monolog/monolog
composer require --dev phpunit/phpunit
composer update guzzlehttp/guzzle --with-all-dependencies
```

`require` normally edits the manifest, resolves the lock, and installs. `--dev` writes the package under `require-dev`.

PSR-4 autoloading

```
composer dump-autoload
```

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$service = new App\Billing\InvoiceService();
```

`App\Billing\InvoiceService` conventionally maps to `src/Billing/InvoiceService.php`. Linux filesystems are commonly case-sensitive, so incorrect casing that survives on Windows can fail after deployment.

A production install

```
composer validate --strict
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
composer audit
```

- `--no-dev` excludes development packages at deployment time.
- `--optimize-autoloader` builds a faster class map and is suited to production.
- Test `--classmap-authoritative` carefully because runtime-generated classes may be absent.
- Do not hide incompatible PHP or extension requirements with `--ignore-platform-reqs`.
- Composer scripts and plugins can execute code during installation; review untrusted projects before running them.

Recommended workflow

1. Resolve dependency changes during development.
2. Review both manifest and lock diffs.

3. Run tests and `composer audit` .
4. Commit both files; normally exclude `vendor/` .
5. Deploy with `composer install` , never an uncontrolled `update` .

2. cURL and HTTP clients

libcurl, PHP's extension, Guzzle, timeouts, TLS, JSON requests, and error handling.

The cURL project produces the `curl` command-line tool and the `libcurl` transfer library. PHP's `ext-curl` binds PHP to libcurl. It is an outbound client, not a web server or a socket server.

A complete request

```
<?php
declare(strict_types=1);

$handle = curl_init('https://api.example.com/v1/users/42');
curl_setopt_array($handle, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Accept: application/json',
        'User-Agent: OmarDevNotes/1.0',
    ],
    CURLOPT_CONNECTTIMEOUT => 3,
    CURLOPT_TIMEOUT => 10,
    CURLOPT_FOLLOWLOCATION => false,
]);

$body = curl_exec($handle);
if ($body === false) {
    $message = curl_error($handle);
    $code = curl_errno($handle);
    curl_close($handle);
    throw new RuntimeException("Network error {$code}: {$message}");
}

$status = curl_getinfo($handle, CURLINFO_RESPONSE_CODE);
curl_close($handle);

if ($status < 200 || $status >= 300) {
    throw new RuntimeException("Unexpected HTTP status: {$status}");
}

$data = json_decode($body, true, flags: JSON_THROW_ON_ERROR);
```

Keep three failure classes separate: transport failure, non-successful HTTP response, and invalid response content.

Sending JSON

```
$payload = json_encode(
    ['email' => 'user@example.com', 'active' => true],
    JSON_THROW_ON_ERROR
);

$handle = curl_init('https://api.example.com/v1/users');
curl_setopt_array($handle, [
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => $payload,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/json',
        'Accept: application/json',
        'Authorization: Bearer ' . $token,
    ],
    CURLOPT_CONNECTTIMEOUT_MS => 1000,
    CURLOPT_TIMEOUT_MS => 5000,
]);
```

Never log authorization headers, cookies, or sensitive bodies.

TLS, redirects, and SSRF

Never disable peer or hostname verification in production. Repair the CA trust configuration instead. Redirects can move a request to a different destination, so set a maximum and validate destinations.

Do not accept an arbitrary user-controlled URL in server-side fetching. Validate scheme and host and block internal, loopback, and metadata endpoints as required by the environment.

Guzzle and standards

Guzzle offers middleware, promises, and pooling. It is not permanently tied to cURL; its handler can use cURL or PHP streams.

```
use GuzzleHttp\Client;

$client = new Client([
    'base_uri' => 'https://api.example.com/',
    'connect_timeout' => 3,
    'timeout' => 10,
]);

$response = $client->get('v1/users/42', [
    'headers' => ['Accept' => 'application/json'],
]);
```

PSR-7 models HTTP messages and PSR-18 defines a client interface. Depending on an interface improves replacement and testing.

Production rules

- Set both connection and total timeouts so a slow dependency does not occupy an FPM worker indefinitely.
- Retry only transient failures, with exponential backoff and jitter.
- Retry non-idempotent writes only with a server-supported idempotency key.
- Respect `Retry-After` for 429/503.
- Limit response sizes and stream large bodies.
- Use `curl_multi_*` or a client pool for bounded concurrency, never unbounded fan-out.

3. OPcache and preloading

PHP bytecode caching, OPcache configuration and monitoring, data-cache differences, and preloading limits.

PHP normally parses source and compiles it to opcodes before execution. OPcache keeps compiled bytecode in shared memory so later requests can skip much of that work.

```
PHP source -> parse -> compile to opcodes -> execute
|
+--> OPcache shared memory
```

OPcache does not cache final HTML, SQL results, or API responses. Those belong to HTTP or application caches such as Redis.

Production baseline

```
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
```

Treat the numbers as a starting point, not universal truth.

- Timestamp validation detects changed scripts according to `revalidate_freq`.
- With validation disabled, restart or gracefully reload FPM on every release.
- CLI OPcache is configured separately through `opcache.enable_cli` and may not help short commands.

Observe before tuning

```
$status = opcache_get_status(false);
if ($status !== false) {
    printf(
        "hit-rate=%.2f%% used=%d free=%d\n",
        $status['opcache_statistics']['opcache_hit_rate'],
        $status['memory_usage']['used_memory'],
        $status['memory_usage']['free_memory'],
    );
}
```

Keep status data behind internal authorization because it can expose file paths. Watch `cache_full`, OOM/hash restarts, hit rate after warm-up, and the number of cached keys.

Preloading

PHP 7.4+ can execute a preload file once at server startup and keep loaded functions, classes, interfaces, and traits available:

```
opcache.preload=/var/www/app/config/preload.php
opcache.preload_user=www-data
```

```
foreach ([
    __DIR__ . '/../src/Domain/Money.php',
    __DIR__ . '/../src/Domain/Order.php',
] as $file) {
    opcache_compile_file($file);
}
```

Preloading increases baseline memory, needs a process restart for code changes, and is useful only for persistent processes. Measure it rather than preloading everything. The PHP manual also notes that preloading is not supported on Windows.

Deployment sequence

Install exact Composer dependencies, optimize the autoloader, test a separate release directory, switch releases atomically, then gracefully reload FPM. Composer's class map speeds file discovery; OPcache caches compiled bytecode. They optimize different stages.

4. PHP memory and garbage collection

zvals, reference counting, copy-on-write, references, objects, cycles, and memory measurement.

The Zend Engine represents a PHP value with an internal `zval`. Compound data such as strings, arrays, and objects can reference additional refcounted structures.



Version-aware correction

The simple `type + value + is_ref + refcount` description reflects PHP 5-era internals. PHP 7/8 reorganized zvals and references. Learn observable semantics instead of depending on a fixed internal memory layout.

Reference counting and copy-on-write

```
$original = range(1, 100_000);  
$copy = $original;           // shared for now  
$copy[] = 100_001;          // modified data must now separate
```

PHP generally delays copying large arrays and strings until one copy changes. This saves work, although the eventual write can produce a temporary memory spike.

Do not interpret a debug refcount as a perfect number of source-level variables. Temporaries, interning, and engine details affect it.

Objects and references

```
$a = new stdClass();  
$a->count = 1;  
$b = $a;  
$b->count++;  
  
echo $a->count; // 2
```

Both variables hold a handle to the same object. Use `clone` for a separate object and implement `__clone()` where nested mutable state needs copying.

```
$value = 10;
$alias =& $value;
$alias = 20;
echo $value; // 20
```

A PHP reference is an alias, not a general C pointer. Do not use `&` as a performance trick.

Cyclic garbage

Refcounting alone cannot free an unreachable cycle:

```
final class Node
{
    public ?Node $next = null;
}

$a = new Node();
$b = new Node();
$a->next = $b;
$b->next = $a;
unset($a, $b);
```

The cyclic garbage collector records candidates and periodically detects unreachable cycles. `gc_collect_cycles()` can force a pass, but it should be a measured tool for long-running workers, not a call after every operation.

Requests, workers, and measurement

Request-owned memory is normally released at request shutdown, while an FPM worker remains alive. Extensions, caches, or allocator fragmentation can keep its resident size high. `pm.max_requests` recycles workers, but does not replace fixing leaks in long-running code.

```
$before = memory_get_usage(true);
$rows = loadReportRows();

printf(
    "current=%s peak=%s delta=%s\n",
    number_format(memory_get_usage(true)),
    number_format(memory_get_peak_usage(true)),
    number_format(memory_get_usage(true) - $before),
);
```

The `true` form reports memory obtained from the system and may exceed currently used payload memory.

Practical reductions

- Stream or paginate instead of loading entire datasets.

- Avoid `fetchAll()` for huge results.
- Watch closures that capture large object graphs.
- Release proven large references inside long loops.
- Keep OPcache/shared caches conceptually separate from the request heap.
- Profile before merely increasing `memory_limit`.

5. PHP-FPM and process management

FastCGI, master and workers, pools, static/dynamic/ondemand modes, capacity, and monitoring.

PHP-FPM is PHP's primary **FastCGI Process Manager**. A trusted web server sends it FastCGI requests; FPM manages PHP processes and returns responses.

```
Nginx/Apache -> FastCGI socket -> FPM master -> worker -> PHP response
```

Never expose FPM to an untrusted network. Bind a Unix socket or an internal interface and configure socket permissions and `listen.allowed_clients` where applicable.

Master, workers, and pools

- The master reads pool configuration and controls lifecycle.
- A worker normally executes one PHP request at a time.
- A pool groups workers under a user/group, listener, limits, and PHP settings.

Separate applications into pools and OS identities where isolation matters.

Process-manager modes

```
; Fixed workers, predictable memory
pm = static
pm.max_children = 20

; Or elastic capacity with warm spare workers
pm = dynamic
pm.max_children = 40
pm.start_servers = 8
pm.min_spare_servers = 4
pm.max_spare_servers = 12

; Or spawn only when requested
pm = ondemand
pm.max_children = 20
pm.process_idle_timeout = 10s
```

`static` avoids spawn latency but reserves memory. `dynamic` is a common balance. `ondemand` helps low-traffic pools at the cost of cold starts.

Capacity from measurements

Estimate:

```
RAM budget for FPM / realistic high-percentile worker RSS
```

Leave headroom for the operating system, web server, OPcache, and other services. Do not divide RAM by `memory_limit`; that limit is not the worker's typical resident set.

Arrival rate also matters. At 100 requests/s and a 200 ms average service time, average concurrency is roughly 20 workers. Peaks and tail latency need additional headroom, while excessive workers can overload the database.

Operational settings

```
pm.max_requests = 500
request_terminate_timeout = 30s
request_slowlog_timeout = 3s
slowlog = /var/log/php-fpm/app-slow.log
catch_workers_output = yes
ping.path = /fpm-ping
pm.status_path = /fpm-status
```

Monitor active/idle processes, listen queue, slow traces, and `max children reached`. Keep status endpoints private.

Use graceful reloads during deployment. `fastcgi_finish_request()` can flush the response before brief follow-up work, but it still occupies a worker; durable or heavy tasks belong in a queue worker.

6. Apache, Nginx, and FastCGI

Web-server responsibilities, Apache MPMs, Nginx's event model, and PHP-FPM over Unix or TCP sockets.

The web server terminates HTTP/TLS, serves static files, applies request limits and routing, and forwards PHP execution. PHP-FPM executes PHP.

Correcting the Apache/Nginx comparison

Apache is not universally “one process per connection.” Its active MPM determines the model:

- `prefork`: multiple non-threaded processes.
- `worker`: processes containing threads.
- `event`: threaded, with improved keep-alive handling.

Nginx workers drive event loops and can manage many connections with few processes. That does not make application execution asynchronous: an active PHP request still occupies an FPM worker.

Apache with FPM

```
<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/app/public

    <Directory /var/www/app/public>
        AllowOverride None
        Require all granted
        FallbackResource /index.php
    </Directory>

    <FilesMatch \.php$>
        SetHandler "proxy:unix:/run/php/app.sock|fcgi://localhost/"
    </FilesMatch>
</VirtualHost>
```

`.htaccess` is useful when hosting does not grant main configuration access. On a server you control, central VirtualHost configuration with `AllowOverride None` avoids per-directory checks and scattered policy.

Nginx with FPM

```
server {
    listen 443 ssl;
    server_name example.com;
    root /var/www/app/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        try_files $uri =404;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param HTTP_PROXY "";
        fastcgi_pass unix:/run/php/app.sock;
        fastcgi_read_timeout 30s;
    }

    location ~ /\. {
        deny all;
    }
}
```

Where possible, forwarding only the front controller `/index.php` reduces the execution surface. Keep `.env`, `vendor/`, and executable uploads outside the public document root.

Unix versus TCP

```
fastcgi_pass unix:/run/php/app.sock;
# or
fastcgi_pass 127.0.0.1:9000;
```

Unix sockets suit services on one host and need correct filesystem ownership. TCP is common across containers or hosts and must remain on a private, firewalled network. Correctness and observability matter more than simplistic performance claims.

`SCRIPT_FILENAME` identifies the script to PHP. A wrong value can yield “Primary script unknown” or expose unintended paths.

Trust forwarded IP/protocol headers only from explicitly trusted proxies. Align web-server upstream timeouts with FPM request limits and the endpoint’s service objective.

7. Environment variables and configuration

getenv, \$_ENV, putenv, .env files, typed validation, and secret handling with PHP-FPM.

Environment variables are values supplied to a process by the operating system, container, or service manager.

```
OS / container / service manager
  -> PHP-FPM master
  -> worker process
  -> getenv() / $_SERVER / $_ENV
```

PHP access paths

```
$dsn = getenv('DATABASE_DSN');
if ($dsn === false || $dsn === '') {
    throw new RuntimeException('DATABASE_DSN is required');
}
```

Use `=== false`; the string `"0"` is a valid value even though it is falsy.

- `getenv()` reads an environment variable.
- `$_ENV` contains values imported by PHP and may be empty when `variables_order` excludes `E`.
- `$_SERVER` can contain environment/CGI values depending on the SAPI.
- `putenv()` changes the current process environment for the request and does not automatically synchronize `$_ENV`.

PHP does not load `.env` files by itself. A framework, a package such as `vlucas/phpdotenv`, or custom bootstrap code does so. Production can inject values directly with no file.

Development files

```
require dirname(__DIR__) . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(dirname(__DIR__));
$dotenv->safeLoad();
$dotenv->required(['APP_ENV', 'DATABASE_DSN']);
```

Commit a non-secret `.env.example`, ignore the real `.env`, and keep it outside the document root. If a secret leaks, revoke or rotate it; deleting one commit does not invalidate the credential.

Parse once into typed configuration

```
final readonly class AppConfig
{
    public function __construct(
        public string $environment,
        public bool $debug,
        public string $databaseDsn,
        public int $httpTimeoutMs,
    ) {}

    public static function fromEnvironment(): self
    {
        $dsn = getenv('DATABASE_DSN');
        if ($dsn === false || $dsn === '') {
            throw new RuntimeException('DATABASE_DSN is required');
        }

        $debug = filter_var(
            getenv('APP_DEBUG') ?: 'false',
            FILTER_VALIDATE_BOOL,
            FILTER_NULL_ON_FAILURE,
        );
        if ($debug === null) {
            throw new RuntimeException('APP_DEBUG must be true or false');
        }

        $timeout = filter_var(
            getenv('HTTP_TIMEOUT_MS') ?: '5000',
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 100, 'max_range' => 60_000]],
        );
        if ($timeout === false) {
            throw new RuntimeException('HTTP_TIMEOUT_MS is invalid');
        }

        return new self(
            getenv('APP_ENV') ?: 'production',
            $debug,
            $dsn,
            $timeout,
        );
    }
}
```

Validate once at startup and inject configuration instead of calling `getenv()` throughout domain classes.

FPM and secrets

FPM commonly clears worker environments. Pass an explicit allowlist through a pool or service manager:

```
clear_env = yes
env[APP_ENV] = $APP_ENV
env[DATABASE_DSN] = $DATABASE_DSN
```

Environment variables are configuration transport, not encryption. They may be visible to diagnostics or child processes. Use a secret manager for rotation, auditing, and policy; never expose `phpinfo()`, full environment dumps, or server secrets to a browser.

8. PHP CLI, php.ini, and extensions

SAPIs, configuration discovery, extensions, and separate development and production policy.

The SAPI changes the environment

PHP may run through CLI, FPM/FastCGI, or an Apache module. The same code can load different `php.ini` files, extensions, users, and environment values.

```
php -v
php --ini
php -m
php -i
php -r "echo PHP_SAPI, PHP_EOL;"
```

When code works in a terminal but not through HTTP, compare the executable, SAPI, loaded configuration, process user, and environment.

Configuration and extensions

`php --ini` shows the main file and scanned `conf.d` directory. Later files may override earlier values.

```
printf(
    "sapi=%s ini=%s memory=%s\n",
    PHP_SAPI,
    php_ini_loaded_file() ?: 'none',
    ini_get('memory_limit'),
);
```

Some directives cannot change at runtime. Declare extension requirements in Composer and run:

```
php --ri opcache
php --ri pdo_mysql
composer check-platform-reqs
```

Environment policy

Development can show detailed errors and load Xdebug. Production should hide errors, log them safely, enable OPcache, set resource limits, and remove unnecessary extensions. The distributed development/production ini files are starting points, not complete application policy.

CLI programs

Use meaningful exit codes, timeouts, logging, signal handling for workers, and a lock when duplicate execution is unsafe. `set_time_limit(0)` does not stop the operating system or orchestrator from terminating a process.

9. Logging and observability

PSR-3, structured events, request IDs, metrics, traces, redaction, and actionable alerts.

Three signal types

- **Logs** are detailed searchable events.
- **Metrics** aggregate numbers over time.
- **Traces** follow work through services, databases, and queues.

Observability needs schemas, retention, access policy, and alerts—not random `error_log` calls.

PSR-3 structured logs

```
use Psr\Log\LoggerInterface;

final class Checkout
{
    public function __construct(private LoggerInterface $logger) {}

    public function run(Order $order, string $requestId): void
    {
        $this->logger->info('checkout.started', [
            'request_id' => $requestId,
            'order_id' => $order->id(),
        ]);
    }
}
```

Pass machine-readable context instead of concatenating text. Monolog is a common PSR-3 implementation.

Correlation and redaction

Create a request ID at the trusted edge and carry it into logs, outgoing HTTP, and queue metadata. Never log passwords, session IDs, tokens, authorization or cookie fields, encryption keys, or unreviewed personal bodies. Use allow-lists or tested redaction.

Metrics and alerts

Track request rate, error rate, latency distributions, and saturation. Useful PHP signals include FPM queue depth, max-children events, database pool pressure, API latency, queue lag, and cache hit ratio.

Alert on actionable user impact such as sustained `5xx` rate or p95 latency. Attach deployment version to signals to identify regressions.

Reference

- [PSR-3 Logger Interface](#)

10. Testing and code quality

PHPUnit, unit/integration/feature tests, data providers, doubles, coverage, static analysis, and style.

A practical test pyramid

- **Unit:** small logic without network or database.
- **Integration:** real collaboration with a database, filesystem, or adapter.
- **Feature/HTTP:** a complete request through the application.
- **End-to-end:** user-visible system behavior; fewer and more expensive.

Do not replace every collaborator with a mock. Test contracts at boundaries and behavior that matters to users.

PHPUnit

```
composer require --dev phpunit/phpunit
vendor/bin/phpunit
```

```
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class DiscountTest extends TestCase
{
    #[DataProvider('cases')]
    public function testDiscount(int $price, int $percent, int $expected): void
    {
        self::assertSame($expected, discount($price, $percent));
    }

    public static function cases(): iterable
    {
        yield 'none' => [1000, 0, 1000];
        yield 'ten percent' => [1000, 10, 900];
    }
}
```

Test boundaries, exceptions, and side effects. Make test names describe behavior.

Doubles and coverage

A stub returns prepared data, a fake is a lightweight working implementation, and a mock verifies interaction. Prefer fakes or stubs where possible; excessive mocks couple tests to implementation.

Coverage finds unexecuted code but cannot prove useful assertions. Focus on branches and critical behavior rather than an isolated 100% target.

Automated quality

Run PHPUnit, PHPStan/Psalm, and PHPCS/PHP-CS-Fixer locally and in CI with the same configuration. Control clocks and randomness, isolate database tests, avoid ordering dependencies, and never copy real production personal data into fixtures.

Reference

- [PHPUnit Manual](#)

11. Data caching and Redis

Cache-aside, TTL, invalidation, stampede protection, distributed locks, and failure policy.

Know the cache layer

OPcache stores PHP bytecode, HTTP caches store responses, application caches store computed/query data, and local in-process caches are fast but not shared. Redis is a common shared data cache, not automatically the source of truth.

Cache-aside

```
$key = "product:{$id}:v1";
$json = $redis->get($key);

if ($json === false) {
    $product = $repository->find($id);
    $json = json_encode($product, JSON_THROW_ON_ERROR);
    $redis->setex($key, 300, $json);
}

return json_decode($json, true, flags: JSON_THROW_ON_ERROR);
```

After a write, update the database first and then invalidate or refresh the cache according to an explicit consistency policy.

Keys, TTL, and stampedes

Version key names, add TTL jitter, avoid `KEYS *` in request paths, define serialization, and never treat an internal cache as a safe secret store. Popular expired keys can trigger a rebuild stampede; consider a short lock, stale-while-revalidate, early refresh, or single-flight.

A lock needs a unique ownership token and safe release. Distributed locking is not a universal consistency solution.

Failure policy

Decide whether the cache is an optional optimization or a required dependency such as a session store. Use short timeouts and monitor hit ratio, evictions, memory, and latency. Prevent cache failure from instantly flooding the database.

For each key document its writer, source of truth, invalidation event, maximum stale time, and failure behavior.

12. Queues, workers, and scheduling

Jobs, at-least-once delivery, idempotency, retry, backoff, DLQs, graceful shutdown, and cron.

Why a queue?

Move slow or retryable work—mail, images, reports, API synchronization—outside the HTTP request. Respond after durably recording the intent, not after spawning an unmanaged background process. Version small payloads and send identifiers instead of serialized object graphs.

Delivery and idempotency

Many systems provide **at-least-once** delivery, so a message can repeat. Deduplicate by job or business idempotency key and make result storage atomic where possible. A transactional outbox ties a database change to publishing intent.

Retry policy

Retry transient failures with capped exponential backoff and jitter. Do not retry permanent validation or credential errors unchanged. After a limit, move the message to a dead-letter queue with diagnosis and alerting.

Worker lifecycle

- Set per-job and I/O timeouts.
- Enforce memory limits and recycle long-lived processes.
- On SIGTERM, stop accepting work and finish current work within a grace period.
- Acknowledge after success.
- Align visibility timeout with runtime and renew it when supported.

Monitor queue depth, oldest-message age, processing latency, retries, and DLQ rate.

Scheduler

Make scheduled work idempotent, prevent unsafe overlap with an expiring lock, and define the timezone. A scheduler can enqueue work instead of performing everything in one process.

13. Deployment, CI/CD, and containers

Reproducible artifacts, pipelines, health checks, migrations, zero-downtime rollout, and rollback.

Reproducible artifact

Build once and promote the same artifact instead of resolving dependencies differently on every server:

```
composer install --no-dev --prefer-dist --no-interaction \  
  --optimize-autoloader --classmap-authoritative  
composer check-platform-reqs
```

Record commit and build IDs. Inject secrets at runtime, never into the image.

Pipeline

```
lint -> unit tests -> static analysis -> integration tests  
-> dependency audit -> build artifact/image -> scan  
-> deploy staging -> smoke test -> production -> verify
```

Fail closed when checks fail. Manual approval is not a substitute for repeatable verification.

Containers and health

Use a small pinned image, explicit main process, non-root user, runtime config, stdout/stderr logs, resource limits, and graceful shutdown. Separate web and worker lifecycles.

Liveness asks whether a process should restart, readiness whether it can receive traffic, and startup whether warm-up is complete. Do not tie liveness to every external dependency and cause a restart storm.

Safe rollout and schema changes

Use rolling or blue-green deployment. Follow Expand/Contract: add the new shape, deploy compatible code, backfill, switch reads, then remove the old shape later. Gracefully reload FPM and version queue payloads.

Rollback may not undo a destructive migration. Prepare roll-forward, backups, feature flags, and automatic success metrics, and rehearse the procedure.

14. Dependency and supply-chain security

Composer audit, policies, platform checks, lock files, plugins, scripts, and CI verification.

The lock file is a build contract

Commit `composer.lock` for applications and use `composer install` in CI and production. `update` selects new versions and belongs in a reviewed, tested change—not deployment.

Audit and platform checks

```
composer validate --strict
composer audit --locked
composer check-platform-reqs
composer outdated --direct
```

Do not ignore an advisory without a documented reason, expiry, and reachability review.

Plugins and scripts

Composer plugins and scripts execute code with the Composer process permissions. Review and allow-list them:

```
{
  "config": {
    "allow-plugins": {
      "trusted/package": true,
      "**": false
    }
  }
}
```

Do not run Composer as root on untrusted packages. `--no-plugins --no-scripts` reduces execution but may also disable required build steps.

Policy and incident response

Use deliberate SemVer constraints, test supported ranges for libraries, review transitive dependencies, update regularly, and remove unused packages. Protect CI tokens and logs and record which artifact was built from which commit.

For a vulnerability, assess reachability, patch or mitigate, run the suite, deploy, and monitor. If a package or script exposed a secret, rotate it; uninstalling the package does not revoke the secret.

References

04

TRACK · 8 LESSONS

Object-oriented programming

Progressive explanations, practical examples, and corrections
that take each idea into real code.

Object-Oriented Programming

A practical PHP track for classes, encapsulation, inheritance, abstraction, interfaces, traits, and magic methods.

This section turns pages **71–77** into a practical, version-aware PHP OOP track.

1. [Classes, objects, and encapsulation](#)
2. [Inheritance and polymorphism](#)
3. [Traits](#)
4. [Abstraction and interfaces](#)
5. [Constants, static, and late static binding](#)
6. [Constructors, readonly, and dependency injection](#)
7. [Magic methods and cloning](#)
8. [OOP review checklist](#)

OOP is not turning everything into a class. Use it to protect invariants, combine state with behaviour, and reduce coupling.

1. Classes, objects, and encapsulation

PHP classes, objects, properties, methods, visibility, and invariant protection.

A class defines state and behaviour; each object is an instance with its own state.

```
final class Product
{
    public function __construct(
        public readonly int $id,
        public string $name,
        private int $priceCents,
    ) {
        if ($priceCents < 0) {
            throw new InvalidArgumentException('Negative price');
        }
    }

    public function changePrice(int $value): void
    {
        if ($value < 0) {
            throw new InvalidArgumentException('Negative price');
        }
        $this->priceCents = $value;
    }
}
```

Encapsulation means hiding representation, exposing meaningful operations, and preventing invalid state. A public setter for every property is not good encapsulation; prefer `withdraw()` over `setBalance()`.

`public` is visible everywhere, `protected` in the class and children, and `private` only in the declaring class. Choose the narrowest visibility; protected state couples children to parent internals.

`$this` refers to the current object inside instance methods and is unavailable in static methods. Declare typed properties explicitly; dynamic undeclared properties are deprecated for most modern PHP classes.

`==` compares object properties while `===` requires the same instance. Value objects should provide domain equality deliberately.

2. Inheritance and polymorphism

is-a relationships, overriding, final members, substitutability, and composition.

Inheritance models a real is-a relationship. PHP supports one parent class, while classes may implement multiple interfaces and use multiple traits.

```
abstract class Notification
{
    abstract public function send(string $message): void;
}

final class EmailNotification extends Notification
{
    #[\Override]
    public function send(string $message): void
    {
        // send email
    }
}

function notify(Notification $channel, string $message): void
{
    $channel->send($message);
}
```

Polymorphism lets the caller use a shared contract while the runtime object supplies behaviour. Overrides must keep a compatible signature and meaning.

A final class cannot be extended; a final method or constant cannot be overridden/redefined. Private parent members are not directly accessible from children; expose meaningful methods rather than making all state protected.

Prefer composition for has-a relationships and swappable collaborators. Use inheritance only when substitutability is real. Avoid giant base classes, unsupported operations in children, and inheritance used merely to reuse a few lines.

3. Traits

Horizontal reuse, precedence, conflict resolution, aliases, visibility, and design limits.

Traits provide horizontal reuse in PHP's single-inheritance model and cannot be instantiated.

```
trait HasTimestamps
{
    private ?DateTimeImmutable $updatedAt = null;

    public function touch(): void
    {
        $this->updatedAt = new DateTimeImmutable();
    }
}
```

A class method overrides a trait method; a trait method overrides an inherited method. Two traits providing the same method require explicit resolution:

```
use JsonLogger, TextLogger {
    JsonLogger::log insteadof TextLogger;
    TextLogger::log as logText;
    JsonLogger::log as protected logJson;
}
```

`insteadof` chooses the winner. `as` adds an alias or changes visibility but does not resolve the conflict alone. Traits may declare abstract requirements.

Keep traits small and cohesive; large traits hide dependencies and state. Direct static access on the trait name is deprecated. PHP 8.3 can mark imported methods final via `as final`; PHP 8.5 changed binding order with parent properties/constants, so test upgrades. Use an interface when callers need a type contract.

4. Abstraction and interfaces

Abstract classes, methods, interfaces, capabilities, and choosing between them.

Abstraction exposes what callers need while hiding implementation details.

```
abstract class PaymentMethod
{
    final public function charge(int $amount): Receipt
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException();
        }
        return $this->performCharge($amount);
    }

    abstract protected function performCharge(int $amount): Receipt;
}
```

An abstract class cannot be instantiated and may hold state, a constructor, concrete methods, and abstract requirements. PHP 8.4 adds abstract properties with get/set requirements, but method contracts remain clearer for multi-version libraries.

```
interface PaymentGateway
{
    public function charge(string $customerId, int $amount): Receipt;
}
```

An interface defines capability without one inheritance tree; a class may implement several interfaces. Split large interfaces so consumers depend only on required operations. Use interfaces at boundaries and for injected collaborators; use abstract classes when related types genuinely share stable state/implementation.

5. Constants, static, and late static binding

Class constants, static members, self vs static, and shared-state risks.

Class constants belong to a class, may have visibility, and may be final. Typed class constants require PHP 8.3+.

Static properties are shared class/process state. Static methods have no `$this`, and calling a non-static method statically throws an Error. Mutable static state often hides global dependencies and complicates tests and long-running workers.

`self::` resolves to the defining class; `static::` uses late static binding and refers to the called class.

```
class Document
{
    protected const TYPE = 'document';
    public static function early(): string { return self::TYPE; }
    public static function late(): string { return static::TYPE; }
}
class Invoice extends Document { protected const TYPE = 'invoice'; }
```

Use `static::` for intentionally extensible behaviour and `self::` for the defining class. Static suits constants and pure named constructors, not hidden service dependencies.

6. Constructors, readonly, and dependency injection

Valid construction, promotion, readonly semantics, and explicit collaborators.

A successful constructor should leave a valid object. Property promotion declares and assigns properties concisely. A child-defined constructor does not automatically invoke the parent constructor.

```
final readonly class EmailAddress
{
    public function __construct(public string $value)
    {
        if (filter_var($value, FILTER_VALIDATE_EMAIL) === false) {
            throw new InvalidArgumentException();
        }
    }
}
```

Readonly classes (PHP 8.2+) make declared properties readonly and prohibit dynamic properties. Readonly is not deep immutability: nested mutable objects can still change. PHP 8.4 changed implicit set visibility to `protected(set)`.

Dependency injection exposes collaborators:

```
final class OrderService
{
    public function __construct(
        private PaymentGateway $payments,
        private OrderRepository $orders,
        private Clock $clock,
    ) {}
}
```

DI does not require a container. Use constructor injection for lifetime requirements, method parameters for operation-specific inputs, and avoid service locators that hide dependencies.

7. Magic methods and cloning

`__get`, `__set`, `__call`, `__toString`, `__invoke`, `__clone`, `__debugInfo`, and serialization.

PHP reserves names beginning with `__` for magic behaviour. Except for `__construct`, `__destruct`, and `__clone`, magic methods must be public.

`__get`/`__set`/`__isset`/`__unset` intercept inaccessible properties. They support dynamic bags and proxies but weaken static analysis and hide typos. `__call`/`__callStatic` intercept inaccessible methods and should fail clearly for unknown names.

```
final class Slugify
{
    public function __invoke(string $value): string
    {
        return strtolower(trim(str_replace(' ', '-', $value)));
    }
}
```

Invokable objects make small injectable strategies.

`clone` is shallow by default; nested object references stay shared. Implement `__clone` for selected mutable children. Do not clone ORM entities without understanding identity and Unit of Work.

`__debugInfo` can redact secrets from `var_dump`, but avoid logging sensitive objects. Prefer `__serialize`/`__unserialize` over legacy hooks and never unserialize untrusted data. Destructors are unsuitable for critical business commits; use explicit methods and `try/finally`.

8. OOP review checklist

A practical review checklist and exercise covering encapsulation, interfaces, composition, DI, and tests.

- Does each class have a clear domain responsibility?
- Does its constructor produce a valid object?
- Are members given the narrowest useful visibility?
- Is inheritance a true is-a relationship, or is composition clearer?
- Is each interface small and required by its consumer?
- Do traits stay cohesive and avoid hidden dependencies?
- Will static state harm tests or long-running workers?
- Does readonly provide the depth of immutability required?
- Do magic methods improve the API or hide mistakes?
- Are collaborators explicit and replaceable in tests?

Combined exercise

Design a checkout where Order protects its state, PaymentGateway is replaceable with a fake, Money and Receipt are

05

TRACK · 19 LESSONS

Security, identity, and authorization

Progressive explanations, practical examples, and corrections that take each idea into real code.

Security, Authentication & Authorization

Practical PHP security covering sessions, passwords, authorization, JWT, OAuth 2.0, OpenID Connect, and SSO.

This section expands pages **60–66** and **87–103** into a practical, modern security track.

1. [Session security](#)
2. [Input validation](#)
3. [CSRF protection](#)
4. [XSS protection](#)
5. [Password hashing](#)
6. [Encryption and HTTP authentication](#)
7. [Authentication and authorization](#)
8. [JWT and secure validation](#)
9. [Federated identity, SSO, SAML, and OIDC](#)
10. [OAuth 2.0 concepts and tokens](#)
11. [Modern OAuth flows](#)
12. [OAuth and OIDC security](#)
13. [Account lifecycle](#)
14. [MFA, TOTP, and passkeys](#)
15. [API keys and machine identity](#)
16. [ABAC, ReBAC, and tenant isolation](#)
17. [Rate limiting and abuse resistance](#)
18. [Security logging and audit trails](#)
19. [Secrets and key management](#)

Treat browser, API, cookie, header, uploaded, and legacy database data as untrusted. Validate at the boundary and encode for the exact output context.

Standards

- [JWT - RFC 7519](#) and [JWT BCP - RFC 8725](#)
- [OAuth 2.0 - RFC 6749](#) and [Security BCP - RFC 9700](#)
- [PKCE - RFC 7636](#) and [Device Authorization - RFC 8628](#)
- [OpenID Connect Core](#)
- [SAML 2.0 Technical Overview](#)

1. Session security

Secure lifecycle, fixation and hijacking defenses, ID rotation, expiry, and cookies.

The server stores session state while the browser normally holds a random identifier cookie. That ID is a credential: stealing it can impersonate the user.

Session fixation makes a victim use an attacker-known ID; hijacking steals a valid ID through XSS, insecure transport, logs, or a compromised device.

```
session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_secure' => true,
    'cookie_httponly' => true,
    'cookie_samesite' => 'Lax',
]);
```

Use HTTPS, never put IDs in URLs, and regenerate after authentication or privilege changes before writing elevated state:

```
session_regenerate_id();
$_SESSION['user_id'] = $user['id'];
$_SESSION['auth_time'] = time();
```

Immediate deletion through `session_regenerate_id(true)` can race concurrent requests or unstable networks. Sensitive systems use a short timestamp-based transition and retire obsolete IDs deliberately.

Implement both idle and absolute expiry; do not confuse garbage collection with access policy. Treat IP/User-Agent changes as risk signals rather than strict identity because mobile networks, NAT, and VPNs change. Close session locks early with `session_write_close()`.

Logout must clear `$_SESSION`, expire the cookie with matching options, destroy storage, and revoke server-side refresh tokens or recorded sessions where applicable.

2. Input validation

Validation vs sanitization, PHP filters, domain rules, and output encoding boundaries.

Validation checks whether data satisfies rules without changing it. Normalization standardizes legitimate representation. Sanitization removes/changes characters and may silently corrupt meaning. Output encoding belongs at the exact HTML/URL/JS sink.

```
$email = filter_var($rawEmail, FILTER_VALIDATE_EMAIL);
if ($email === false) {
    throw new InvalidArgumentException('Invalid email');
}

$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 18, 'max_range' => 120],
]);
if ($age === false) {
    throw new InvalidArgumentException('Invalid age');
}
```

Use `=== false` because zero can be valid. `FILTER_DEFAULT` is `FILTER_UNSAFE_RAW`; it is not automatic safety.

```
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT, [
    'options' => ['default' => 1, 'min_range' => 1],
]);
```

Shape validation is insufficient: a valid order ID does not prove ownership. Validate type/range, then authorize the current user against the loaded resource. Use allow-lists for enums, sort fields, and identifiers; prepared SQL and context-aware output encoding remain separate controls.

3. CSRF protection

How authenticated browsers are tricked and how tokens, SameSite, and origin checks help.

CSRF tricks an authenticated browser into sending an unwanted state-changing request. Cookies may be attached automatically, so the server sees a valid session without proof of user intent.

```
session_start();
$_SESSION['csrf_token'] ??= bin2hex(random_bytes(32));
```

Place the value in a hidden form field and verify before changing state:

```
$sent = $_POST['csrf_token'] ?? '';

if (!is_string($sent)
    || !isset($_SESSION['csrf_token'])
    || !hash_equals($_SESSION['csrf_token'], $sent)) {
    http_response_code(403);
    exit('Invalid CSRF token');
}
```

Keep tokens out of URLs/logs. Do not mutate state through GET. Add suitable SameSite cookies, Origin checks for sensitive requests, strict accepted content types, and re-authentication/MFA for high-risk actions.

CORS is not CSRF protection, and simple cross-site requests may avoid preflight. XSS can often read tokens or issue same-origin requests, so XSS prevention is essential. Signed double-submit cookies are possible, but prefer framework-reviewed protection. Per-session and per-form tokens are both valid designs; per-request rotation can break concurrent tabs.

4. XSS protection

Reflected, stored, and DOM XSS with context-aware output encoding.

XSS occurs when untrusted data is interpreted as browser code. It may be reflected in one response, stored for later victims, or introduced by DOM code using a dangerous sink.

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

Use this for HTML text and quoted safe attributes. For a URL query value, apply `rawurlencode` first and HTML-encode the final attribute. Encoding does not make a `javascript:` scheme safe; validate protocols and hosts.

Prefer JSON rather than string concatenation for JavaScript data, with `JSON_HEX_TAG` | `JSON_HEX_AMP` | `JSON_HEX_APOS` | `JSON_HEX_QUOT`. In DOM code, prefer `textContent` over `innerHTML`.

If users must author HTML, use a maintained allow-list sanitizer; regex is not an HTML parser. Avoid untrusted data in scripts, styles, comments, tag/attribute names, and event handlers.

Template auto-escaping, CSP, Trusted Types, secure cookies, and correct MIME types provide defense in depth. CSP is not a substitute for encoding and sanitization.

5. Password hashing

Safe storage with `password_hash`, `password_verify`, `Argon2id`, `bcrypt`, and rehashing.

Passwords should be verified, not recovered. Store a slow salted password hash, not reversible encryption or a fast general-purpose hash.

```
$hash = password_hash($password, PASSWORD_DEFAULT);
if ($hash === false) {
    throw new RuntimeException('Hashing failed');
}

if (!password_verify($password, $user['password_hash'])) {
    throw new AuthenticationException('Invalid credentials');
}

if (password_needs_rehash($user['password_hash'], PASSWORD_DEFAULT)) {
    $users->updatePasswordHash(
        $user['id'],
        password_hash($password, PASSWORD_DEFAULT)
    );
}
```

Use a `VARCHAR(255)` column because `PASSWORD_DEFAULT` may evolve. Argon2id is available in compatible builds; benchmark memory/time costs on production-like hardware. PHP embeds salt and options in the result; do not create salts or compare hashes manually.

Allow long passphrases, avoid silent trimming/truncation, rate-limit attempts, add MFA for sensitive accounts, and never log passwords. `bcrypt` historically only uses the first 72 bytes.

A server-side pepper is optional and adds key-management complexity. Keep it in a secret manager. Password-reset tokens must be random, short-lived, one-time, and stored hashed; revoke relevant sessions after a successful reset.

6. Encryption and HTTP authentication

Authenticated encryption, key management, Basic auth, Bearer tokens, and hashing differences.

Use encryption when plaintext must be recovered; passwords use hashing. Prefer authenticated encryption APIs that provide confidentiality and tamper detection.

```
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);  
$cipher = sodium_crypto_secretbox($plainText, $nonce, $key);  
$stored = base64_encode($nonce . $cipher);
```

The key must be securely random with the required length and stored in a secret manager. Base64 is encoding, not encryption. Keep key versions with ciphertext, restrict decrypt permission, audit use, and plan rotation/revocation.

HTTP Basic sends Base64-encoded `username:password` on each request:

```
Authorization: Basic dXNlcjpwYXNz
```

It requires HTTPS and rate limiting. PHP may expose credentials through `PHP_AUTH_USER` and `PHP_AUTH_PW`, depending on server configuration. Basic can suit constrained internal tools but has limited lifecycle/logout controls.

A Bearer token grants access to whoever holds it. Treat it as a secret and design expiry, scope, rotation, and revocation. Not every token is JWT, and JWT is neither encrypted nor automatically revocable.

7. Authentication and authorization

Secure login, RBAC, ownership policies, IDOR prevention, and least privilege.

Authentication asks who the user is; authorization asks whether that identity may perform this action on this resource; auditing records what happened. Login success never grants universal access, and hiding a UI button is not authorization.

A login flow validates input, loads the account, verifies the password, applies rate/risk controls, regenerates the session ID, stores minimal identity state, and redirects. Return generic credential errors and never log passwords or tokens.

RBAC handles broad roles. Ownership and policy checks remain necessary:

```
function canUpdatePost(array $user, array $post): bool
{
    return $user['role'] === 'admin'
        || ($user['role'] === 'editor'
            && $post['author_id'] === $user['id']);
}
```

Where suitable, scope the query itself:

```
SELECT * FROM posts
WHERE id = :post_id AND author_id = :user_id
```

This helps prevent IDOR/BOLA. Middleware may enforce authentication and broad permission, while domain rules must still apply to HTTP, CLI, and queued jobs.

Use 401 for missing/invalid authentication, 403 for authenticated but forbidden, and sometimes 404 to conceal a resource's existence. Deny by default, apply least privilege, use MFA/step-up auth for sensitive actions, revoke sessions after security changes, protect audit logs, and test cross-user resource access.

8. JWT and secure validation

JWT structure, signatures, claims, issuer/audience/time validation, and key management.

A common signed JWT is a JWS with three Base64url segments:

```
BASE64URL(header).BASE64URL(payload).BASE64URL(signature)
```

The signature protects integrity and authenticity when verified with the correct key. Base64url provides no confidentiality: anyone holding the token can read its payload.

Claims to validate

Claim	Meaning
<code>iss</code>	exact trusted issuer
<code>sub</code>	token subject, not necessarily an email
<code>aud</code>	intended recipient API/client
<code>exp</code>	expiration
<code>nbf</code>	not valid before
<code>iat</code>	issued at
<code>jti</code>	unique token identifier

Short-lived tokens limit stale authorization. Roles embedded in a token do not automatically update when the account changes.

Algorithms

`HS256` uses a shared secret, so every verifier with that key can also issue tokens. Asymmetric signatures separate a private signing key from public verification keys. Configure an explicit allowed algorithm; never trust the token's `alg` choice by itself or accept `none` accidentally.

```
composer require firebase/php-jwt
```

```

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$payload = [
    'iss' => 'https://id.example.com',
    'sub' => 'user_123',
    'aud' => 'https://api.example.com',
    'iat' => time(),
    'nbf' => time(),
    'exp' => time() + 300,
    'scope' => 'orders:read',
];

$token = JWT::encode($payload, $privateKey, 'RS256', 'key-2026-09');
$claims = (array) JWT::decode($token, new Key($publicKey, 'RS256'));

if (($claims['iss'] ?? null) !== 'https://id.example.com') {
    throw new RuntimeException('Invalid issuer');
}

if (!in_array('https://api.example.com', (array) ($claims['aud'] ?? []), true)) {
    throw new RuntimeException('Invalid audience');
}

```

The library verifies cryptography and time claims, while the application still owns issuer, audience, token type, and authorization policy checks.

Rotation and JWKS

`kid` is untrusted input. Resolve it only inside a key set already bound to a trusted issuer. Never turn it into an arbitrary filename or follow a token-supplied `jku` URL. Cache JWKS with bounded refresh and retain retiring verification keys until outstanding tokens expire.

Checklist

- Require TLS.
- Allowlist algorithms and expected token type.
- Validate signature, issuer, audience, expiration, and not-before.
- Use a small, explicit clock skew.
- Separate ID-token and access-token validation rules and keys.
- Never log complete tokens.
- Choose short lifetimes, a targeted denylist, or opaque tokens where immediate revocation is required.

9. Federated identity, SSO, SAML, and OIDC

Federation and single sign-on, IdP/SP roles, SAML assertions, and OpenID Connect identity.

Federated identity lets an application rely on an external identity provider. Single Sign-On is the experience of reaching multiple applications after a central sign-in.

```
User -> Application / Service Provider
      -> Identity Provider
      <- signed identity result
      <- local session
```

Separate the protocols

Technology	Primary purpose	Typical result
OAuth 2.0	delegated API authorization	access token
OpenID Connect	authentication layered on OAuth	ID token and identity claims
SAML 2.0	XML identity/assertion exchange, common in enterprise SSO	SAML response/assertion

OAuth alone is not a login protocol. Inferring identity from an arbitrary access token endpoint creates unsafe pseudo-authentication; use OIDC.

SAML browser SSO

The Identity Provider authenticates the user and issues an assertion. The Service Provider validates it and creates its own session.

```
Browser -> SP: protected page
SP -> Browser -> IdP: AuthnRequest
IdP authenticates user
IdP -> Browser -> SP ACS: SAMLResponse
SP validates response/assertion and creates a session
```

Validate the XML signature with trusted IdP metadata, issuer, audience restriction, recipient/destination, time conditions, and `InResponseTo` where applicable. Detect replay by assertion ID. Use a mature SAML library and safe XML parsing; do not implement XML signatures yourself.

OpenID Connect

The `openid` scope makes an OAuth authorization request an OIDC request:

```
scope=openid profile email
```

- ID token: authentication result for the client.
- Access token: authorization credential for an API.
- UserInfo: optional additional claims obtained with an access token.

Never substitute an ID token as the API's access token.

Validate an ID token

1. Use metadata rooted in a preconfigured trusted issuer.
2. Verify the signature with that issuer's JWKS.
3. Exactly match `iss` and require the client ID in `aud`.
4. Check expiration and relevant time claims.
5. Match the transaction-specific `nonce`.
6. Apply `azp` rules for multiple audiences.
7. Require UserInfo `sub` to match the ID-token subject.

An email can change and may be unverified. Use the pair `iss + sub` as the stable external identity key.

Choose OIDC for modern web/mobile login, SAML where enterprise federation requires it, and OAuth for delegated API access. Security depends on exact validation and trust configuration, not the protocol name alone.

10. OAuth 2.0 concepts and tokens

Delegated authorization, roles, scopes, consent, client types, access tokens, and refresh tokens.

OAuth 2.0 is a delegated authorization framework. It lets a client obtain limited access to a resource server without receiving the resource owner's password.

Four roles

1. Resource Owner: can grant access, often a user.
2. Client: application requesting access.
3. Authorization Server: issues tokens after authorization.
4. Resource Server: protected API accepting access tokens.

```
Resource Owner -> Authorization Server: approve  
Client -> Authorization Server: grant/code  
Authorization Server -> Client: access token  
Client -> Resource Server: token + API request
```

Client registration and types

Registration commonly defines a public `client_id`, exact redirect URIs, allowed grants, and authentication methods.

- Public clients cannot keep a secret, including browser JavaScript and native apps.
- Confidential clients run on a backend capable of protecting credentials.

Putting a client secret into a distributed application does not make it confidential.

Scope and consent

Scopes describe requested capabilities:

```
orders:read orders:write profile
```

Use least privilege and clear capability names. Consent is the resource owner's approval experience, not a replacement for authorization-server policy.

Access tokens

An access token is a credential presented to a resource server. It can be opaque or a JWT; OAuth does not require one format. Restrict it by lifetime, audience/resource, and scope. An API must enforce both intended audience and permitted action.

Refresh tokens

A refresh token goes only to the authorization server's token endpoint to obtain a new access token. It never goes to the resource API.

```
Client --refresh token--> Authorization Server
Client <--new access token (+ rotated refresh token)--
```

For public clients, use rotation with reuse detection or sender-constraining as appropriate.

API keys differ

An API key often identifies a project or caller for quota and service access. It does not automatically model a user, consent, or dynamic scopes. Protect it as a secret, but do not call it user authentication.

```
{
  "access_token": "opaque-or-structured-value",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_token": "long-lived-secret",
  "scope": "orders:read"
}
```

Protect token responses with TLS and `Cache-Control: no-store`; never log them.

OAuth is not authentication, JWT is only a token format, a client ID is not secret, and token validity never replaces object ownership and domain-policy checks.

11. Modern OAuth flows

Authorization Code with PKCE, Client Credentials, Device Authorization, refresh tokens, and obsolete grants.

```
User + web/browser/native -> Authorization Code + PKCE
Service to service        -> Client Credentials
TV/CLI without browser    -> Device Authorization
Renew previous access     -> Refresh Token
```

Authorization Code + PKCE

Browser	Client	Authorization Server
--- start --->	-- authorize + challenge	
<----- redirect to login -----		
----- login/consent ----->		
<---- redirect: code + state -----		
-- callback -->	-- code + verifier ----->	
	<-- access/refresh token--	

```
function base64Url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

$codeVerifier = base64Url(random_bytes(32));
$codeChallenge = base64Url(hash('sha256', $codeVerifier, true));
$state = base64Url(random_bytes(32));
$nonce = base64Url(random_bytes(32));

$_SESSION['oauth'] = [
    'state' => hash('sha256', $state),
    'verifier' => $codeVerifier,
    'nonce' => $nonce,
    'created_at' => time(),
];
```

Send `response_type=code`, exact redirect URI, scopes, state, OIDC nonce, code challenge, and `code_challenge_method=S256`. At callback, consume the bound state once, then exchange the code with the verifier through the token endpoint.

PKCE binds a code to the client instance that started the transaction. Use a new high-entropy verifier and S256 for every attempt.

Client Credentials

Use this for a confidential service acting as itself:

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Authorization: Basic BASE64(client_id:client_secret)

grant_type=client_credentials&scope=reports:write
```

There is no end user. Prefer independent workload identities and consider mTLS or `private_key_jwt` for stronger client authentication.

Device Authorization

A constrained device obtains a secret `device_code`, a human `user_code`, and a verification URI. The user approves on a second device while the original device polls.

Respect the returned polling interval. Increase it by five seconds on `slow_down`, and stop on denial or expiration.

Refresh Token

Send refresh tokens only to the token endpoint and never request broader scope. Rotation replaces each used refresh token and detects reuse of an older member of the token family.

Assertion grants

RFC 7522 can exchange a SAML bearer assertion as an OAuth grant or use it for client authentication. This is a federation bridge, not the default SAML browser SSO flow. Validate signature, issuer, audience, time, and replay.

Do not start new systems with these

- The Implicit Grant exposes access tokens through the front channel; use Authorization Code + PKCE.
- The Resource Owner Password Credentials grant exposes user passwords to clients and is incompatible with modern MFA/WebAuthn. The OAuth Security BCP says it must not be used.

12. OAuth and OIDC security

State, nonce, PKCE, redirect URIs, browser token storage, BFF, revocation, and monitoring.

Control	Primary purpose
session-bound <code>state</code>	callback CSRF and transaction swapping
OIDC <code>nonce</code>	authentication-response replay/substitution
PKCE	authorization-code theft or injection
exact redirect URI	redirecting results to an attacker

Use transaction-specific values and consume them once.

Callback skeleton

```
session_start();

$tx = $_SESSION['oauth'] ?? null;
unset($_SESSION['oauth']);

if (!is_array($tx) || time() - $tx['created_at'] > 600) {
    throw new RuntimeException('OAuth transaction expired');
}

$returnedState = $_GET['state'] ?? '';
if (
    !is_string($returnedState)
    || !hash_equals($tx['state'], hash('sha256', $returnedState))
) {
    throw new RuntimeException('Invalid state');
}

if (isset($_GET['error'])) {
    throw new RuntimeException('Authorization was not completed');
}

$code = $_GET['code'] ?? null;
if (!is_string($code) || $code === '') {
    throw new RuntimeException('Missing authorization code');
}
```

Do not log the callback query. Use `Referrer-Policy: no-referrer` and avoid third-party content on callback pages.

Redirect safety

Register and exactly match complete HTTPS redirect URIs. Avoid production wildcards and open redirects. Store post-login destinations server-side or restrict them to internal paths. Bind the transaction to the expected issuer when one client supports multiple authorization servers.

Browser architectures

A Backend for Frontend keeps tokens server-side:

```
Browser --Secure HttpOnly session cookie--> BFF
BFF --access token--> APIs
```

Protect the cookie session against CSRF and fixation. In browser-only clients, memory limits persistence; long-lived `localStorage` tokens are available to any JavaScript executing in that origin. Never ship a client secret to a SPA.

Keep tokens out of URLs, history, logs, and analytics.

Lifecycle

- Keep access tokens short-lived and resource/audience restricted.
- Store refresh tokens with stronger protection.
- Rotate refresh tokens and detect reuse.
- Revoke the token family on compromise, logout, or security changes.
- Use revocation or introspection where immediate control is required.
- Consider mTLS or DPOP for sender-constrained tokens in higher-risk systems.

OIDC checklist

1. Start discovery from a preconfigured trusted issuer.
2. Use Authorization Code + PKCE S256.
3. Bind state/PKCE and nonce to one browser transaction.
4. Require an exact registered redirect URI.
5. Validate signature, issuer, audience, expiration, and nonce.
6. Link accounts by `iss + sub`.
7. Do not turn email/domain claims into local privileges without an explicit policy.
8. Create a fresh local session and rotate its identifier.

Log issuer, client ID, flow, outcome, error code, and correlation ID, but never tokens, codes, verifiers, secrets, or assertions. Monitor invalid grants, refresh-token reuse, issuer/redirect mismatch, signature/audience failures, unusual scopes, and device-flow polling.

13. Account lifecycle

Registration, verification, password recovery, identity changes, reauthentication, and session revocation.

Registration and enumeration

Normalize identifiers according to a documented policy, validate product rules, rate-limit abuse, hash passwords, and delay sensitive privileges until required contact details are verified. Use similar public responses whether an account exists while keeping useful internal security logs.

Verification token

```
$token = bin2hex(random_bytes(32));  
$tokenHash = hash('sha256', $token);  
// Store hash + user_id + expires_at + used_at.  
// Send the raw token once over an HTTPS link.
```

Hash the received value, verify expiry and one-time status, then consume it inside a transaction. Do not store raw bearer tokens.

Password recovery

Use a short-lived one-time random token, a generic public response, and no guessable security questions. After success, rotate session identifiers and revoke other sessions or refresh tokens according to policy. Send a security notification.

Sensitive changes

Require recent authentication or MFA before changing password, email, recovery factors, or performing high-risk actions. Track `auth_time`; possession of an old session is not sufficient.

Verify a new email, notify the old address, and enforce uniqueness in the database. Define suspension, deactivation, deletion, export, and retention explicitly. Test races such as double-clicked links and concurrent resets.

14. MFA, TOTP, and passkeys

Authentication factors, TOTP, recovery codes, WebAuthn/passkeys, step-up, and safe recovery.

Factors and TOTP

Factors are something known, possessed, or inherent/local user verification. Two steps from the same category are not strong MFA. SMS may improve password-only login but is exposed to phishing and SIM swap.

For TOTP, generate a random secret, prove setup with a code before enabling it, encrypt the secret under a managed key, use a small clock window, rate-limit checks, and avoid logging codes or secrets.

Recovery codes

Generate random one-time codes, show them once, store strong hashes, consume each atomically, notify the user, and require reauthentication to regenerate the set.

WebAuthn and passkeys

During registration the server sends a challenge and stores the resulting credential ID and public key. During authentication it verifies a fresh challenge, signature, origin, RP ID, user-verification policy, and relevant authenticator metadata. The server never stores the private key.

Passkeys resist phishing because credentials are scoped to the relying party and origin.

Step-up and recovery

Ask for stronger assurance before changing recovery factors, high-value payments, or secret access—not every click. Bind the result to a short-lived action context.

Recovery must not silently bypass MFA. Use risk-appropriate evidence, delays and notifications for sensitive changes, audited support procedures, and explicit revocation of lost credentials.

Reference

- [OWASP Authentication Cheat Sheet](#)

15. API keys and machine identity

Issuing, storing, scoping, rotating, and revoking API keys, service accounts, and mTLS.

A key is not a user identity

An API key usually identifies an application or integration. Bind it to a clear service/client principal with owner, purpose, environment, and least privilege.

Use a recognizable public prefix/identifier plus a random secret. Look up by identifier and store only a hash of the secret. Display the secret once.

Lifecycle

- Small deny-by-default scopes.
- Expiry where practical.
- Safe last-used metadata.
- Rotation with a short overlap.
- Immediate revocation.
- Separate keys per system and environment.

Send keys in an authorization or dedicated header over TLS, never in a query string. Apply per-key and per-tenant quotas. Treat IP restrictions only as an additional layer.

Service accounts and mTLS

Machine accounts should have no interactive login unless required, minimum privileges, and attributable audit events. Prefer short-lived workload identity over static key files when available.

Mutual TLS authenticates both endpoints but still requires certificate lifecycle management and operation-level authorization.

16. ABAC, ReBAC, and tenant isolation

Attribute and relationship policies, multi-tenant boundaries, deny-by-default decisions, and testing.

Beyond RBAC

RBAC uses roles, ABAC uses subject/resource/context attributes, and ReBAC uses relationships such as owner, member, or manager. Adopt complexity only when it represents real policy.

Centralize decisions in policy objects or an authorization service. A UI may hide a button, but the server must authorize every HTTP, CLI, and queue path.

Tenant isolation

Derive tenant context from trusted identity, not only request data. Enforce it in queries, cache keys, object paths, queue messages, search indexes, exports, and administrative tooling.

Shared database, schema-per-tenant, and database-per-tenant models have different isolation and operational trade-offs; none is secure automatically.

Deny and test

Deny unknown actions by default. Define allow/deny precedence and record policy version plus an internal decision reason without exposing sensitive details.

Test same-tenant owners, unrelated users, cross-tenant ID collisions, privileged roles, missing resources, and every background path. Explicitly test IDOR by swapping identifiers.

Reference

- [OWASP Authorization Cheat Sheet](#)

17. Rate limiting and abuse resistance

Fixed/sliding windows, token buckets, login defenses, credential stuffing, CAPTCHA, and failure policy.

Choose a key per operation

IP alone is insufficient because users share NAT and attackers distribute traffic. Combine signals: account and network for login, contact and network for reset, key/tenant/endpoint for APIs, and user plus cost for expensive searches. Keep a global capacity limit.

Algorithms

Fixed windows are simple but burst at boundaries. Sliding windows are more precise and expensive. Token buckets allow controlled bursts; leaky buckets smooth output. Updates must be atomic in shared storage across servers.

Return `429 Too Many Requests` and an appropriate `Retry-After` without revealing account existence.

Login abuse

Avoid permanent lockouts that let attackers deny access to victims. Use progressive delay, layered limits, breach-aware password policy, MFA/passkeys, and user notification for unusual activity.

CAPTCHA adds friction but is not complete proof of humanity. Trigger it from risk signals and account for accessibility and privacy.

Define fail-open or fail-closed behavior when the limiter store is unavailable. A privileged login and a public read endpoint need different policy.

18. Security logging and audit trails

Security events, structured schemas, redaction, tamper resistance, alerting, and investigation.

Different records

Operational logs support debugging, security events support detection, and an audit trail records who did what, when, and to which resource with stronger integrity and retention requirements.

Use a structured schema containing UTC time, event name, request ID, actor, tenant, action, resource, result, and a stable reason code. Prefer identifiers over unnecessary personal values and version the schema.

Events and exclusions

Record login/MFA/recovery outcomes, identity and role changes, key lifecycle, authorization denial, administrative activity, exports/deletion, secret access, and policy changes.

Never record passwords, raw tokens, session IDs, API secrets, or recovery codes.

Integrity and detection

Send records to centralized restricted storage, separate write and deletion authority, define retention and backup, synchronize clocks, and detect ingestion failure or tampering.

Alert on patterns such as distributed failures, privilege escalation, a new key followed by a large export, or unusual recovery. Give every alert an owner and runbook, and preserve request/trace IDs for investigation.

Reference

- [OWASP Logging Cheat Sheet](#)

19. Secrets and key management

Secret managers, envelope encryption, rotation, revocation, access policy, break-glass, and incident response.

Identify the secret

Database passwords, API keys, private keys, encryption keys, and signing secrets are sensitive. Public environment names and URLs usually are not. A `.env` file is a development loading mechanism, not a secret manager, and Base64 is not encryption.

Lifecycle

```
generate -> store -> distribute -> use -> rotate -> revoke -> destroy
```

Every secret needs an owner, purpose, consumers, lifetime, and rotation policy. Avoid sharing one credential across many services.

Secret managers and envelope encryption

Let the workload authenticate with least privilege and prefer short-lived or dynamic credentials. Cache only briefly in memory and never write values to logs.

With envelope encryption, a KMS/HSM master key protects a data-encryption key, while the DEK encrypts application data. Store ciphertext, encrypted DEK, algorithm, and key version. Use authenticated encryption with correct nonces and contextual associated data.

Rotation

Store a key ID/version. Create a new key, write new data with it, keep old versions readable, re-encrypt where needed, then revoke after a safe window. Destroying an encryption key may permanently destroy access to data.

Access and incident response

Apply least privilege, separate administration from runtime, audit access and changes, provide controlled break-glass procedures, alert on unusual access, and test encrypted backup recovery.

After exposure, restrict use, rotate or revoke, determine scope from audit records, fix the source, and inspect history, artifacts, and logs. Removing a value from Git does not revoke it.

06

TRACK · 5 LESSONS

Databases and PDO

Progressive explanations, practical examples, and corrections that take each idea into real code.

Databases & PDO

Secure connections, prepared statements, ORM, transactions, ACID, isolation, locks, and indexes.

This section turns pages **67–70** into a complete path from PDO connection to robust transactional design.

1. [PDO and prepared statements](#)
2. [ORM and data-access patterns](#)
3. [Transactions and ACID](#)
4. [Isolation, locking, and deadlocks](#)
5. [Schema, indexes, and migrations](#)

Learn SQL and PDO before relying on an ORM: abstractions do not remove the need to understand queries, indexes, and transactions.

1. PDO and prepared statements

PDO connections, error handling, prepared statements, fetching, and injection prevention.

PDO provides one interface for several database drivers, but SQL and capabilities remain driver-specific.

```
$pdo = new PDO(
    'mysql:host=127.0.0.1;dbname=shop;charset=utf8mb4',
    getenv('DB_USER'),
    getenv('DB_PASSWORD'),
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

Keep credentials out of Git, specify charset in the DSN, and never expose exception details to users.

```
$stmt = $pdo->prepare(
    'SELECT id, name FROM users
    WHERE email = :email AND status = :status'
);
$stmt->execute(['email' => $email, 'status' => 'active']);
$user = $stmt->fetch();
```

Placeholders separate values from SQL structure. They cannot represent table/column names, keywords, sort direction, or a whole `IN` list. Allow-list identifiers and create one placeholder per list value.

`fetch()` returns one row or false; `fetchAll()` may use substantial memory; `fetchColumn()` returns one column. `rowCount()` behavior for SELECT is driver-specific. Stream large results and select only needed columns.

`bindValue` binds the current value; `bindParam` binds a variable by reference. Passing an array to `execute` is often simplest. Log database details securely while returning a generic error and request ID.

2. ORM and data-access patterns

Active Record, Data Mapper, relationships, N+1 queries, and abstraction limits.

An ORM maps rows and relationships to objects. It reduces repetitive SQL but does not remove query cost or database semantics.

Active Record combines row state and persistence; Laravel Eloquent is a PHP example. It is convenient for CRUD but can couple domain and storage concerns.

Data Mapper keeps entities closer to domain logic while a mapper/repository handles persistence; Doctrine is a PHP example. It offers stronger separation with more concepts such as Unit of Work and Identity Map. Django is a Python comparison, not a PHP package.

Model one-to-one, one-to-many, and many-to-many relationships while preserving database foreign-key constraints. Configure cascades deliberately.

N+1 occurs when loading a relation triggers one query per parent. Use eager loading or a tailored join/select, but measure because eager-loading everything can over-fetch.

Never mass-assign raw `$_POST`; allow-list validated fields so attackers cannot set `is_admin`. Do not serialize entities containing hashes/secrets.

Use direct SQL or a query builder for complex reports, bulk operations, database-specific features, and measured hot paths. ORM, query builder, and PDO can coexist.

3. Transactions and ACID

Auto-commit, commit, rollback, ACID, and safe multi-step operations.

A transaction prevents partially completed multi-step changes. Under normal auto-commit, each statement is independent until an explicit transaction begins.

ACID means atomicity (all or rollback), consistency (constraints/rules remain valid), isolation (concurrent work behaves according to the selected level), and durability (committed data survives under system guarantees).

```
$pdo->beginTransaction();

try {
    $debit->execute(['amount' => $amount, 'id' => $from]);
    $credit->execute(['amount' => $amount, 'id' => $to]);
    $pdo->commit();
} catch (Throwable $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    throw $e;
}
```

For balance transfer, lock/check both accounts consistently and update both before commit. Exact locking syntax is database-specific. Keep transactions short; validate first and avoid waiting for external APIs or email while holding locks.

Database rollback cannot undo an already-sent email. Use a transactional outbox: store an event in the same transaction and publish it later.

Some databases implicitly commit DDL such as `CREATE TABLE` or `DROP TABLE`; never assume all schema changes roll back. Constraints remain necessary even inside transactions.

Oracle deserves the same warning: DML transactions provide ACID semantics, while DDL normally issues implicit commits before and after the statement. Do not mix schema changes into a business transaction expecting one rollback boundary.

4. Isolation, locking, and deadlocks

Concurrency anomalies, isolation levels, pessimistic/optimistic locking, retries, and savepoints.

Transactions can overlap; isolation controls what each sees. Anomalies include dirty reads, non-repeatable reads, phantoms, lost updates, and write skew.

Levels commonly range from Read Uncommitted through Read Committed and Repeatable Read to Serializable. Exact MVCC and locking behavior differs across MySQL, PostgreSQL, and other systems; verify your database rather than relying on a generic table.

Pessimistic locking uses database-specific syntax such as `SELECT ... FOR UPDATE`. Good indexes keep the lock scope focused.

Optimistic locking updates only when a version matches:

```
UPDATE products
SET stock = :new_stock, version = version + 1
WHERE id = :id AND version = :old_version
```

Zero affected rows indicates a conflict.

Deadlocks are expected concurrency outcomes. Access resources in consistent order, keep transactions short, index queries, and use bounded retry with backoff/jitter for retryable errors. Make the operation idempotent or use an idempotency key.

PDO has no portable true nested transactions. Some databases support savepoints, but rolling back to one does not finish the outer transaction. Choose isolation from business invariants and combine it with constraints, atomic updates, and deliberate locking.

5. Schema, indexes, and migrations

Types, constraints, relationships, composite indexes, query plans, pagination, and safe schema change.

Protect invariants in the database as well as application code. Use `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys, and appropriate types. Store money as integer minor units or an appropriate decimal type, not binary float; define a timezone policy.

Indexes speed lookup, join, and ordering at the cost of storage and write work:

```
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at);
```

Column order matters. Indexes must be chosen for measured query patterns, not added to every column.

Use `EXPLAIN` and, when supported, actual-analysis plans. Test production-like volumes and monitor slow queries and latency percentiles. Watch for N+1, unnecessary `SELECT *`, deep OFFSET pagination, casts/functions blocking indexes, and excessive round trips.

Keyset pagination often scales better:

```
SELECT *
FROM orders
WHERE id < :last_seen_id
ORDER BY id DESC
LIMIT 20
```

Migrations should be versioned, reviewed, automated, and tested with realistic data. Back up and test restoration, monitor locks, separate heavy backfills, and do not assume DDL rollback. Expand/contract deployments add compatible structure, migrate readers/writers and data, then remove old structure later.

Keep schema migrations distinct from development/reference seeds, make seeds idempotent, and never copy production personal data into fixtures.